多核缓存层次结构

Multi-Core Cache Hierarchies

Rajeev Balasubramonian

Norman P. Jouppi

Naveen Muralimanohar

中文修订学习版

根据 Morgan & Claypool Publishers 于 2011 年出版的英文原著整理；非官方授权译本。

译稿说明

本书保留 2011 年原著的历史语境、原作者信息及文献引用顺序。修订主要恢复章节层级和遗漏图表，清除原版页眉页码等识别杂质，补译表格及说明文字，并统一中文科技图书版式。参考文献中的英文原始篇名为便于检索而保留，原版插图内的英文技术标签也暂予保留。

版权信息

原书书名：Multi-Core Cache Hierarchies

作者：Rajeev Balasubramonian、Norman P. Jouppi、Naveen Muralimanohar

出版商：Morgan & Claypool Publishers

丛书：Synthesis Lectures on Computer Architecture（计算机体系结构综合讲座，第 17 讲）

丛书编辑：Mark D. Hill（威斯康星大学）

出版年份：2011

ISBN：9781598297539（平装）、9781598297546（电子书）

DOI：10.2200/S00365ED1V01Y201105CAC017

原著版权归 Morgan & Claypool Publishers 所有。本中文修订学习稿并非出版社的授权译本。

摘要

缓存层次结构是决定系统整体性能与功耗的关键因素之一，因为访问片外内存所需的时钟周期和能量远高于片上访问。此外，多核处理器预计会对内存系统提出越来越高的带宽需求。因此，提高片上缓存效率、尽量避免片外内存访问显得尤其重要。未来的多核处理器将包含多个通过互连网络相连、由多个核心共享的大型缓存存储体。这也提出了几个关键问题：如何在核心间分配缓存资源，如何将数据放置在靠近访问核心的缓存存储体中，以及如何识别值得长期保留的重要数据。此外，现有工艺技术的微缩越来越困难，促使研究者适应并利用新的技术约束。

本书综合梳理了近年来针对多核处理器的缓存研究成果，重点关注体系结构方面的创新。它为希望了解近期缓存研究整体面貌的研究生、科研人员和工程实践者提供入门资料，也可作为高阶计算机体系结构课程以及 VLSI 工程师的参考书。

关键词：计算机体系结构；多核处理器；缓存层次结构；共享与私有缓存；非均匀缓存访问（NUCA）；服务质量；缓存划分；替换策略；内存预取；片上网络；存储单元。

题献

献给始终给予我们支持的家人和同事。

目录

译稿说明 ii

版权信息 iii

摘要 iv

题献 v

前言 viii

致谢 xi

第一章 大型缓存设计的基本要素 1

1.1 共享缓存与私有缓存 . . . 1

1.1.1 共享 LLC . . . 1

1.1.2 私有 LLC . . . 3

1.1.3 工作负载分析 . . . 6

1.2 集中式与分布式共享缓存 . . . 7

1.3 非均匀缓存访问 . . . 8

1.4 包含性 . . . 12

第二章 在 CMP 末级缓存中组织数据 14

2.1 大型共享 NUCA 缓存的数据管理 . . . 14

2.1.1 D-NUCA 的放置/迁移/搜索策略 . . . 14

2.1.2 共享缓存中的复制策略 . . . 20

2.1.3 基于操作系统的页面放置 . . . 23

2.2 一组私有缓存的数据管理 . . . 31

2.3 讨论 . . . 35

第三章 影响缓存命中率的策略 36

3.1 面向吞吐量与服务质量的高速缓存划分 . . . 36

3.1.1 引言 . . . 36

3.1.2 吞吐量 . . . 38

3.1.3 QoS 策略 . . . 45

3.2 为大型共享缓存选择高有用性的群体 . . . 48

3.2.1 替换/插入策略 . . . 48

3.2.2 关联性的新颖组织 . . . 55

3.2.3 块级优化 . . . 57

3.3 小结 . . . 64

第四章 大型缓存内的互连网络 66

4.1 基本大型缓存设计 . . . 66

4.1.1 缓存阵列设计 . . . 66

4.1.2 缓存互连 . . . 67

4.1.3 分组交换路由网络 . . . 68

4.2 互连设计对 NUCA 和 UCA 缓存的影响 . . . 74

4.2.1 NUCA 缓存 74
4.2.2 UCA 缓存 76
4.3 面向大型缓存的创新网络架构 79

第五章 存储技术 **86**
5.1 静态 RAM 的局限性 86
5.2 参数变化 87
5.2.1 建模方法 88
5.2.2 缓解工艺偏差的影响 88
5.3 容忍硬错误和软错误 90
5.4 利用 3D 堆叠解决 SRAM 问题 92
5.5 新兴技术 93
5.5.1 3T1D RAM 95
5.5.2 嵌入式 DRAM 96
5.5.3 非易失性存储器 96

第六章 结束语 **99**

参考文献 **100**

作者简介 **115**

前言

多核革命方兴未艾。首批主流多核处理器大约出现在 2005 年。如今，几乎买不到只含单个核心的台式机或笔记本电脑。趋势显而易见：芯片上的核心数量很可能每两到三年翻一番。这类处理器芯片将广泛应用于高性能计算领域：超级计算机、服务器和高端台式机。正如低端设备（如智能手机）的销量预计会增长一样，高端设备（数据中心服务器）的销量预计也会增长。后一趋势很可能是因为用户将越来越依赖“云”进行数据存储和计算。

几十年来，整体系统性能的关键决定因素之一一直是存储层次。访问片外存储器会消耗许多周期和大量能量。处理器芯片的缓存能容纳并找到的数据越多，性能和能效就越高。在多核时代，这一点依然成立。事实上，多核处理器预计会给存储系统带来更大的压力：芯片上的引脚数量预计将基本保持不变，而需要供给数据的核心数量预计将急剧上升。这使得最小化片外访问变得更加重要。

存储层次的效率在很大程度上取决于片上缓存的访问延迟及其命中率。未来的末级缓存（LLC）预计将占据处理器芯片裸片面积的一半，并容纳许多兆字节的数据。LLC 很可能由散布在芯片各处的许多存储体组成。访问数据将需要在长导线上导航，并穿过多个路由元件。因此，每次访问都将需要数十个周期的延迟和数纳焦耳的能量，具体取决于必须传输的距离。缓存资源将不得不在线程之间分配，LLC 的一部分可能对某个线程私有，而其他部分可能由多个线程共享。

因此，由于多核平台和工作负载带来的新挑战，缓存技术在未来几年将经历演变。缓存策略现在必须考虑线程间的干扰，以及缓存存储体与核心之间数据传输的大且非均匀的延迟和能量。片上非局部导线的缩放性持续不佳，增加了互连在缓存访问中的作用。因此，我们迫切需要（i）设计减少长距离通信的缓存策略，以及（ii）创建低开销网络，以便在需要时更好地处理长距离通信。一些新的技术现象也将在缓存内部要求创新。这些现象包括参数变化的出现、硬错误和软错误率、缓存中的漏电能量，以及 3D 堆叠带来的热约束。

考虑以下多核对缓存策略产生颠覆性影响的例子。在多年依赖类 LRU 策略进行缓存替换之后，近年来出现了多篇论文，

表明其他方法在多核 LLC 中的替换效果要好得多。同样，过去十年中出现了许多论文，考虑私有和共享 LLC 的变体，试图结合两者的优点。因此，过去和未来的几十年是缓存研究的激动人心的时期。回顾过去，多核处理器即将到来本应是显而易见的；20 世纪 90 年代的许多论文已经指出了这一趋势。然而，总体而言，学术界在拥抱多核研究方面稍显迟缓。结果，多核研究的步伐仅在首批商用多核处理器问世后才加速。

仍有许多工作要做，尤其是未来众核处理器的存储层次。

本书结构

本书旨在综合近年来专注于多核处理器创新的缓存研究。对于任何希望了解近期缓存工作全景的研究人员或从业者，我们希望本书能成为理想的起点。我们也期望早期研究生能从这样的综合讲座中受益。本书还应作为高级计算机体系结构课程的良好参考书。我们期望这里的材料对计算机科学家和 VLSI 工程师都易于理解。本书并非旨在替代阅读相关完整论文和追溯更早的参考文献。本书有望提高读者对众多缓存主题的广度和意识，同时使特定主题的研究更加高效。

鉴于存储层次主题的广阔性，我们必须为本书值得收录的内容设定一些参数。我们主要关注与多核使用有强烈关联的近期工作（2004 年及以后）。我们将覆盖范围集中在架构研究的四个主要会议之一上发表的论文：ISCA、MICRO、ASPLOS 和 HPCA。然而，本书也讨论了一些在其他场合发表并在社区内产生明显影响的论文。尽管我们尽了最大努力，肯定遗漏了一些值得提及的论文；我们希望能在后续版本中纠正一些疏忽。我们鼓励读者联系我们指出遗漏之处。

多核缓存领域与计算机体系结构内的其他几个领域有很强的重叠。我们明确将其中一些领域排除在本书之外，因为它们已被其他综合讲座涵盖：

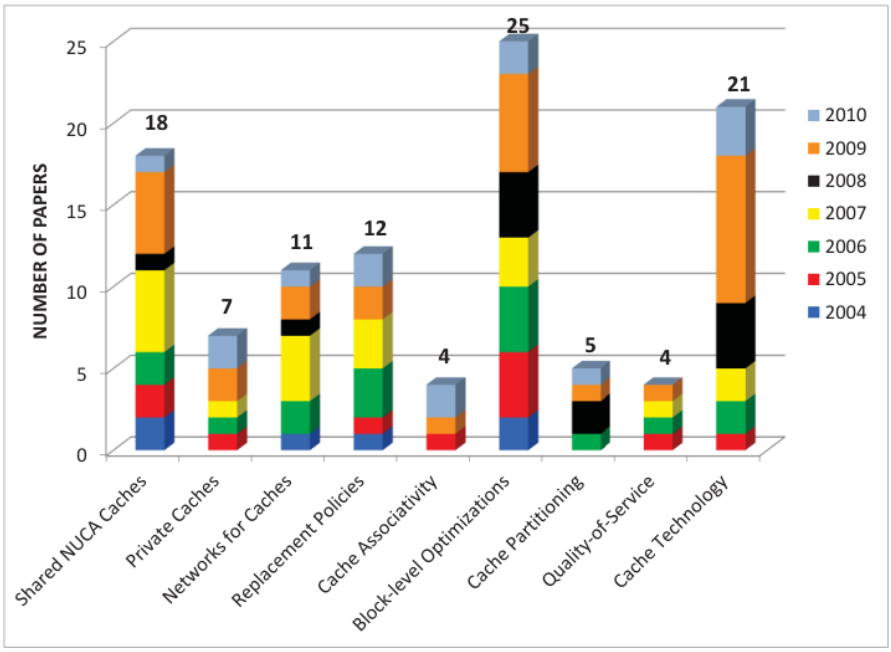
- 片外存储系统 [10]；
- 片上网络设计 [11]；
- 核心存储组件（加载／存储队列、L1 缓存）[12]；
- 缓存一致性与一致性模型 [13]；
- 相变存储器 [14]；
- 功耗优化 [15]。

本书的讨论试图突出论文中的关键思想。我们力求传达每篇论文的新颖性和定性贡献。我们通常没有总结每个想法的定量改进。我们意识到，引用论文中的具体数字可能会产生误导，因为每篇论文采用了不同的基准测试和模拟基础设施参数。

第二章提供了多核缓存层次结构的背景和分类。第三章则探讨了弥合共享 LLC 与私有 LLC 之间差距的策略。第二章中的论文通常假设 LLC 由一组 bank 组成，访问每个 bank 的延迟各不相同。所考虑的策略试图将数据放置在这些 bank 中，以便访问延迟最小且命中率最大化。第三章也关注命中率优化，但针对的是单个缓存 bank。所考虑的策略不是在 bank 之间移动数据，而是通过更好的替换策略、更好的相联组织以及块级优化（预取、死块预测、压缩等）来提高命中率。该章还探讨了如何在多个线程之间划分单个缓存 bank 以实现高吞吐量和服务质量。由于访问 LLC 通常需要经过片

上网络，第四章描述了与缓存策略有强烈交互的片上网络创新。第五章描述了现代技术趋势可能如何影响未来缓存的设计。它涵盖了现代技术现象，如 3D 芯片堆叠、参数变化、错误率上升以及新兴的非易失性存储器。第六章总结了对未来工作方向的思考。

原书前言的图 1 采用与上述相同的分类，并显示了过去七年在四大体系结构会议上每个缓存主题下出现的论文数量。请注意，论文主题可能有多种分类方式，数据应被视为近似值。该数据可作为多核缓存中热门话题的指标。活动似乎最活跃的是技术现象、块预取和共享缓存。



前言图 1：2004—2010 年间 ISCA、MICRO、ASPLOS 和 HPCA 四大会议中各类缓存研究论文的数量。横轴依次为共享缓存、私有缓存、缓存网络、替换策略、相联组织、缓存块级优化、缓存划分、服务质量和缓存技术；不同颜色表示不同年份。

Rajeev Balasubramonian、Norman P. Jouppi、Naveen Muralimanohar　2011 年 5 月

致谢

我们感谢所有对本综合讲座早期草稿提供评论和反馈的人，特别是 Mark Hill、Aamer Jaleel、Gabriel Loh、Mike Morgan 以及犹他大学体系结构研究组的学生们。

Rajeev Balasubramonian、Norman P. Jouppi、Naveen Muralimanohar　2011 年 5 月

第一章　大型缓存设计的基本要素

本章介绍了研究和开发中常用的缓存层次结构实现的概况，并确定了它们的关键区别特征。这些特征包括：共享与私有、集中与分布、均匀与非均匀访问。关于什么构成最优的缓存层次结构实现，业界几乎没有共识。缓存层次结构的某些层级采用私有和均匀访问缓存，而其他层级则采用共享和非均匀访问。我们将指出选择每个特征的优缺点，研究工作中为其基线实现选择任何特征组合都是完全合理的。本书的很多焦点在于片上末级缓存（LLC）的设计。过去，大多数片上缓存层次结构由两级（L1 和 L2）组成，但在缓存层次结构中纳入三级（L1、L2 和 L3 LLC）正变得越来越普遍。正如我们在本章和下一章所解释的，未来的 LLC 如果实现为共享缓存，则更有可能优化缺失率、延迟和复杂性。本章还讨论了理解现代缓存创新所需的其他基础知识。

在开始之前，有必要澄清几个术语。在缓存层次结构中，靠近处理器的缓存层级被视为“上层”缓存，而靠近主存的缓存层级被视为“下层”缓存。我们还将交替使用术语“缓存行”和“缓存块”，两者都旨在表示缓存取指和替换过程中处理的最小数据单元。

1.1　共享缓存与私有缓存

大多数现代高性能处理器在单个芯片内包含多个缓存层次结构层级。在多核处理器中，每个核通常有自己的私有 L1 数据缓存和 L1 指令缓存。考虑到每个核几乎每个周期都必须访问 L1 缓存，因此通常不会让多个核共享单个 L1 缓存（无论是数据缓存还是指令缓存）。L1 缓存中的缺失会发起对 L2 缓存的请求。在大多数讨论中，我们将假设 L2 是 LLC。但同样的论点也适用于三级层次结构中的 L3 LLC，其中 L1 和 L2 是每个核私有的。我们首先比较共享和私有 LLC 的特性。

1.1.1　共享 LLC

单个大型 L2 LLC 可能由芯片上的多个核共享。由于来自核的请求会被其 L1 缓存过滤，因此单端口 L2 缓存有可能满足多个核的需求。图 1.1 展示了一种示例组织。在该设计中，八个核

共享单个大型 L2 缓存。当一个核在其 L1 缓存中找不到数据时，它会将请求放置在所有核共享的总线上。L2 缓存控制器从该总线上拾取请求并执行必要的查找。在这样的大型共享 L2 缓存中，内存块没有重复副本，但给定块可能缓存在多个不同的 L1 缓存中。必须在 L1 和 L2 之间维护一致性。在图 1.1 基于总线的示例中，一致性通过

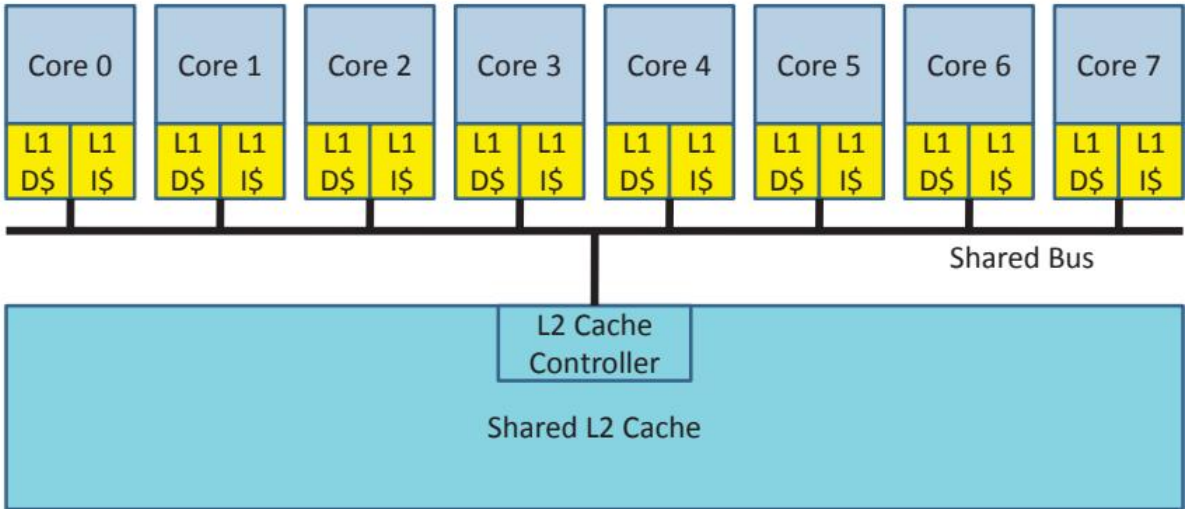


图 1.1　具有大型共享 L2 缓存和每核私有 L1 缓存的多核缓存组织。通过连接 L1 和 L2 的总线实现基于侦听的缓存一致性协议。

基于侦听的协议来维护。假设 L1 缓存采用写回策略。当一个核将请求放到总线上时，其他每个核都会看到该请求并查找其 L1 缓存，以查看是否有请求块的副本。如果一个核拥有该块的修改状态副本，即该副本恰好是最新版本且是该块的唯一有效副本，则该核必须通过将请求数据放到总线上来响应。如果没有核拥有修改状态的块，则 L2 缓存必须提供请求数据。L2 缓存控制器通过检查一组控制信号来判断它必须响应，这些信号表明各核已完成侦听且没有修改状态的块。如果请求核正在执行写操作，则在侦听操作期间，其他 L1 缓存中该块的副本将被无效。当然，这种基本的基于侦听的协议可以有许多变体 [16，17]。如果 L1 缓存采用写直达策略，则 L1 缺失总是由 L2 服务。写直达策略可能导致显著的总线流量和能量消耗；在常见情况下这种开销不值得。同样，写更新缓存一致性协议也是流量密集型的，不常用。然而，在现代单芯片多核且互连相对便宜的背景下，其中一些设计指南值得重新审视。

上述示例主要说明了访问共享缓存之前所需的接口。本质上，需要一种机制来确保共享 L2 和多个私有 L1 之间的一致性。如果共享 L2 的核心数量相对较少（16 个或更少），共享总线和基于监听的一致性协议可能会很好地工作。对于更大规模的系统，通常采用可扩展互连和基于目录的一致性协议。如图 1.2 所示，核心和 L2 缓存通过某种可扩展网络连接，广播请求不再是一种选择。核心将其请求发送到 L2 缓存，每个 L2 块都与一个目录相关联，该目录跟踪其他 L1 缓存是否具有该块的有效副本。如有必要，会单独联系其他缓存以使数据无效或获取数据的最新副本。

采用共享缓存有许多优点。首先，可用存储空间可以在多个核心之间动态分配，从而更好地利用整体缓存空间。其次，如果数据由多个核心共享，则在 L2 中只维护一个副本，这同样可以更好地利用空间并提高缓存命中率。第三，如果数据由多个核心共享并遭受许多一致性未命中，则必须导航缓存层次结构，直到遇到一致性接口和共享缓存。遇到共享缓存越早，一致性未命中就能越早解决。

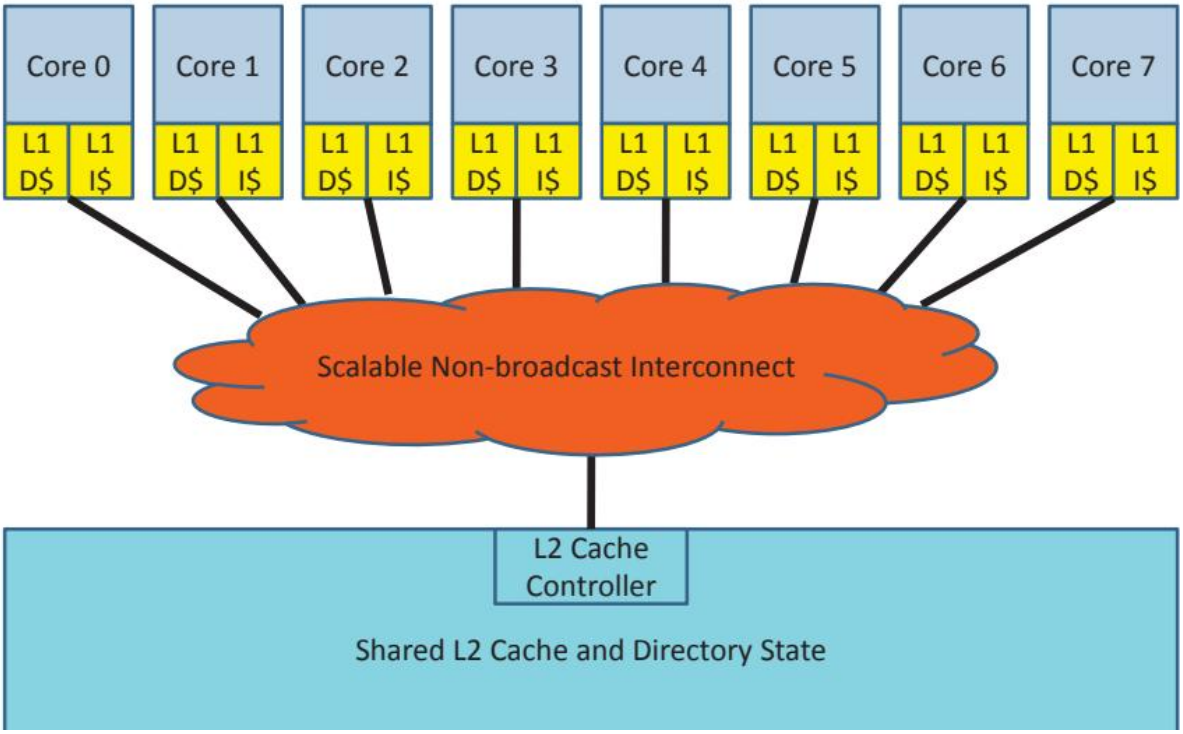


图 1.2 多核缓存组织，具有一个大型共享 L2 缓存和每个核心私有的 L1 缓存。可扩展网络连接 L1 缓存和 L2，并采用基于目录的缓存一致性协议。L2 缓存中的每个块维护目录状态，以跟踪在 L1 中缓存的副本。

共享缓存的主要缺点如下。不同核心的工作集可能相互干扰并影响彼此的未命中率，可能导致服务质量变差。如上所述，访问共享 L2 需要导航一致性接口：如果核心主要处理非多个核心共享的数据，这可能会带来开销。最后，许多论文指出，单个大型共享 L2 缓存的平均访问时间可能相对较长。此外，核心在尝试访问多个核心共享的资源时可能会经历许多争用周期。然而，我们随后（第 2.1 节）将表明，这两个缺点都可以轻松缓解。

必须注意，我们的许多示例假设 L2 缓存是包含性的，即，如果数据块存在于 L1 中，则它必然也存在于 L2 中。在第 1.4 节中，我们将讨论选择包含性和非包含性实现时的考虑因素。

1.1.2 私有 LLC

单一共享 LLC 的一种流行替代方案是私有末级缓存的集合。假设采用两级层次结构，核心现在与私有 L1 指令和数据缓存以及私有统一（处理数据和指令）L2 缓存相关联。L1 中的未命中会触发对核心私有 L2 缓存的查找。这种组织的一些优点/缺点已经显而易见。

在不同核心上执行的线程的工作集不会导致彼此 L2 缓存的干扰。每个私有 L2 缓存相对较小（相对于必须由多个核心共享的单个 L2 缓存），从而平均而言 L2 命中的访问时间更短。访问私有 L2 缓存无需导航一致性接口，也无需竞争共享资源，从而为

主要处理非共享数据的线程带来性能优势。

私有 L2 缓存的一个主要缺点是，多个线程共享的数据块将在每个线程的私有 L2 缓存中复制。相对于总容量相似的共享 L2 缓存，这种数据块的复制导致有效组合 L2 缓存容量降低。换句话说，四个私有 256 KB L2 缓存由于块的重复副本，将容纳少于 1 MB 的数据，而 1 MB 的共享 L2 缓存确实可以容纳 1 MB 的数据。私有 L2 缓存组织的另一个缺点是在核心之间静态分配 L2 缓存空间。在上面的示例中，每个核心被分配一个 256 KB 的私有 L2 缓存，即使某些核心可能需要更多或更少。在 1 MB 的共享 L2 缓存中，如果一个核心的工作集大小远大于其他核心上的线程，则一个核心有可能占用（例如）总空间的 512 KB。

通过采用私有 L2 缓存，一致性接口被下推到缓存层次结构的较低级别。首先，考虑采用基于总线的监听一致性协议的小规模多核机器。在 L2 未命中时，请求在总线上广播。其他私有 L2 缓存执行监听操作并将其响应放在总线上。如果确定其他私有 L2 缓存都无法响应，则控制器将此请求转发到层次结构的下一级（L3 缓存或主内存）。当访问共享数据时，这种私有 L2

组织比具有私有 L1 和共享 L2 的模型带来更大的延迟开销。关键的区别性开销如下：（i）在将请求放在总线上之前，会查找私有 L2 缓存；（ii）监听需要更长时间，因为必须搜索更大的标签集；（iii）从另一个大型私有 L2 数据阵列读取数据比从另一个小型私有 L1 缓存读取数据需要更长时间。

如果采用基于目录的协议（如图 1.3 所示），一致性接口甚至更加复杂。在 L2 未命中时，请求不能广播到所有核心，而是发送到目录。该目录可以是集中式或分布式的，但无论哪种情况，都可能需要穿越较长的片上距离。该目录必须跟踪片上缓存的所有块，并且它本质上复制了所有私有 L2 缓存的标签。需要高度关联的搜索来检测请求的地址是否在任何私有 L2 缓存中。如果四个 256 KB 私有 L2 缓存中的每一个都是 4 路组关联，则目录查找将需要 16 次标签比较来确定块的状态。通过保持 L1 和 L2 之间的包含性，无需复制 L1 标签。如果在另一个私有 L2 缓存中检测到块，则目录和核心之间交换消息，以将数据的最新副本移动到请求核心的私有缓存。另一方面，共享 L2 缓存只需将目录与块在

L2，从而消除了复制 L2 标签的需要。简而言之，在包含性假设下，使用共享的片上 LLC 使得检测片上是否存在缓存副本变得更加容易。

私有和共享 L2 缓存组织之间的差异总结在表 1.1 中。还值得注意的是，未来的处理器可能会采用私有和共享缓存的组合。例如（图 1.4），在 16 核处理器中，每四个核组成的集群可以共享一个 L2 缓存，并且有四个这样的 L2 缓存，每个缓存对其四个核的集群是私有的。基于侦听的缓存一致性首先在一个集群内的四个 L1 数据缓存和 L2 缓存之间维护；基于侦听的缓存一致性再次在四个私有 L2 缓存之间维护。

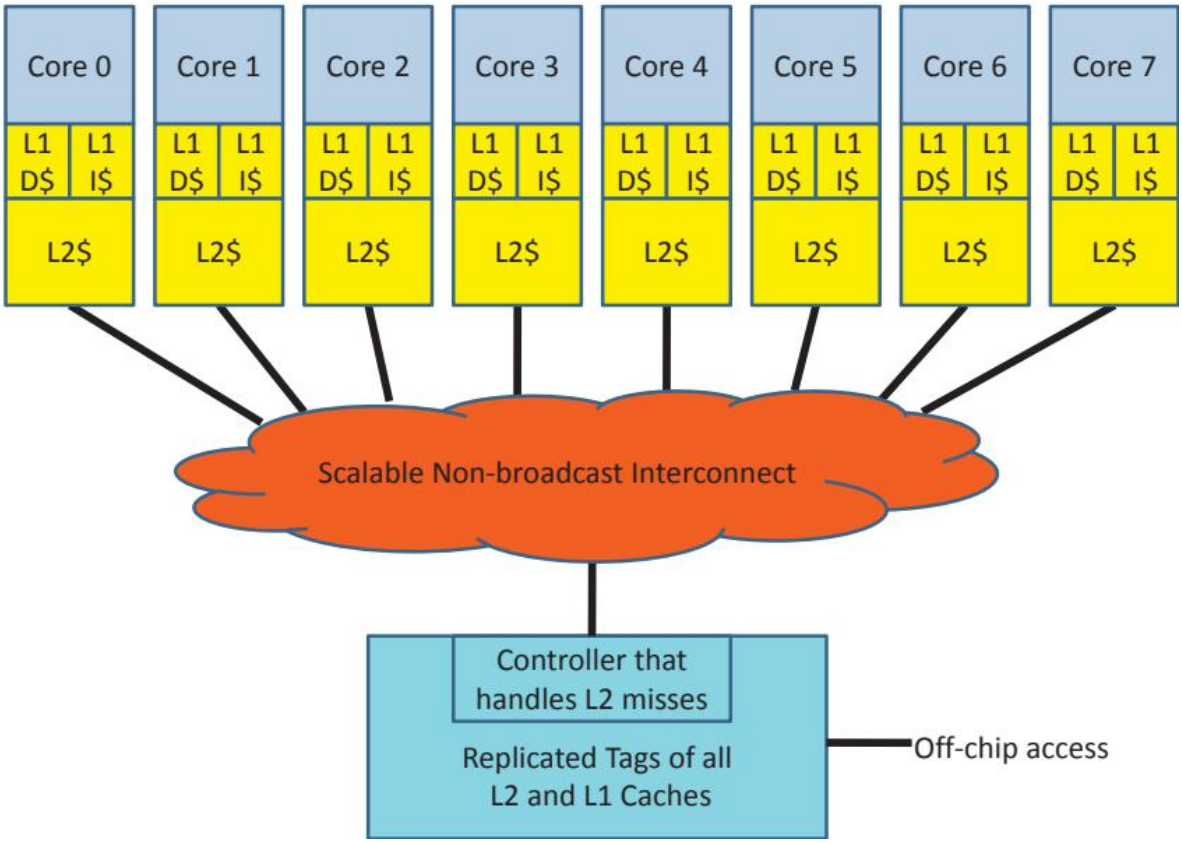


图 1.3 多核缓存组织，其中每个核心具有私有 L2 缓存，并且通过基于目录的协议在可扩展的非广播互连上在私有 L2 缓存之间维护一致性。

表 1.1 共享 L2 缓存与私有 L2 缓存的优缺点比较

共享 L2 缓存	**私有 L2 缓存**
共享数据无须复制，有效容量高	共享数据可能重复存储，有效容量降低
线程／核心动态分配空间，容量利用率高	核心间的缓存容量在设计时固定分配
较快通过一致性接口，共享数据访问延迟低	一致性接口遍历较慢，共享数据延迟高
目录实现无须复制 L2 标签，面积开销低	目录实现需复制 L2 标签，面积开销高
线程间干扰较强，不利于 QoS	线程间干扰小，有利于 QoS
命中时平均布线距离较长，延迟较高	命中时布线距离较短，延迟较低
共享总线、缓存可能发生较多争用	私有 L2 访问不发生共享缓存争用

1.1.3　工作负载分析

最近的一些论文专注于分析基线共享和私有 LLC 对各种多线程工作负载的影响。Jaleel 等人 [18] 的工作描述了生物信息学工作负载的行为。他们表明超过一半的缓存块是共享的，并且绝大多数 LLC 访问都是针对这些共享块的。鉴于这种行为，共享 LLC 明显优于由许多私有 LLC 组成的 LLC。Bienia 等人 [19] 展示了 SPLASH-2 和 PARSEC 基准套件的工作负载分析，包括缓存未命中率和线程间数据共享的程度。许多其他缓存论文也在其分析中报告了工作负载特征，最值得注意的是 Beckmann 等人 [20] 和 Hardavellas 等人 [3] 的工作。

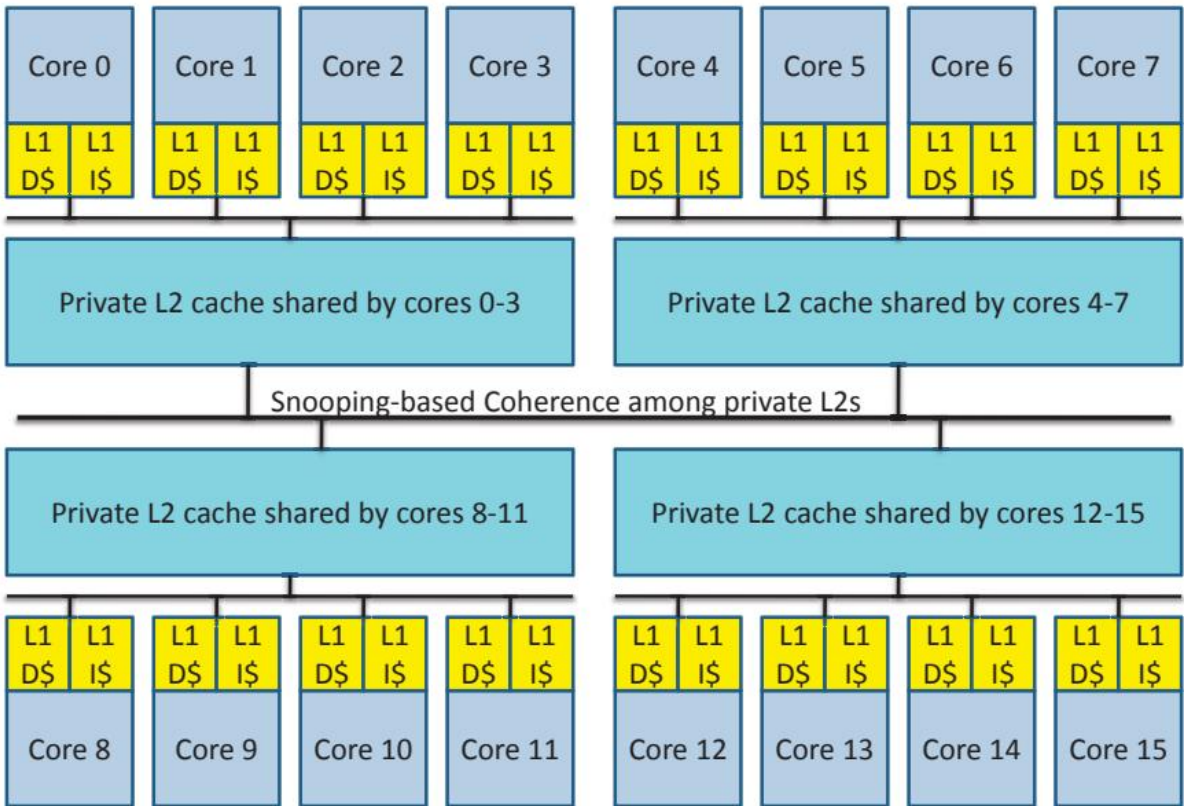


图 1.4　16 核机器，其中每四个核组成的集群有一个私有 L2 缓存，由其四个核共享。这里有两个层次化的一致性接口：一个在一个集群内的 L1 数据缓存和 L2 缓存之间，另一个在四个私有 L2 缓存之间。

本节主要讨论共享末级缓存的不同实现。在本节末尾，我们解释了其中一些原则如何也适用于私有缓存的集合。

我们已经在图 1.1 和 1.2 中考虑了两种共享 L2 缓存组织。在这两个示例中，L2 缓存及其控制器被表示为单个集中式实体。当产生 L1 缺失时，集中式 L2 缓存控制器从总线（图 1.1）或可扩展网络上的链路（图 1.2）接收该请求。然后它继续在 L2 缓存结构内定位相应的块。如果 L2 缓存很大（通常如此），它本身被划分为众多缓存存储体，并且必须导航某种互连网络以访问其中一个存储体内的数据（更多细节将在后续章节中介绍）。因此，可能必须导航某种形式的网络结构才能到达集中式 L2 缓存控制

器，并且还需要导航另一种结构才能到达 L2 缓存内的适当存储体。根据

1.2 集中式与分布式共享缓存

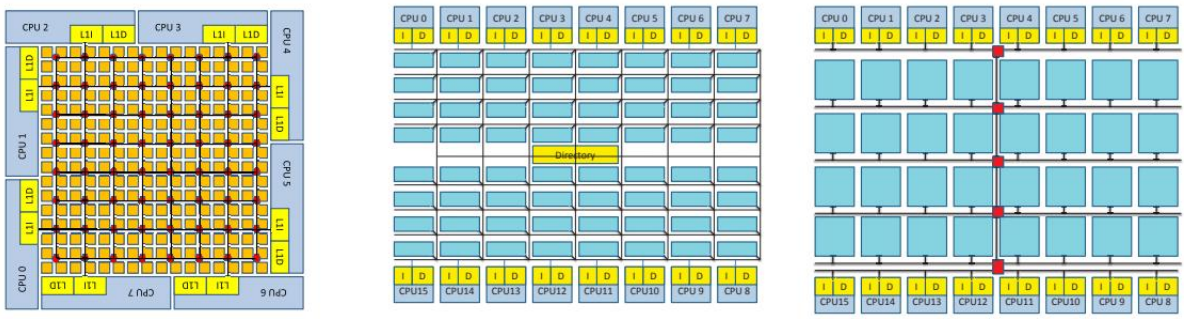


图 1.5 具有集中式布局的共享 L2 缓存（L2 缓存占据芯片中间的连续区域，并被核心包围）。需要片上网络将许多缓存存储体彼此连接并连接到核心。在所有这些情况下，缓存控制器的功能在每个存储体中复制，以避免必须通过中央实体。

所使用的结构类型，可能可以合并两者。在这种情况下，导航单个结构以将请求从核心直接发送到存储该块的 L2 存储体。L2 存储体现在需要一些逻辑来处理必要的缓存一致性操作；换句话说，L2 缓存控制器的功能在每个存储体中复制，以消除必须通过单个集中式 L2 缓存控制器的需要。

这种集中式共享 L2 缓存的一些示例物理布局如图 1.5 所示。这些布局中的每一种都已在研究评估中使用（例如，[2, 7, 21, 22]）。尽管缓存是分存储体的，并且控制器功能分布在各个存储体中，但我们将这些设计称为集中式，因为 LLC 占据芯片上的连续区域。这种集中式缓存结构试图提供一个中央数据池，可以被其周围的核心快速高效地访问。通过保持缓存存储体彼此靠近，简化了存储体之间的数据移动（如果需要）。通过将所有缓存存储体聚集在一起，简化了 L2 缓存存储体与层次结构下一级（例如，片上内存控制器）之间所需的互连。如果 L2 缓存在作为 3D 堆叠芯片一部分的单独晶片上实现（假设单个晶片到晶片总线在 CPU 和单个 L2 缓存控制器之间通信请求和响应），它最终也会成为集中式结构。

这种模型的一个明显扩展是分布式共享 L2 缓存。尽管 L2 缓存在逻辑上是共享资源，但它在物理上可以分布在芯片上，使得 L2 的一个存储体可以放置在每个核心附近。核心、其 L1 缓存和 L2 缓存的一个存储体（或切片）一起构成一个瓦片。单个片上网络用于连接所有瓦片。当核心发生 L1 缺失时，其请求通过片上网络路由到预期在其 L2 存储体中具有该块的瓦片。这种瓦片化和分布式的缓存组织是可取的，因为它允许制造商设计单个瓦片，并根据面积预算实例化尽可能多的瓦片。因此，它更适合可扩展的设计/验证成本、易于制造具有不同瓦片数量的处理器系列，以及简单升级到新技术世代。这种分布式共享缓存也已在最近的 Tilera 多核处理器中实现。图 1.6 显示了这种物理布局的两个示例。我们还将随后看到数据放置的架构机制如何利用这种物理组织。这种组织的主要缺点是在 L2 缓存存储体和内存层次结构的下一级之间移动数

据/请求的成本较高。

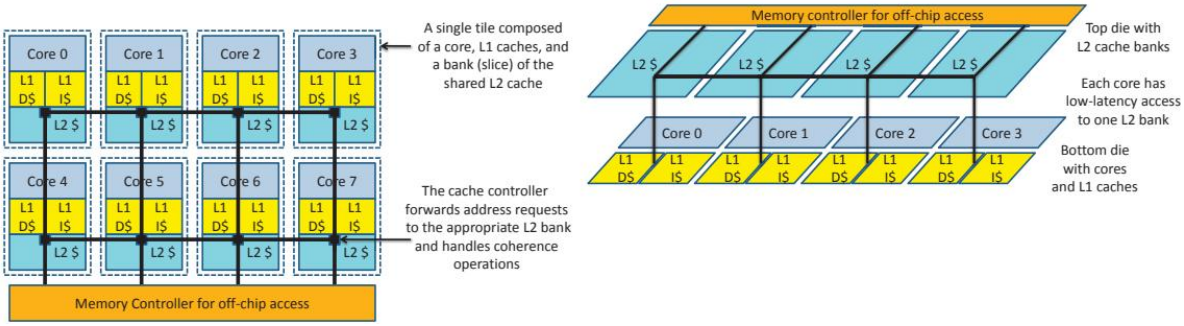


图 1.6　物理分布式布局的共享 L2 缓存。每个核心关联一个 L2 缓存存储体（或“切片”）。“瓦片”由一个核心、其 L1 缓存以及关联的 L2 存储体组成。右图将 L2 存储体物理分离到 3D 堆叠中的独立晶片上，但保留了与左图相同的逻辑组织。

对于中等核心数量的处理器，集中式 L2 缓存结构可能是一种合理的设计选择，但对于众核处理器而言，它可能效率低下。散热和互连（可扩展性和线长）限制可能阻碍众多核心围绕单个大型集中式 L2 缓存。因此，众核处理器极有可能采用分布式 L2 缓存，其中每个核心至少能非常快速地访问共享 L2 缓存的一个存储体。分布式 L2 缓存对功率密度和散热也有积极影响。

我们对集中式和分布式缓存的定义仅作为推理缓存特性时的非正式指南。集中式缓存定义为在芯片上占据连续区域的缓存。分布式缓存定义为每个存储体紧密耦合到单个核心或一组核心的缓存。在图 1.6（a）中，如果核心 4-7 的布局是核心 0-3 的镜像，则该缓存将同时被归类为集中式和分布式，这确实有些奇怪。

在实现私有 L2 缓存组织时，将核心的私有 L2 缓存放置在远离核心的位置几乎没有意义。因此，私有 L2 缓存组织的物理布局与刚才描述的分布式共享 L2 缓存非常相似。换言之，对于这两种组织，一个瓦片都包含一个 L2 缓存存储体；决定 L2 缓存空间是共享还是私有的，是在众多 L2 存储体中放置和管理数据的逻辑策略。

对于私有 L2 缓存组织，如果核心在其私有 L2 缓存中未命中，请求必须转发到一致性接口。假设采用基于目录的一致性，请求被转发到目录结构，该结构本身可以是集中式或分布式的。实现集中式/分布式片上目录的考虑与实现集中式/分布式共享 L2 缓存的考虑非常相似。因此，片上网络主要用于处理一致性操作和对下一级存储层次的访问。分布式共享 L2 缓存的片上网络也大量用于处理 L2 命中。

1.3　非均匀缓存访问

直到最近，大多数处理器中的缓存结构都被设计为无论访问哪个块都具有统一的访问时间。换言之，缓存访问的延迟被设置为任何块的最坏情况延迟。这种统一缓存访问（UCA）架构无疑简化了任何相关的指令调度逻辑，特别是当核心流水线必须知晓缓存命中延迟时。然而，随着缓存变得更大并碎片化为众多存储体，要求每次缓存

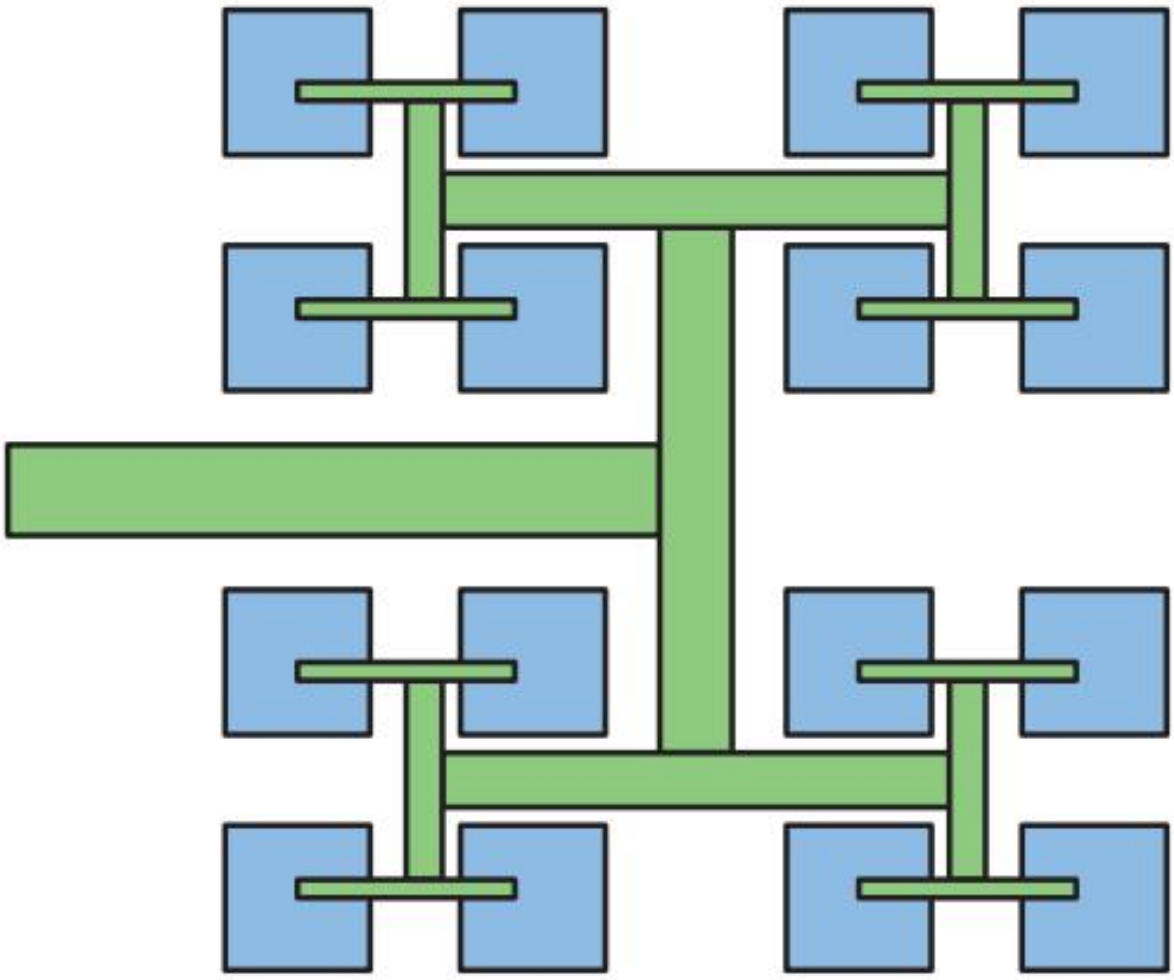


图 1.7 具有 16 个阵列的统一缓存访问（UCA）架构的 H 树网络示例。

访问都承受访问最远存储体的延迟惩罚显然效率低下。对网络结构的简单创新可以使缓存支持非均匀访问时间；事实上，本章迄今为止讨论的许多分体缓存组织都是非均匀缓存访问（NUCA）架构的示例。

UCA 分体缓存设计通常采用 H 树拓扑作为连接存储体与缓存控制器的互连结构。在 H 树拓扑中（示例如图 1.7），每个存储体到缓存控制器的距离相等，从而实现到每个存储体的统一访问时间。Kim 等人在其开创性论文 [1] 中描述了支持 10 1. 大型缓存设计的基本要素

NUCA 架构所需的创新，并量化了其性能优势。首先，L2 的可变访问时间是可接受的，因为核心流水线不会根据预期的 L2 访问时间来调度指令。其次，采用网格拓扑而非 H 树拓扑来连接存储体与缓存控制器。存储体的延迟是其大小以及在存储体和缓存控制器之间路由请求/数据所需的网络跳数的函数。值得注意的是，此网格网络上的消息可能具有某种不规则的模式，需要在每一跳使用复杂的机制来支持路由和流控制。这些机制在 H 树网络中是不需要的，在 H 树网络中请求以流水线方式从缓存控制器辐射出去。网络的复杂性是 NUCA 架构为提供对部分缓存数据的低延迟访问所付出的最大代价。可以说，大多数未来架构无论如何都需要复杂的片上网络来处理众多核心和

缓存存储体之间某种任意的消息传递。特别是在诸如图 1.6 所示的瓦片架构中，不同缓存存储体不可避免地会因网络距离而产生不同的访问延迟。

在下一章中，我们将讨论 NUCA 架构的创新，这些创新试图巧妙地将数据放置在“最优”缓存存储体中。这些创新进一步强调了流量模式具有某种任意性，需要复杂的路由/流控制机制。在第 4 章中，我们描述了适用于特定形式 NUCA 设计的片上网络创新。在本节的剩余部分，我们描述 Kim 等人在其论文 [1] 中提出的基本 NUCA 设计。

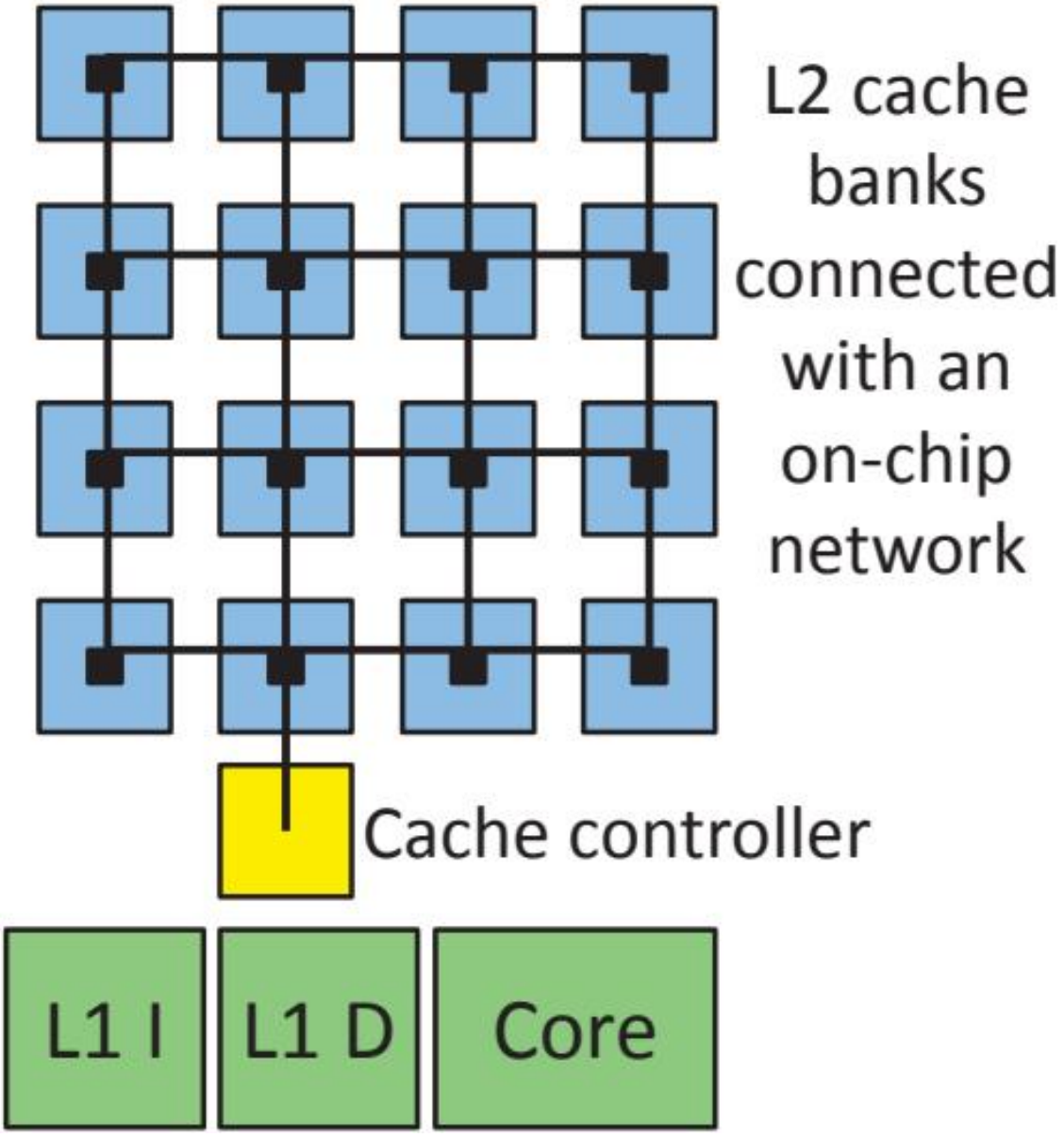


图 1.8　连接到单个核心的 NUCA L2 缓存 [1]。

物理设计。 Kim 等人 [1] 考虑了一个大型 L2 缓存，其中单个缓存控制器为一个处理器核提供服务（见图 1.8）。在物理布局方面，他们考虑了两种实现方式。第一种在每个缓存存储体与缓存控制器之间采用专用通道。虽然专用通道可以为每次访问提供低争用和低路由开销，但这些专用通道对金属面积的高要求令人望而却步。这种设计也不容

易扩展到多个缓存控制器或核。第二种布局采用具有网格拓扑的包交换片上网络。这确保了可容忍的金属面积要求，同时仍提供高带宽和相对较低的争用。通过将多个缓存控制器（核）连接到网格外围的路由器（或任何路由器），可以轻松支持它们。虽然 Kim 等人 [1] 提倡使用支持 1 周期跳的“轻量级”路由器，但尚不清楚能否在设计此类路由器的同时有效支持缓存网络的流控制需求（关于路由器的更多内容见第 4 章）。如果这种轻量级路由器存在，Kim 等人正确地指出，高度分 bank 的缓存结构是可取的：它减少了 bank 内的访问时间，减少了 bank 和路由器处的争用，支持更高的总带宽，并提供对缓存资源的更细粒度控制。

逻辑策略。 数据管理的逻辑策略必须解决以下三个问题：（1）映射：数据块的可能位置，（2）搜索：定位数据块所需的机制，以及（3）移动：改变块位置所需的机制。

最简单的映射策略将缓存的组分布到各个 bank，同时将一个组的所有路放在一个 bank 中。因此，块地址及其对应的缓存索引位足以定位容纳该组的唯一 bank。请求被路由到该 bank，对该组中的所有路进行标签比较，并将适当的数据块返回给缓存控制器。由于数据块到 bank 的映射是唯一的，这种架构被称为 Static-NUCA 或 S-NUCA。这种设计不支持块在 bank 之间移动，也不需要搜索块的机制。

另一种映射策略将路和组分布到各个 bank。一个组的 W 路可以分布到 W 个不同的 bank。必须定义策略来确定在取指时块放置的位置。同样，需要策略在路之间移动数据块，以最小化平均访问时间。由于允许块在 bank 之间移动，这种架构被称为 Dynamic-NUCA 或 D-NUCA。最后，需要一种搜索机制来快速定位可能位于 W 个不同 bank 之一的块。Kim 等人为每种情况考虑了几种策略，这里描述了其中重要的几种。

块的搜索可以增量方式进行，即首先查找最近或最可能的 bank，如果在那里没有找到块，则查找下一个可能的 bank。或者，可以执行多播搜索操作，其中请求被发送到所有候选 bank，并同时搜索它们。第二种方法将产生更高的性能（除非它引入过多的网络争用），但也会带来更高的功耗。两者的组合是可能的，例如，在最可能的 bank 上进行多播，然后在剩余 bank 上进行增量搜索。Kim 等人提出了一种智能搜索机制，其中每个块的部分标签（六位）存储在缓存控制器中。查找这个部分标签结构有助于识别可能拥有请求数据的一小部分 bank，现在只需搜索这些 bank。虽然这种方法对于所有块替换都通过缓存控制器发生的单核设计非常有效，但这种设计不能很好地扩展到多核设计，因为在多核设计中，部分标签必须在可能许多缓存控制器上冗余维护。下一章讨论了 Chishti 等人 [23, 24] 提出的替代解决方案。迄今为止，D-NUCA 缓存中的高效搜索仍然是一个未解决的问题。

为了允许频繁访问的块迁移到更靠近缓存控制器的 bank，Kim 等人采用了一种代际提升机制（并考虑了几种替代方案）。在缓存未命中时，取指的块被放置在离缓存控制器最远的 bank 中的某一路。在随后的每次命中时，该块与位于相邻 bank（路）中的块交换位置，并逐渐靠近缓存控制器。

总结

Kim 等人表明，NUCA 缓存明显优于类似大小的 UCA 缓存和多级层次结构（尽管请注意，N 路 D-NUCA 缓存的行为类似于非包含性 N 级缓存层次结构）。D-NUCA 策略比 S-NUCA 提供约 10% 的性能优势，如果结合智能搜索机制，这一优势将增长到 17%。这支持使用“巧妙”的数据放置，但 D-NUCA 机制的可行性有些值得怀疑。数据移动本质上是复杂的；考虑一个例子，在块正在迁移的过程中搜索该块，以及必须纳入处理此类边缘情况的机制。智能搜索的可行性，特别是在多核领域，是一个重大挑战。虽然 D-NUCA 的性能很有吸引力并引发了大量研究，但最近的工作表明，可能可以设计出结合 S-NUCA 的硬件简单性和 D-NUCA 的高性能的缓存架构。第 2 章将讨论这些相关工作中的几个。

在本书的大部分讨论中，我们将假设包含性缓存层次结构，因为它们更容易推理。然而，许多研究评估和商业处理器也采用非包含性层次结构。理解这一重要设计选择的影响是值得的。不幸的是，研究论文（包括本书作者的论文）经常忽略提及关于包含性的假设。第 3.2.1 节讨论了一篇最近的论文 [25]，该论文评估了定义包含策略时的几个考虑因素。

如果 L1-L2 层级是包含式的，这意味着 L1 中的每个块在 L2 中都有一个备份副本。以下策略可确保包含性：当一个块从 L2 中被逐出时，其在 L1 缓存中的副本也会被逐出。如果单个 L2 缓存由多个 L1 缓存共享，则所有 L1 中的副本都会被逐出。这一操作与缓存一致性协议中的 L1 块无效操作非常相似。

1.4　包含性

在多核中，包含式层级的主要优势在于 L1 未命中时易于定位数据块——要么在 L2 中找到该块的副本，要么 L2 指向某个 L1 中该块的修改版本，要么 L2 未命中表明请求可以直接发送到层级的下一级。L2 缓存也是处理来自层级较低级别（L3 或片外）的一致性请求时的中心点。包含式层级的缺点是空间浪费，因为大多数 L1 块在 L2 中都有冗余副本。

有些 L1-L2 层级被设计为互斥式（数据块要么在某个 L1 缓存中找到，要么在 L2 缓存中找到，但不会同时存在于两者中）或非包含式（不保证 L1 块在 L2 中有备份副本）。在这种设置下，数据块搜索更为复杂：在 L1 未命中时，必须查找其他 L1 缓存和 L2。如果在 L1 和 L2 之间采用基于监听的一致性协议，这并非主要开销，因为每次 L1 未命中时都会在所有 L1 上进行广播和搜索。即使处理来自层级较低级别的一致性请求时，也必须执行这种对 L1 的搜索。然而，正如基于监听的协议一样，随着 L1 缓存数量的增加，搜索操作的可扩展性不佳。当然，其优势在于整体缓存容量更高，因为块的重复很少（或没有）。

另一个基本实现选择是使用写直达或写回策略。两者都可以轻松兼容包含式 L1-L2 层级，如果得到足够高的互连带宽和功耗预算支持，写直达策略能带来更高的性能。写直达策略确保共享块可以在 L2 缓存中快速找到，而无需查看另一个核心的 L1 缓存。

写回缓存通常适用于非包含式层级。

第二章　在 CMP 末级缓存中组织数据

多核很可能在其片上末级缓存中容纳数兆字节的数据。正如第 1 章所讨论的，末级缓存（LLC）可以由多个核心逻辑共享，并且在片上可以是物理分布或物理连续的。或者，每个核心可以维护一个私有 LLC。接下来我们将考察若干架构创新，这些创新试图巧妙地将数据块放置在 LLC 内，以优化未命中率、访问时间、服务质量和吞吐量等指标。本章主要关注对不同缓存 bank 具有非均匀访问的缓存架构，以及试图最小化块访问时间的技术（本章同时考虑共享和私有 LLC）。下一章关注对缓存访问的非均匀特性不敏感或不可知的数据放置技术。

2.1　大型共享 NUCA 缓存的数据管理

我们首先考察由所有片上核心共享的 LLC。预计无论 LLC 缓存在片上是物理连续还是物理分布，如此大型的数兆字节缓存都会对 bank 提供非均匀延迟。大多数创新将适用于任一种物理布局，在后续讨论中，我们会指出哪些地方并非如此。

在第 1.3 节中，我们已经讨论了 NUCA 实现的基础知识 [1]，其中 bank 通过片上网络连接，缓存访问延迟由网络距离和每个请求的争用决定。我们还讨论了数据映射、移动和搜索所需的基本机制。请注意，Kim 等人 [1] 的工作仅涉及单核处理器的 NUCA 设计。本节讨论的论文试图为支持多核的 NUCA 缓存开发数据管理解决方案。我们首先讨论几篇以使用复杂搜索机制为特点、但在数据放置和迁移方面允许极大灵活性的论文。然后讨论一些关于 LLC 数据复制的有限工作。最后以讨论通过不需要复杂搜索的方案来影响数据放置的论文结束。

第 2 章

2.1.1　D-NUCA 的放置/迁移/搜索策略

Kim 等人 [1] 的工作引入了两种主要形式的 NUCA 缓存：静态（S）和动态（D）NUCA。由于 D-NUCA 预期性能更好，早期 NUCA 工作大多集中在 D-NUCA 上，并允许块在 LLC 内迁移。本小节几乎完全关注此类 D-NUCA 设计。然而，所有这些工作都不得不承受相当复杂的搜索机制带来的开销，这个问题至今没有令人信服的解决方案。

Beckmann 和 Wood，MICRO’04

Beckmann 和 Wood[2] 提出了第一个详细的多核 NUCA 架构。他们假设一种布局（如图 2.1 所示），其中共享 NUCA 缓存位于芯片中部，被八个核心包围。向多核的转

变促使对 Kim 等人的 NUCA 架构做出以下基本改变：1. 请求从片上八个分布式位置注入 NUCA 网络（这导致流量模式更加不规则）。2. 缓存被划分为更多的 bank（多达 256 个 bank）以支持更高的带宽需求（后来的研究 [7] 表明这种过度的 bank 划分并非必要）。3. 通过假设更大的链路宽度并将每个路由器连接到四个 bank 来缓解片上网络开销。

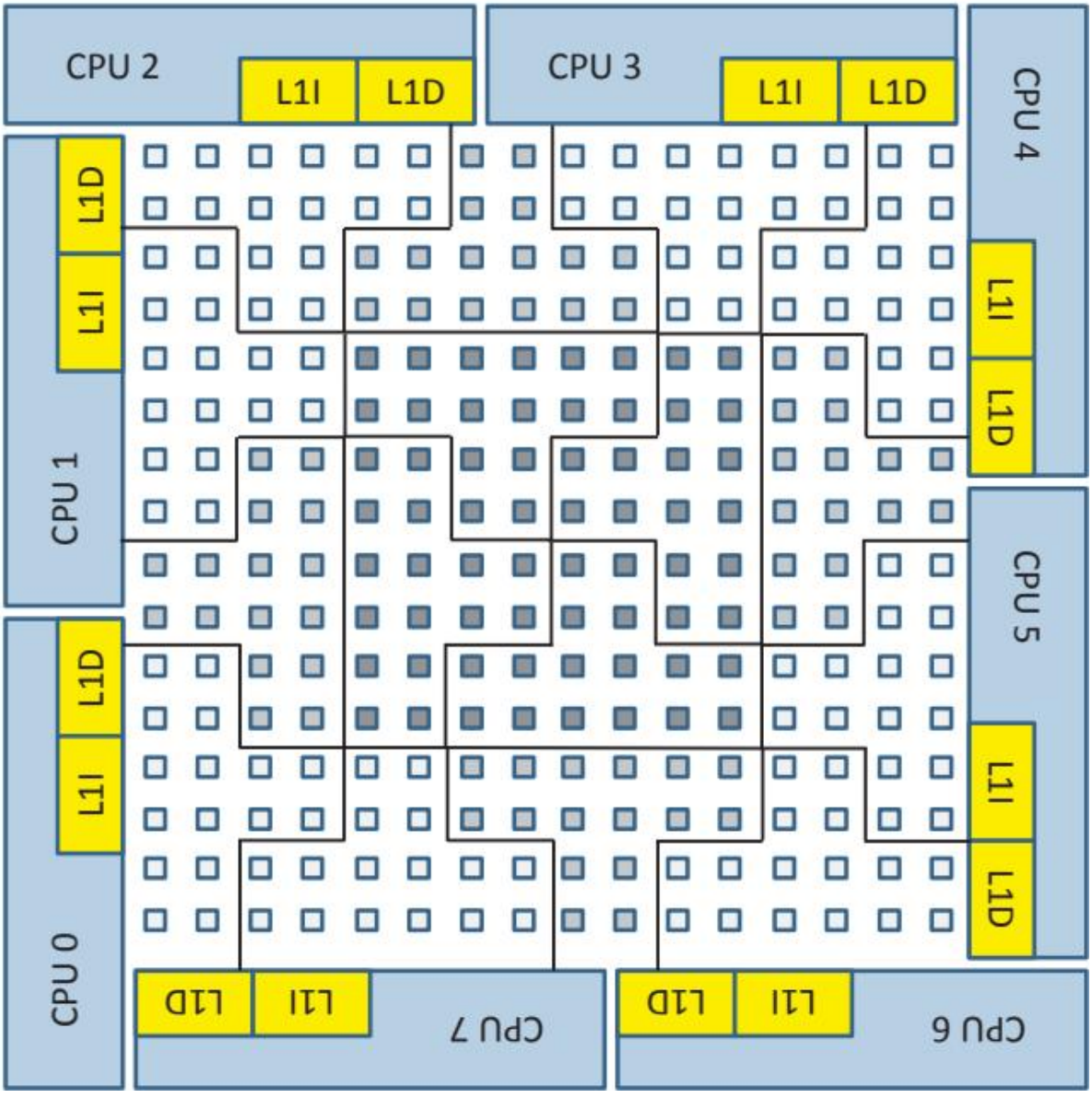


图 2.1 8 核处理器中的 NUCA L2 缓存 [2]。

这项工作的一大贡献是将存储体划分为区域，并制定架构策略，使数据块能够迁移到使总体访问时间最小的区域。如图 1.5a 和 2.1 所示，256 个缓存存储体被组织成 16 个俄罗斯方块形状的区域，即存储体簇。缓存采用 16 路组相联组织，每个区域容纳所有组中的一路。因此，给定数据块可以驻留在 16 个区域中的任意一个，具体取决于它被放置在哪一路。16 个区域中有 8 个因其与对应 8 个核的物理邻近性而被归类为本地区域（图中浅色阴影区域）。理想情况下，核应将其私有数据缓存在其本地区域。四

个中心区域（深色阴影）预期缓存由多个核共享的数据。四个中间区域预期缓存由两个相邻核共享的数据。

数据块放置（初始分配）和移动的规则很简单。初始放置有些随机（基于数据块的标签位）。此后，数据块可以根据访问它的核逐渐迁移到不同区域。随着时间的推移，当核将数据块“拉”向它们的本地区域时，数据块最终会稳定在其所有请求的“重心”处。以下迁移规则允许数据块移动，而无需为每个数据块维护状态。“其他本地”指的是除请求核之外的某个核的本地存储体。

其他本地 => 其他中间 => 其他中心 => 我的中心 => 我的中间 => 我的本地

Beckmann 和 Wood 正确地指出，上述架构最显著的问题在于定位数据块的困难。在将路分布到各存储体的动态 NUCA 策略中，包含数据的路（存储体）事先未知。Beckmann 和 Wood 采用多播搜索策略，首先将数据块地址多播到最可能包含所请求数据块的六个存储体（本地、中间和四个中心区域）。如果所有六个存储体都返回未命中，则将请求多播到其余 10 个存储体。虽然这种搜索机制速度相当快，但它带来了显著的网络和存储体负载。在所有 16 个存储体上顺序执行搜索可能过于耗时。Kim 等人 [1] 采用了一种部分标签结构，使单个核能够以适度的开销快速确定存储体。将该方案扩展到多核平台将产生高得多的存储开销和网络开销，以在每次迁移后保持所有部分标签结构一致。上述搜索机制（包括本节末尾讨论的其他创新）在现代功耗和连线受限的世界中似乎都不可接受。此外，任何搜索机制还必须认识到数据块可能正在存储体之间传输，否则即使数据块确实存在于缓存中，也可能发出未命中信号。这需要一种机制来检测不一致性（例如，目录认为存在缓存副本，但搜索未能检测到），然后在所有未完成的迁移完成后重新执行搜索。因此，缺乏稳健的搜索机制显然是任何将路分布到各存储体的多核动态 NUCA 策略的致命弱点。为了估计搜索的影响，Beckmann 和 Wood 还评估了一种理想化的完美搜索机制，该机制能神奇地将请求发送到正确的存储体。

他们的结果可以最好地总结如下。与不需要搜索但不追求数据邻近性的静态 NUCA 策略相比，采用多播搜索的动态 NUCA 策略几乎不提供整体收益。这是因为数据块迁移的收益

（数据邻近性）被昂贵的搜索机制的开销所抵消。需要理想化的完美搜索才能获得几个百分点的提升。即使采用理想化搜索，改进也难以实现，因为许多共享数据块往往驻留在中心存储体中，无法以非常低的延迟访问[1]。Beckmann 和 Wood 表明，使用流缓冲区将数据预取到 L1 中，或许是一种更实用的方法，可以最大限度地减少大型共享 NUCA 缓存中长访问时间的性能影响（尽管请注意，预取会加剧网络功耗问题）。该论文还考虑了传输线，并在第 4.3 节中进行了讨论。Beckmann 和 Wood 论文的一个主要信息是动态 NUCA 的性能潜力有限且复杂度极高，而这一创新在单核环境中曾主要具有积极属性。

Huh 等人，ICS’05

[1]在第 4.3 节中，我们描述了 Guz 等人 [6] 提出的新颖的 Nahalal 布局，该布局在类似的 D-NUCA 架构中减少了共享块的延迟。

在 Beckmann 和 Wood 的论文发表后不久，Huh 等人 [22] 的论文验证了上述许多观察结果。Huh 等人评估了一个 16 核芯片多处理器，其中间有一个大型分体式 NUCA 缓存（见图 1.5b）。他们证实，S-NUCA 策略平均会导致较长的访问时间。如果将路分布到各存储体，并允许数据块在存储体之间以 D-NUCA 策略迁移，则会有轻微的 performance 改进。他们考虑了 1 维和 2 维数据块移动，其中 1 维移动阻止数据块移出其指定列。D-NUCA 未能带来实质性性能改进，主要归因于搜索数据块时涉及的复杂性。为了避免访问大量存储体，Huh 等人实现了一组分布式的复制部分标签。在每个存储体列的顶部/底部，存储该列中每个数据块的部分标签。查询此存储可以揭示该列中是否有一个或多个存储体可能包含所请求的数据块。这些对部分标签和存储体的额外查询（比 S-NUCA 情况多近 50%）抵消了 D-NUCA 数据邻近性所带来的一半收益。它们还导致功耗和存储体访问率增加。每次数据块被替换/迁移时，都必须通过片上消息更新各种标签存储。Huh 等人评估了 L1 预取，并表明无论是否使用 D-NUCA，它都是一种隐藏 L2 访问时间的有效技术。

该论文还提出了一种缓存实现，可以在共享-私有频谱的多个点上运行。共享度（Sharing Degree，SD）定义为共享一部分缓存的处理器数量。频谱的一端是由所有处理器共享的缓存（图 1.5b 中 SD 为 16），另一端是将缓存划分为 16 个片段，每个片段作为一个核心的私有缓存（图 1.5b 中 SD 为 1）。当缓存完全共享（SD=16）时，一致性只需在各核心的私有 L1 之间维护，并且每个 L2 块都维护目录状态。当缓存的共享度小于 16 时，还必须在多个 L2 片段之间维护一致性。在芯片中部维护一个目录，每次在 L2 片段中未命中后都必须查找该目录。Huh 等人表明，不同的

应用程序在不同的共享度下能获得最佳性能，并主张采用一种动态方案，根据每个应用程序甚至每个块来选择 SD 值。这种重配置的逻辑预计会相当复杂，尤其是因为逐块重配置可能需要复杂的索引机制。

可变共享度的理念源于共享缓存和私有缓存之间固有的权衡，这已在表 1.1 中解释过。随着共享度的增加，缓存提供更好的命中率，但 S-NUCA 在共享缓存中有些随机的块放置会导致缓存访问时间增加。因此，Huh 等人的工作提出了一种合理的方法，不仅平衡共享/私有缓存的优缺点，还平衡 S-NUCA/D-NUCA 的优缺点。通过采用 S-NUCA 和可调的共享度，他们消除了复杂的搜索，允许可管理的访问时间，并允许可管理的命中率。主要开销是必须重新配置的索引逻辑以及能够在 16 个 L2 片段之间维护一致性的集中式 L2 目录。Huh 等人认为，具有固定共享度 2 或 4 的简单 S-NUCA 设计提供了性能和低复杂度的最佳组合。正如我们将在本章后面看到的，页面着色是一种替代方法，用于形成共享/私有区域，它消除了对基于硬件的 L2 一致性的需求，但它依赖于软件操作系统支持。

Liu 等人，HPCA’04

Liu 等人 [26] 的工作是多核共享 LLC 领域的首批工作之一。他们的工作没有考虑 NUCA 缓存，但查找机制引入了访问延迟的非均匀性。LLC 被分成多个 bank，所有 bank 都连接到所有核心共享的总线（类似于 Barroso 等人的 Piranha 处理器 [21]）。本

质上，每个 bank 包含 LLC 中每个组的一些路。当核心发起 L2 访问时，首先查找一部分 bank 中的路（阶段 1）。如果在这里没有找到数据，则查找剩余 bank 中的路（阶段 2）。这类似于 Kim 等人 [1] 首次引入的多播搜索。为了适应每个核心的不同需求，Liu 等人允许每个核心 i 在阶段 1 查找特定的 W_i 路集合。操作系统通过设置硬件表来确定每个核心的这组 W_i 路。Liu 等人还观察到，这种组织（类似于后来 Beckmann 等人 [2] 和 Huh 等人 [22] 引入的设计）提供了共享缓存和私有缓存之间的混合，因为：（i）类似于私有缓存，每个核心可以将其数据本地化到少数 bank；（ii）类似于共享缓存，命中率高，因为数据不被复制，并且 LLC 空间可以根据需求非均匀地分配给核心。

Dybdahl，HPCA’07

Dybdahl 和 Stenstrom[27] 提出了另一种混合共享/私有 LLC。他们假设 LLC 是共享且分布的（类似于图 1.6）。一个组的路分布在许多切片上，因此它是 D-NUCA 实现，并受到 D-NUCA 问题（复杂搜索和迁移）的影响。他们的关键创新是在每个切片中保留一些路作为本地核心的私有路。其他路被视为共享。对于任何访问，首先查找核心的私有路

，然后查找所有其他路（类似于 Liu 等人 [26] 的策略）。在每次访问时，请求的块被带入该核心在 LLC 内的私有路中。如果必须从私有路中逐出一个块，它会被溢出到许多共享路之一（不一定是该切片中的路）。每个核心被分配缓存中一定数量的预先指定的路（当必须逐出共享路中的块时会注意这一点）。这种将路指定为私有和共享的做法导致了与先前 D-NUCA 设计中采用的不同的替换策略。私有路还根据各种运行时统计信息按核心分配。这些统计信息包括核心私有配额内 LRU 路的命中次数（以估计如果核心被分配少一路时增加的未命中率）以及跟踪每个私有域最后逐出块的影子标签的命中次数（以估计如果核心被分配多一路时增加的命中率）。此类统计信息也已被其他论文采用，并将在第 3 章中更详细地讨论。

通过引入感知未命中率的按核心分配路，Dybdahl 和 Stenstrom 防止共享缓存让 LRU 决定核心间的路划分。路分配统一应用于所有组，因此代表相对粗粒度的划分。

NuRAPID 和 CMP-NuRAPID，Chishti 等人，MICRO’03 和 ISCA’05

虽然 D-NUCA 逐渐被认为“过于复杂而难以实现”，但 Chishti 等人 [24] 的一些早期工作考虑了在单核环境中性能更高且可能更简单的替代方案。他们在 MICRO’03 上的论文仅关注单处理核心的 NUCA 架构，而他们在 ISCA’05 上的论文则考虑了多核。

第一篇文章做出了两项关键贡献：（i）Chishti 等人没有将标签和数据块放在一起，而是提出将整个标签阵列实现为靠近处理核心/缓存控制器的集中式结构。缓存访问从标签查找开始，然后请求被直接发送到拥有数据的 NUCA 存储体。这种设计消除了通过逐步查找多个存储体来搜索块的需要。然而，每次块移动都需要通知集中式缓存控制器，以便更新标签。在多核环境中，标签存储和标签更新的开销将急剧增加。(ii) Chishti 等人提出将标签和数据块的放置解耦。现在允许一个块驻留在任何缓存存储体的任何行中，而标签存储（以传统方式组织）携带指向数据块确切位置的指针。当块被访问时，它们可能移动到更靠近缓存控制器的存储体，与任何可能最近未被访问的块交换

位置。由于现在可以在任意两个块之间进行交换，块移动策略允许最近的存储体容纳“全局最热”（最频繁和最近被访问）的块，而不仅仅是每个组中最热的块。请注意，传统的 D-NUCA 会将每个组限制为仅将一小部分路放置在靠近 CPU 的位置，而 Chishti 等人的 NuRAPID 策略（非均匀访问，使用距离关联性的替换和放置）允许将一个热组的所有路都放置在附近的存储体中。这种灵活性可以使 NuRAPID 优于 D-NUCA，特别是当应用程序非均匀地对其组施加压力时。它还可以减少存储体间的流量，特别是当存储体足够大时。提供这种灵活性的开销是数据块需要存储反向指针，以标识其在标签阵列中的条目，以便在发生交换时更新相应的标签。我们可能还需要独立跟踪大型数据存储体各行之间的 LRU，尽管论文声称在交换期间随机选择块就足够了。

因此，该论文提出了一种有趣的组织结构，将标签和数据块解耦，并允许数据块更灵活地移动。这种解耦的组织结构也被最近的提案所采用，以减少冲突并提高命中率（[28，29]，在第 3 章中讨论）。它不会遇到复杂的搜索问题，因为与 Kim 等人的部分标签存储 [1] 类似，整个标签首先在缓存控制器处检查，然后将请求发送到唯一的存储体。然而，这种灵活性带来了实现复杂性的增加，并且在多核环境中块查找仍然存在问题。

在后续论文中，Chishti 等人将其方案扩展以处理多核 [23]。正如在 NuRAPID [24] 设计中一样，CMP-NuRAPID [23] 设计也将数据和标签阵列解耦。数据阵列是共享资源；任何核心都可以将其数据放置在数据阵列的任何行中；数据阵列被组织为具有非均匀访问时间的多个存储体。这些是与本节到目前为止描述的 NUCA 共享缓存的相似之处。设计的其余部分代表了一种具有私有 L2 缓存的组织。每个核心维护一个私有标签阵列，其条目能够指向共享数据阵列的任何行。保持标签阵列一致很棘手：Chishti 等人假设通过在未命中/移动/替换时向所有标签阵列广播更改来保持标签阵列一致。标签阵列必须足够大，以允许对共享数据阵列的实质性覆盖，并允许数据块的重要共享。对于 4 核处理器，标签阵列组合所需的条目数是共享数据阵列的两倍。这限制了该方法的可扩展性。所提出的设计克服了私有 L2 组织的三个缺点：（i）如果多个核心读取同一个数据块，则会在每个私有缓存中创建数据副本，导致整体容量降低。（ii）如果多个核心读/写同一个块并在各自的私有 L2 缓存中拥有副本，则会出现频繁的 L2 一致性未命中、无效和重复副本。（iii）L2 缓存容量在核心之间静态分区。

Chishti 等人通过在共享 L2 中维护共享块的单一副本并让多个标签阵列指向该块来解决问题（i）和（ii）。问题（iii）通过维护大型标签阵列来解决，这些标签阵列允许核心将块映射到超过其公平份额的 L2 空间（仅受每核标签阵列大小的限制）。数据块维护指向标签条目的单个反向指针——因此，任何块移动都需要广播，以便所有标签阵列可以更新其前向指针。此外，允许对只读块进行受控复制。这些只读块的管理类似于私有缓存组织中的管理：每个标签阵列可以指向附近数据存储体中块的副本，而写操作需要总线广播以使其他缓存副本无效并将块的状态从共享升级为已修改。共享缓存中块的复制将在第 2.1.2 节中更详细地讨论。

因此，CMP-NuRAPID 设计是私有和共享 L2 缓存之间的一种有趣的混合。它具有

共享缓存的许多性能潜力，并且允许对只读块进行选择性复制（使其能够优于传统的共享 NUCA 缓存）。但它也带来了与私有 L2 缓存相关的许多复杂性，特别是涉及标签维护和一致性的复杂性。从这个意义上说，CMP-NuRAPID 继承了 D-NUCA 的主要缺点：如果允许块任意移动并且在数据阵列中没有固定归属，则必须花费精力在需要访问时搜索该块（D-NUCA），或者必须花费精力使每个核心都有指向该块的准确指针（CMP-NuRAPID）。

D-NUCA 块搜索的创新

我们已经提到过在 D-NUCA 缓存中定位一个块所涉及的几个问题。Kim 等人 [1] 最初的 NUCA 论文提出在核心处使用部分标签来定位块，存储开销适中。将这一解决方案扩展到多核环境会带来问题，因为现在每个核心都必须复制这些部分标签，而且每次块移动都会触发广播以更新这些部分标签。Huh 等人 [22] 采用了一种布局和块放置策略，使搜索更加易于管理。核心构成网格网络的顶行和底行，一个块被限制在其静态分配的列中。来自核心的请求被水平路由到该列的顶部或底部，在那里它们查询该列中各 bank 的部分标签，然后将请求路由到适当的 bank 组。Chishti 等人 [23, 24] 假设标签阵列（用于大部分 L2 缓存）位于每个缓存控制器处。标签查找精确地识别出缓存数据块的 bank，并最小化 bank 活动。但这带来了与复制部分标签相同的问题：高存储需求以及每次块移动时更新标签的广播。

Ricci 等人 [30] 使用基于预测的方法来缩小块搜索范围，而不会产生高存储或网络开销。他们依靠每个核心处的布隆过滤器 [31] 来紧凑地表示一个块在每个 bank 中的可能位置。虽然与部分标签结构相比，这将存储开销降低了近一个数量级，但它引入了相当数量的误报。此外，Ricci 等人仅在核心访问块时更新每个布隆过滤器，即，在块获取或移动时不会更新所有布隆过滤器。虽然这减少了片上网络流量，但它引入了漏报，因为过滤器偶尔无法检测到 bank 中块的存在。因此，这种基于布隆过滤器的机制产生对块可能位置的预测，误预测并不罕见。在 L2 访问时，首先搜索布隆过滤器指示的 bank；如果未定位到块，则必须查找所有其他 bank，然后才能发出未命中信号。

尽管有这些创新，搜索机制仍然是 D-NUCA 的一个挑战。对有效的 D-NUCA 搜索机制的探索似乎已经失去了一些动力。不仅 D-NUCA 迁移和搜索的复杂性难以克服，而且似乎还有其他解决方案即将出现（应用于 S-NUCA 的页着色），它们结合了 S-NUCA 和 D-NUCA 的最佳特性以及共享和私有缓存的最佳特性。这种方法将在第 2.1.3 节中简要讨论。

2.1.2　共享缓存中的复制策略

在具有私有 L2 缓存的系统中，每个 L2 缓存被允许保留一个块的只读副本，从而允许对块的低延迟访问。在具有共享 L2 缓存的系统中，单个 L2 只能维护每个块的一个副本。该副本可能不靠近访问该块的核心，导致较长的访问延迟。这在早期的 S-NUCA 设计中尤其如此，其中块放置由块地址位决定，分配的 bank 可能有些随机。这个问题

可以通过允许单个共享 L2 缓存维护一个块的多个副本来缓解，这样每个核心都可以在相对较近的位置找到该块的副本。这种缓存块的复制将需要一个机制来保持副本的一致性。

受害者复制，Zhang 和 Asanovic，ISCA'05

Zhang 和 Asanovic 提出了一种简单而巧妙的机制来实现块复制，而不会在 L2 缓存中的副本之间产生太多一致性开销。作为设计平台，他们假设共享 L2 缓存物理分布在芯片上，类似于前面图 1.6 中讨论的瓦片布局。采用 S-NUCA 设计，因此每个块映射到唯一的瓦片/bank（称为归属 bank）。复制按如下方式实现：当一个块被核心的私有 L1 逐出时，它被放置在本地 L2 bank 中，即使它不是该块的归属 bank。这个新的 L2 块副本被称为副本，我们将相应的核心称为该块的复制核心。幸运的是，由于该块最近缓存在复制核心的 L1 中，目录（与归属 bank 中的块副本一起维护）仍然在共享者列表中列出复制核心。因此，随后对该块的写入（由任何核心）将触发向复制核心/瓦片发送无效消息。在收到无效消息时，核心必须在其 L1 和本地 L2 切片中查找该块，并使找到的任何副本无效。这是一种非常干净的方法来维护 L2 副本之间的一致性，通过依赖现有的 L1 一致性状态，并在每次无效时要求额外的 L2 bank 查找。一致性协议的其余部分保持不变。

当核心在其私有 L1 中未命中时，它会在将其请求转发到该块的归属 bank 之前查找其本地 L2 切片（以找到潜在的副本）。因此，如果本地副本不存在，则在此设计中会产生额外的 L2 查找；相反，如果找到本地副本，则没有额外的 L2 bank 查找，并且还节省了片上网络流量。如果在本地 L2 切片中找到副本，则将其带入 L1 并从本地 L2 切片中移除。

该设计被称为受害者复制，因为复制是针对从 L1 逐出的块进行的。并非每个被逐出的块都在本地 L2 切片中复制。如果本地 L2 切片恰好是该块的归属 bank，则当然不会复制。如果本地 L2 切片没有空闲空间，也不会复制，即，副本只能在当前由无效行、现有副本或没有共享者的归属块占用的位置创建。

缓存索引受到 Victim Replication 方案的影响。通常，索引的最高有效位表示存储体编号。当在本地存储体中查找副本时，索引时必须忽略存储体编号（因为它指向归属存储体），而必须将其作为标记的一部分包含在内，以正确指示该块的命中。这导致标记阵列略有增长。

因此，Victim Replication 试图结合共享缓存和私有缓存的有利特性。通过允许很大一部分访问由本地 L2 切片中的副本服务，L2 访问时间类似于小型私有 L2 的访问时间。该设计继续享有共享缓存的低缺失率优势，因为只有在存在“空闲空间”时才会创建副本。原则上，该设计与为每个 L1 设置单独的牺牲缓存并无不同。虽然传统的牺牲缓存小而固定，但在所提出的设计中，牺牲块可以占据本地 L2 切片中的大片区域，并且会小心避免驱逐其他有用的 L2 块。请注意，Victim Replication 仅适用于每个核心都有指定本地 L2 切片的 S-NUCA 设计。只要不允许副本迁移（以保持目录的准确性），它可以在 D-NUCA 设计中经过少量修改后使用。在采用智能页面着色（在第

2.1.3 节中讨论）的 S-NUCA 设计中，Victim Replication 的价值会有所降低。

ASR，Beckmann 等人，MICRO’06 和 CMP-NuRAPID，Chishti 等人，ISCA’05

关于共享缓存内复制概念的后续工作很少。这可能证明 Victim Replication 思想在性能和实现复杂性方面几乎没有改进空间。我们在此简要讨论两个相关的工作。

Beckmann 等人 [20] 提出了一种动态概率系统来选择适当的复制级别。增加复制会降低 L2 命中的平均访问时间，但也会降低有效容量。当一个块从 L1 中被驱逐时，它以概率 N 在本地 L2 切片（共享或私有）中被复制。概率 N 根据成本效益估计定期增加或减少。引入了各种结构来近似估计假设概率水平 N 较低或较高时本地命中率和内存访问的变化。更多细节将在第 2.2 节中讨论。然而，这些方案更适用于私有缓存组织。Victim Replication 策略已经在其内部编码了隐式的成本效益分析：副本的创建仅以被认为可丢弃的块为代价。我们可以增强该策略，以概率方式允许副本替换“不可或缺”的块，但可能几乎没有改进空间（迄今为止尚未量化这一差距）。

第二个允许在共享缓存中进行复制的相关工作是我们之前描述的 Chishti 等人 [23] 的工作。回想一下，他们采用了一种不寻常的设计，具有私有的每核心标签和共享的数据阵列。当 L2 数据块被远程核心多次读取时，Chishti 等人采用复制。当一个块被复制时，该设计非常类似于私有缓存组织。每个核心的私有标签阵列指向数据块的附近副本。由于标签阵列以共享状态存储该块，因此对该块的写入之前会在总线上执行无效操作，以清除其他共享副本。本讨论出现在本书的共享缓存

部分，是因为他们允许多个标签阵列指向数据阵列中的单个共享块（不仅简化了生产者-消费者共享，而且有时通过避免复制来提高容量）。

SP-NUCA 和 ESP-NUCA，Merino 等人，HPCA 2010

Merino 等人 [32, 33] 的 SP-NUCA 架构为每个核心在共享缓存中引入了单独的索引函数。这使他们能够设计一种混合的共享-私有 NUCA（SP-NUCA）架构。在他们的实现中，最后一级缓存被组织为共享的 S-NUCA。一个块首先被假定为私有块，并且只查找本地存储体（地址的存储体编号位不用于索引，而是作为标记的一部分存储）。如果在本地存储体中未找到该块，则假定它是共享的，并采用常规的 S-NUCA 索引（地址的存储体编号位用于将请求发送到适当的存储体）。如果仍未找到该块，则在将请求发送到内存之前，会查找其他每个核心的私有存储体。在其 L2 生命周期内，一个块最初被分类为“私有”，然后如果被另一个核心触及，则可能以“共享”状态转移到不同的存储体。虽然 SP-NUCA 在本地化私有数据方面很高效，但它需要多次存储体查找来定位共享数据。如果工作负载完全是多道程序，SP-NUCA 会退化为类似于私有缓存组织的行为（更高的缺失率）。除了私有数据的有限空间外，SP-NUCA 还遭受共享块的更长延迟。Merino 等人 [33] 引入了 ESP-NUCA 架构来缓解这些问题。一些共享块允许被复制。此外，存储体的路可以跨私有、共享和副本动态分配。使用类似于动态集合采样（稍后在第 3.1.1 节中讨论）的技术来识别最佳分区。

2.1.3 基于操作系统的页面放置

采用 S-NUCA 策略的共享 L2 缓存具有几个理想特性：它最小化 L2 缓存未命中率并允许简单的块查找。该领域的大多数早期工作指出了这种设计的一个主要缺点：块基于其物理地址静态放置在存储体中；假设块物理地址有些随机，块倾向于均匀地分散在许多存储体中。因此，平均而言，请求由位于芯片另一侧的存储体服务。这推动了关于 D-NUCA 的几项工作，允许块驻留在任何一个存储体中，并试图将相关块带到更近的存储体。这也推动了几项关于私有缓存组织的研究，因为小型私有缓存的命中时间低于大型共享 S-NUCA 缓存的平均命中时间。

然而，与早期研究中的假设不同，数据块的物理地址未必是随机的。利用基于操作系统的页着色 [34] 可以控制数据块的物理地址，从而控制它们在存储体中的放置位置。这些原理多年前就为人所知，甚至在 20 世纪 90 年代就被用于改善 cc-NUMA 多处理器系统中的局部性 [35, 36, 37, 38, 39, 40]。直到 Cho 和 Jin 在 MICRO 2006 上发表论文 [41]，这一思想在 NUCA 领域才得到发掘。在过去几年中，人们越来越清楚地认识到，这种方法或许融合了

S-NUCA 和 D-NUCA 的优点，兼具共享缓存与私有缓存之长，并平衡了性能与实现复杂度。

Cho 和 Jin，MICRO’06

我们首先考察在 S-NUCA 缓存中数据块如何映射到存储体。图 2.2 展示了一个 32 位物理地址，以及用于索引一个 16 MB、8 路、划分为 16 个存储体的 L2 缓存所用的位。这个例子可以很容易地扩展到现代系统中典型的更大物理地址。假设缓存行大小为 64 字节，则最后（最低有效）六位（偏移量）用于选择缓存行内的各个字节。该缓存有 32K 个组，每组包含 8 路。组索引需要 15 位。在 S-NUCA 缓存中，

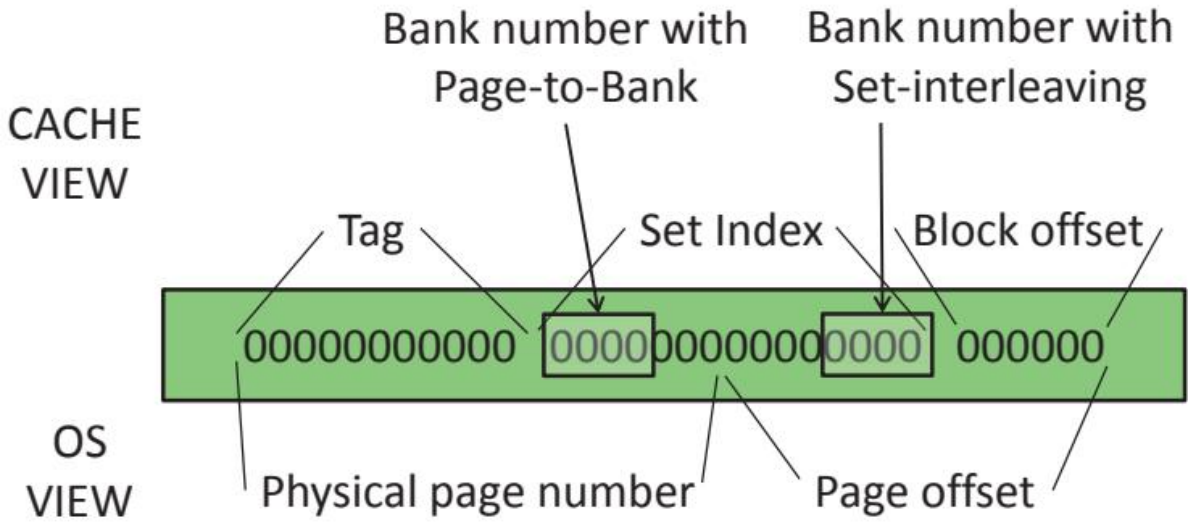


图 2.2 物理地址位与缓存放置及页映射的交互关系。

组被分布到各个存储体，但整个组（全部 8 路）驻留在单个存储体中。因此，组

索引的一个子集用于标识包含该数据块的存储体。早期研究假设组索引的最低有效位用作存储体编号。我们将此称为组交错映射。这导致连续的组被放置在不同的存储体中，并倾向于将程序工作集均匀分布到所有存储体。这有助于将负载分散到各存储体，并减少每个存储体的争用和容量压力。不幸的是，这意味着来自某个核心的请求由最远存储体服务的可能性与由最近存储体服务的可能性一样大。

或者，如果组索引的最高有效位用作存储体编号，则若干连续的组将被放置在同一存储体中。我们将此称为页到存储体映射，因为整个操作系统页可能被放置在一个存储体中。由于程序具有空间局部性，对同一页的一系列请求将由同一存储体服务，导致高争用和不均衡的容量压力。

现在考虑操作系统对数据块物理地址的看法（同样如图 2.2 所示）。假设操作系统页大小为 16 KB，则地址的最低有效 14 位

表示页偏移，其余位表示物理页号。当操作系统将程序的虚拟页分配给物理页时，它最终决定了该页内所有缓存行的组索引位的子集。如果我们采用组交错映射，则页号位（对于图 2.2 中的例子）不与存储体编号位相交；因此，将虚拟页分配给物理页并不决定该页映射到的唯一存储体。该页被分布到所有存储体。如果我们采用页到存储体映射，则存储体编号位是页号位的子集。因此，通过将虚拟页分配给物理页，操作系统也决定了整个页映射到的缓存存储体，因而称为页到存储体映射。这种映射提供了利用操作系统来优化共享缓存中数据放置的选择，而不是采用硬件密集且复杂的 D-NUCA 策略。采用标准术语，表示页号位子集的存储体编号位可称为页颜色。如果有 16 个存储体（因此有 16 种颜色），操作系统根据颜色将其空闲页组织成 16 个链表。当请求新页时，操作系统首先确定该页的最佳存储体，然后从相应颜色链表中选出一个空闲页。通过适当地为页着色，操作系统确保整个页被缓存在最佳存储体中。

回顾一下，我们采用具有 S-NUCA 策略的大型共享缓存，以及将一个完整操作系统页映射到一个存储体的索引机制。预计未来的共享缓存将分布在芯片上，每个核心与一个或多个存储体共置（图 1.6）。这种布局结合适当的页着色策略，还可以为许多工作负载模拟私有缓存组织。假设运行在不同核心上的线程主要访问私有数据，即核心所触及的页集合在很大程度上不相交，那么操作系统可以对页进行着色，使它们被放置在访问核心的本地存储体中。因此，每个 L2 存储体都可以作为其对应核心的私有缓存。事实上，这种设计可以胜过采用传统私有 L2 缓存的组织（图 1.3），因为在 L2 缺失时，请求可以直接发送到下一级层次结构，而无需检查其他私有 L2 缓存的内容。在处理共享页时，这种设计确实偏离了私有缓存组织。共享页在共享 L2 缓存中将有唯一实例，并且恰好驻留在一个存储体中。因此，与私有缓存设置不同，数据块不能放置在每个访问该块的核心的本地存储体中。如果这些块主要是只读的，这可能对性能不利。

如上所述，一种明显的操作系统页着色策略——也曾用于 cc-NUMA 多处理器 [36, 38, 39, 40]——是首次触及颜色分配。在该策略中，页被分配一种颜色，使其被放置在最接近首次触及该页的核心的缓存存储体中。其前提是对该页的大多数后续访问

也将来自同一核心。对于主要由单线程程序组成的工作负载，这当然是一个有效的前提。对于大多数工作集恰好是每个线程私有的多线程应用，这也是一个有效的前提。

让我们首先考虑由单线程程序组成的工作负载。假设每个块仅被一个核心访问，那么首次触碰着色分配允许每个 L2 访问由本地 bank 服务，从而降低 L2 访问时间。但相应地，每个核心

仅享有总共享 L2 容量的一小部分，因为其所有数据都被导向单个 bank。这种行为与私有缓存组织非常相似。为了超越私有缓存组织，必须增强页着色策略，使得每个核心在需要时能够拥有更大比例的共享 L2，而不会导致较长的 L2 延迟。

Cho 和 Jin[41] 提出了页扩展策略，增强了基线首次触碰页着色策略。当一个缓存 bank 经历高“压力”时，来自该核心的后续页请求被分配颜色，将页映射到相邻 bank。这允许一个核心拥有超过其公平份额的 L2 缓存空间，这是共享缓存具有更高命中率的根本原因。由于这些策略试图将核心的工作集本地化到相邻和邻近的 bank 中，L2 访问时间将继续相对较短。Cho 和 Jin 提出使用基于布隆过滤器的计数器来跟踪核心最近访问的唯一页数；计数器值作为缓存压力的度量。Awasthi 等人 [42] 形式化并评估了 Cho 和 Jin 的页扩展方法。本质上，每个页请求被映射到一个 bank，该 bank 最小化一个估计缓存压力和 bank 与请求核心之间距离的数学函数。结果，缓存命中率接近基线组交错共享缓存的命中率，并且大多数 L2 请求由相邻或附近的 bank 服务。这样的设计有望在几乎所有由单线程程序组成的工作负载上超越私有缓存组织（随后提到一些注意事项）。

在多线程工作负载中，理想情况下每个页应放置在代表对该页所有请求的重心的 bank 中。首次触碰策略（无论是否带有页扩展）可能最终将共享页放置在次优的 bank 中。这可以通过随时间迁移页来补救，这种方法将在稍后讨论。没有这种动态迁移，共享页的行为很可能与基线组交错共享 L2 缓存中的行为非常相似。首次触碰策略将继续为此类工作负载中的许多私有页提供低访问时间。

总之，在考虑性能和实现复杂性时，使用 S-NUCA 共享缓存结合操作系统页着色策略可能是一个引人注目的设计点。它提供了 D-NUCA 策略的大部分好处，而没有太多复杂性开销；因此，它代表了原始组交错 S-NUCA 和 D-NUCA 之间的一个有利设计点。此外，在 S-NUCA 共享缓存上提出的基于操作系统的方法提供了私有缓存相对于共享缓存的最大优势：快速访问私有数据块。页扩展策略还允许缓存满足基线组交错共享缓存的高命中率潜力。但请注意，在处理某些多线程应用程序时，私有缓存组织可以超越所提出的基于操作系统的方法：在前者中，共享块可能经常在本地私有缓存中找到，而在后者中，共享块将只有一个（可能遥远的）驻留地。

在后续工作中，Jin 和 Cho[43] 利用编译器提示来指导操作系统选择接近最优的页颜色。在编译时，程序使用测试输入执行，并收集每个核心每页的访问计数。页被聚类成组，并且该组的访问模式

与少数几种众所周知的访问模式进行匹配。基于访问模式，二进制文件中包含一个提示，以指导操作系统将每个页映射到预期成为该页主要访问者的 tile。目前尚不清

楚编译器提示是否比首次触碰提供的信息显著更多。一些页被标记为被所有核心平等共享，并且通过修改该页的缓存索引函数，这些页被分散到所有 tile 中。Jin 和 Cho 还为共享页的复制提出了初步理由，尽管未考虑这些复制页的一致性问题。

Marty 和 Hill[44] 的一篇论文提到，如果每次 L1 缓存缺失都必须联系远程目录，多核芯片中的一致性开销可能很高。当核心跨虚拟机（VM）分配且有效数据很可能在附近的 tile 中找到时，这尤其浪费。Marty 和 Hill 通过两级一致性协议解决了这个问题，其中第一级协议在 VM 使用的核心内存储目录。该目录中的缺失会调用更昂贵的第二级协议。这是解决 S-NUCA 缓存中局部性问题的另一种方法。然而，如 Cho 和 Jin[41] 所建议的，使用页着色将 largely 解决局部性问题，且几乎不需要硬件支持，也不需要两级层次一致性协议。换句话说，如果采用首次触碰页着色，大多数页将缓存在属于 VM 的 tile 中；因此，访问 LLC bank 和强制执行基于目录的一致性的开销不会很高。

Madan 等人 [9] 也采用了页着色来减少 3D 可重构 S-NUCA 缓存中的通信开销。这项工作在第 5.4 节中更详细地描述。

Awasthi 等人和 Chaudhuri，HPCA'09

Cho 和 Jin 的首次触碰页着色策略结合页扩展在平衡私有页的延迟和容量需求方面非常有效。但在为共享页寻找最佳归属 bank 方面不太有效。如果线程迁移并将工作集留在原始 bank 中，它也不有效。根据程序如何经历不同阶段，线程也可能在其本地 bank 中缓存相对不活跃的页，并将更活跃的页“扩展”到相邻 bank，导致数据到 bank 的次优映射。其他动态条件（如程序完成或启动）也可能导致缓存 bank 使用波动，使先前的页映射决策变得次优。为了处理上述弱点，我们可能需要动态策略来跟踪缓存使用情况，并在页的“最佳”归属 bank 与首次触碰时估计的不同时影响页迁移。这种迁移方法将开始类似于已经讨论过的 D-NUCA 策略。为了实现这一点，必须在 DRAM 中将页从其当前物理地址复制到具有适当最佳颜色的新物理地址。此外，必须从旧缓存 bank 和所有核心的 TLB 中清除旧物理页的所有痕迹。

Awasthi 等人 [42] 指出，迁移是有益的，因为首次接触决策往往会随时间推移而变得次优，尤其是对于具有大量共享页面的多线程应用。为了避免在 DRAM 中进行昂贵的页面复制，他们提出了一种机制，在芯片上引入另一层

间接层。考虑一个页面，最初放置在物理地址 A 并映射到某个 bank X。为了将该页面移动到不同的 bank Y，片上 TLB 被更新为通过新的物理地址 A' 来引用该页面。但该页面继续驻留在 DRAM 中的同一物理地址 A 处；它只是在芯片上（L1 和 L2）有了一个新名称。如果在 L2 中未找到某个块，并且必须离开芯片访问内存，则向 DRAM 发出对原始物理地址 A 的请求。当一个页面在芯片上从 A 重命名为 A' 时，影子地址区域用于 A'，以确保新名称尚未被另一个页面使用。影子地址区域表示在 DRAM 中实际不存在的可寻址内存部分；例如，在具有 2 GB DRAM 的 32 位系统中，地址范围 2-4 GB 就是影子地址空间。选择 A' 使其具有适当的新页面颜色，并且原始页面颜色也被编码到 A' 中。这使得从 A' 重建 A 变得容易，当发生 L2 未命中并且必须向 DRAM 发出请求时，需要执行此操作。这种片上重命名机制的主要开销是一个相对较大的片

上结构，称为转换表。它跟踪每个页面迁移（$A \rightarrow A'$），类似于一个大型的片上二级 TLB。请注意，每次页面迁移仍然需要从缓存和 TLB 中刷新原始页面的内容。

除了这种低成本的迁移方案外，Awasthi 等人还引入了操作系统策略，定期检查硬件计数器并执行迁移。基于缓存使用情况和命中率，bank（页面颜色）被分类为接受者（需要更多缓存空间）和捐赠者（可以将一些缓存空间让给其他程序）。需要更多缓存空间的程序允许将其新页面映射到捐赠者颜色；压力过大的接受者颜色允许将其部分页面移动到捐赠者颜色。捐赠者颜色通过计算一个考虑缓存使用情况以及核心与 bank 之间距离的函数来选择。

在同一会议上的并行工作中，Chaudhuri 通过 PageNUCA 设计提出了类似的论点 [45]。他也表明，L2 内的页面迁移对于多线程和多程序工作负载都非常有益。为了在 L2 内迁移页面，操作系统分配的地址 (P) 被转换为具有适当页面颜色的不同地址 (P')。由于不使用影子地址，另一个虚拟页面可能被映射到 P'；这个页面必须被交换到地址 P。由于 L1 缓存和主存继续使用旧映射，因此需要在核心处（dL1Map 和 iL1Map）以及 L2 处（正向和反向 L2Maps）使用表来在 P 和 P' 之间来回转换。提出了两种技术来识别需要迁移的页面：（i）一种基于硬件的技术，使用页面访问计数器表（PACT）动态跟踪访问模式，以及（ii）一种手动代码插桩过程，在程序开始时将页面亲和性指定给核心。当识别出要迁移的页面时，L2 被锁定，同时块在源 bank 和目标 bank 之间显式复制，并且各种映射表被更新（这与 Awasthi 等人的技术不同，后者使原始页面块无效，并在新页面中的块被访问时按需填充新 bank）。总体而言，为了确定需要迁移的页面并在迁移时保持各种表更新，引入了相当多的复杂性。Chaudhuri 声称页面粒度适合迁移（与 D-NUCA 的块粒度相比），因为（i）页面迁移本质上对该页面中的多个块执行预取迁移，（ii）批量传输更高效，因为它可以在可扩展互连上高效地流水线化，以及（iii）所有簿记表中的开销都减少了。

在现实世界中，这种迁移方案很可能需要适应不断变化的工作负载和操作条件。这些迁移方案试图将先前提出的 Cho 和 Jin 方法（具有首次接触页面着色的 S-NUCA）进一步推向 D-NUCA。迁移确实涉及一些开销，尚不清楚这些开销是否会被容忍。Awasthi 等人和 Chaudhuri 的论文试图减轻其中一项开销（DRAM 复制），但最终引入了不同的非平凡开销（转换表、L1/L2Maps 等）。因此，对优雅迁移方案的探索仍在继续。

反应式 NUCA（R-NUCA），Hardavellas 等人，ISCA'09

在最近的工作中，Hardavellas 等人 [3] 提出了一种新颖的 NUCA 架构，该架构依赖于操作系统对大型共享 L2 缓存中页面的管理，并且不需要复杂的搜索机制。虽然这与 Cho 和 Jin 的目标相似，但 Hardavellas 等人采用了不同的方法，其核心是对页面的性质进行分类并区别对待每一类页面。每个核心也被允许拥有自己的索引函数，使每个核心能够对共享 L2 缓存有不同的视图——这允许私有页面在 bank 之间迁移，而无需在 DRAM 中进行页面复制或复杂的硬件结构。除了高效处理共享页面和私有页面外，它还促进了各种粒度的复制。虽然没有明确关注迁移和竞争线程之间的缓存容量分配，但他们的方法可以扩展以处理这些问题。因此，它代表了当今最佳的设计点之一。上

述特性将在后续讨论中阐明。

作者首先观察到，页面通常可以分为三类。

（i）共享指令页：这些页面通常由多线程应用程序中的所有线程以只读模式访问。此类页面可以在每个 L2 缓存体中复制，而无需在共享 L2 内使用一致性机制。为了防止复制导致的高缓存压力，一个页面在由 I 个缓存体组成的簇中复制一次，其中 I 可以在运行时确定（在论文 [3] 中 I 假定为 4）。

（ii）私有数据页：这些页面为每个线程私有，必须仅放置在本地缓存体中。为了处理异构线程间的空间容量分配，可以修改策略以允许私有页面放置在 P 个本地/附近缓存体之一（具体的索引机制将很快描述），其中 P 可以为每个线程在运行时确定（在论文中所有线程的 P 设置为 1）。

（iii）共享数据页：对于服务器工作负载，数据页通常由所有线程共享，并表现出读写操作。因此，每个块必须在单个缓存体中实例化（以避免需要一致性），并且页面可以均匀分布在 S 个缓存体中（以最小化缺失率和

热点，并且因为难以估计每个页面的最佳缓存体）。在论文中 S 设置为 16（核心数）。

在我们考虑块如何放置在缓存中之前，让我们首先描述将页面分类为上述三类的过程。该过程完全基于操作系统，并且需要在页表和 TLB 中进行记录。在首次访问时，页面被标记为指令页或私有数据页，取决于它源自 L1 指令缓存还是数据缓存。如果某个核心发生数据 TLB 缺失，并且操作系统识别出该页面在其他地方被标记为“数据私有”，则将该页面重新分类为“数据共享”，并将该页面从其他核心的 TLB 和 L2 缓存体中清除。

接下来，考虑缓存体中的数据放置。作为示例，假设缓存体组织为大小为 n 的簇。假设每个簇是不相交的。一旦页面分配给某个簇，它根据页面地址位放置在 n 个缓存体之一中。该簇内的所有核心都识别该簇中缓存体的统一编号方案，因此给定一个地址，任何核心都可以轻松确定存储该页面的缓存体。为了最大化数据与请求核心的邻近性并平衡各簇之间的负载，Hardavellas 等人使用了一种巧妙的旋转交错机制。假设每个核心是以其为中心的簇的一部分（从而确保该核心的数据放置在其附近）。显然，簇现在是重叠的（除非簇大小为 1）。每个缓存体被赋予 0 到 n-1 之间的编号（称为旋转 ID），使得每个簇最终拥有具有唯一旋转 ID 的缓存体（图 2.3 显示了 n=4 的示例）。由于一个缓存体现在属于四个不同的簇，其负载大致是平均负载

在这四个簇中（从而导致缓存体之间更好的负载分布）。由于每个缓存体具有固定的旋转 ID，所有核心再次识别缓存体的统一编号方案，并且页面请求可以轻松定向到适当的缓存体。

如上所述，采用了三种不同大小的逻辑簇——共享数据页假设缓存体组织为大小为 16 的簇，指令页假设簇大小为 4，私有数据页假设簇大小为 1。因此，如果某个核心发出对私有数据页的请求，

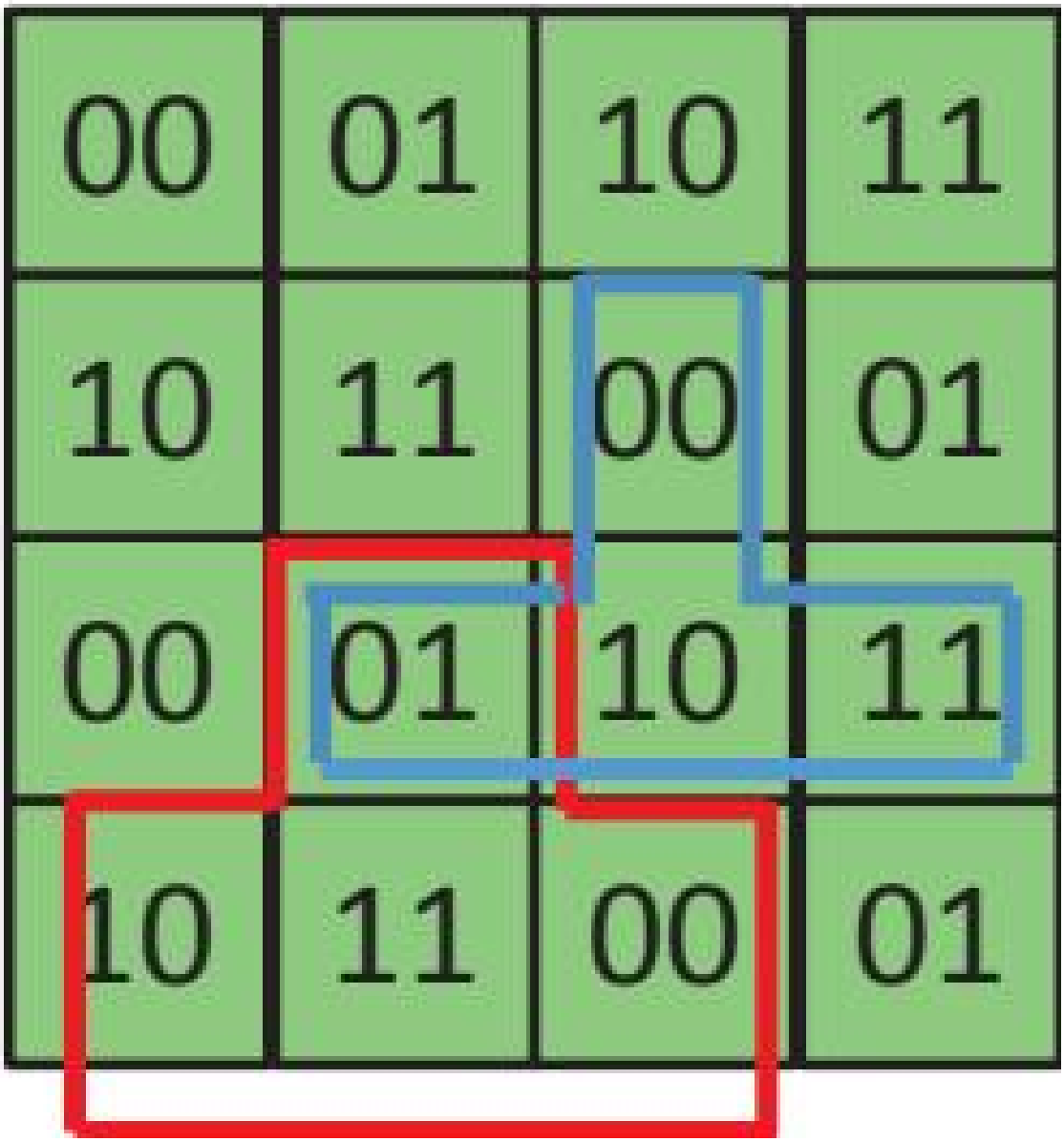


图 2.3 具有旋转 ID 编号系统的 R-NUCA 缓存，n=4，以及两个示例簇（红色和蓝色）[3]。

它知道该页面放置在本地簇中；由于这是大小为 1 的簇，请求被发送到本地缓存体。如果某个核心发出对指令页的请求，它知道该页面放置在其大小为 4 的本地簇中的一个缓存体中；页面地址的两位指定该缓存体（这是缓存体的旋转 ID），请求被发送到那里。注意，该缓存体在其簇中的其他核心也将以相同方式访问此指令页（所有这些核心通过相同的旋转 ID 引用该缓存体）；因此该页面基本上由 4 个核心共享。该页面也可能存在于其他 3 个缓存体中（在 16 核处理器中），从而导致受控复制。簇大小指定了指令页的复制（和共享）程度。如果某个核心发出对共享数据页的请求，它知道该页面在组成大小为 16 的簇的缓存体之一中；页面地址的四位用于识别该缓存体并将请求发送到那里。具有该缓存体在其簇中的所有核心使用相同的过程。注意，每个核心都知道页面所属的类别（页面类别存储在 TLB 中），并且可以轻松地以适当的方式索引到 L2 缓存体。当页面从“数据私有”重新分类为“数据共享”时，由于簇大

小的变化，页面将以不同的方式索引到缓存中。因此，页面的旧内容必须从单个缓存体中清除并在新的缓存体中恢复（DRAM 物理地址保持不变，因此不需要 DRAM 复制）。

R-NUCA 架构具有简单的索引，允许受控复制，并适当地处理私有和共享页面。给操作系统带来的额外负担是在页表和 TLB 中进行记录以跟踪页面分类；不需要基于操作系统的页面着色，因为页面分类和相应的簇大小决定了页面的放置位置。没有引入辅助硬件结构，因为没有显式迁移（除了页面从“数据私有”到“数据共享”的强制移动）。一旦页面被指定为“数据共享”，迁移被认为是不值得的，因为观察到共享页面大致由所有（16 个）核心平等共享。虽然作者没有评估跨核心的异构缓存容量分配，但这可能通过允许每个核心具有不同的 P 值而不是默认大小 1 来实现。然而，每次 P 改变时，所有私有页面将不得不被清除并迁移到它们的新位置。由于 DRAM 物理地址从未改变，同样不需要 DRAM 复制。尽管动态估计 P 和实施迁移存在开销，但该设计没有 Awasthi 等人 [42] 和 Chaudhuri[45] 工作中存在的其他迁移开销（用于转换的各种表）。

Lin 等人，HPCA’08 和 SC’09

在最近的一项研究中，Lin 等人 [46] 在配备双核 Xeon 的真实系统中评估了页面着色对共享缓存管理的影响。这项研究验证了先前基于模拟的研究，并揭示了一些新的观察结果。由于 Xeon 不支持非均匀缓存访问，页面着色并非用于改善数据邻近性，而仅仅是为了在两个竞争的核心/程序之间强制实施共享缓存分区。评估了一种静态策略，其中工作集大小事先已知；同时也考虑了一种动态策略，其中每个时间段的运行时测量指导页面颜色向核心的增量分配。性能提升平均

在 10% 左右。动态策略在页面被重新分配给不同颜色时需要在 DRAM 中进行页面迁移。Lin 等人指出这可能是一个问题，并实现了一种惰性页面迁移机制，将页面迁移带来的性能下降平均限制在仅 2%。如果迁移频繁，或者核心更多且对内存系统的需求更大，这些开销可能会更高。他们还指出，当程序的缓存分配的一部分被给予共同调度的程序时，程序的性能有时会提高。这是因为整体更低的 L2 缺失率使得第一个程序能够从更短的内存排队延迟中受益。

Lin 等人 [47] 在 HPCA’08 工作的基础上，提出了一种利用小型硬件结构和操作系统控制策略的解决方案。假设有一个大型共享 UCA L2 缓存，并将缓存划分为 C 种颜色。一个物理页面完全映射到其中一种颜色。为了动态控制跨线程的容量分配，页面的颜色可以动态更改；页面的物理地址不变，只是用于查找 L2 缓存的索引位发生变化。L2 之前有一个区域映射表，存储每个页面的新缓存颜色。显然，这可能导致一个很大的表；因此，颜色以区域为更粗的粒度重新分配（在 [47] 中约为 16 MB）。由于新颜色也可以存储在 TLB 中，因此不必在每次访问时查找该表。因此，区域映射表在精神上与 Awasthi 等人 [42] 的转换表非常相似，但由于数据以更粗的粒度重新映射，其复杂性可控。使用另一级间接寻址并将硬件帮助与软件策略相结合的整体方法也与 Awasthi 等人 [42] 和 Chaudhuri [45] 的方法相似。同样，页面的移动不需要在 DRAM 中

复制，但确实需要缓存刷新和 TLB 击落。这项工作的主要进步是使用粗粒度的数据到缓存颜色的重新映射，以实现具有良好加速比和极低复杂性的解决方案。重新映射设施必须与良好的硬件剖析单元和软件策略相结合，以计算区域到颜色的最优分配。Lin 等人采用的剖析单元使用影子标签来估计缺失率与相联度的函数关系（影子标签将在第 3.1.2 节中更详细地讨论）。

2.2 一组私有缓存的数据管理

讨论了共享 LLC 的创新之后，我们现在将注意力转向 LLC（L2 或 L3）由逻辑和物理上对每个核心私有的存储体组成的组织。回顾第 1.1 节中的讨论，私有缓存有助于将核心最流行的数据保持在最近的缓存存储体中，但由于数据复制和每个核心缓存空间的静态分配，它们通常提供较差的命中率。私有 LLC 中的缺失通常还需要在请求发送到片外之前查找其他片上私有缓存。大多数针对私有 LLC 的创新试图改善其负面特性（低命中率），同时保持其正面属性（命中时的低平均延迟）。然而，其中一个负面特性（所需复杂结构

用于私有缓存之间的一致性）在大多数针对私有 LLC 提出的创新中仍然存在。

协作缓存，Chang 和 Sohi，ISCA’06 以及 Speight 等人，ISCA’05

私有缓存组织往往具有较低的命中率，因为每个核心被静态分配固定大小的私有缓存，并且由于数据块的复制。Chang 和 Sohi [48] 试图通过允许一个缓存存储体容纳另一个缓存存储体逐出的块来放宽第一个限制。第二个限制通过修改替换策略以倾向于逐出复制的块来缓解。他们的基线实现假设私有 L2 之间采用基于目录的一致性协议。在 L2 缓存缺失时，先联系中央目录，然后再将请求转发到片外。该目录本质上复制了所有片上缓存结构的 L1 和 L2 标签（类似于第 1.1 节中的描述），并且可以识别片上是否存在所请求块的脏副本。Chang 和 Sohi 为片上私有 LLC 提出了三项修改。

（i）缓存到缓存共享：第一项修改是众所周知的，但通常不被传统的多芯片多处理器采用。在缓存缺失时，一致性协议可以允许兄弟缓存响应数据，如果这比从下一级层次结构服务请求更快。在传统的多处理器中，这通常不成立，因为兄弟缓存和下一级都需要片外和网络访问。但在现代多核中，通过片上网络访问同一芯片上的兄弟缓存比片外访问下一级层次结构要快得多。因此，Chang 和 Sohi 提倡使用缓存到缓存共享，由目录负责识别可以响应的兄弟缓存并将请求转发给它。

（二）驱逐副本块：第二项修改针对私有缓存的替换策略。当需要驱逐一个块时，替换策略首先尝试驱逐一个可能在芯片其他位置存在副本的块，从而试图降低私有缓存中块复制的程度。这一操作并非在每次驱逐时都执行，而是以概率方式对所有驱逐中的 N% 执行，其中 N 可根据工作负载确定。为了实现这一改动，每个缓存必须掌握其块是副本还是单件的信息。目录负责收集这些信息，并在块状态每次变化时将其传播给各缓存。显然，这会带来不小的片上通信开销，尤其是因为缓存不能再静默地驱逐块了。

（三）溢出被驱逐的块：如果一个单件块必须从私有缓存中被驱逐，则会进一步尝试将其保留在片上。以给定的概率 M，该块被移交给一个兄弟私有缓存。兄弟缓存接纳该块（希望在此过程中驱逐一个副本），并将其标记为 LRU。这为块争取了一些时间，以期被某个核心访问，从而延长其在片上的生命周期。块会附带一些位，用于限制此类块可被溢出到兄弟缓存的次数（Chang 和 Sohi 在评估中采用了一次性转发）。一如既往，任何块的移动都必须通知目录。

这三项修改合称为协作缓存（Cooperative Caching）。与当时未经优化的共享和私有基线相比，它可以带来性能提升，而这种提升往往与 N 和 M 密切相关。遗憾的是，如上所述，随之而来的目录开销不容小觑。

Chang 和 Sohi 的协作缓存框架之前，Speight 等人 [49] 的工作也提出了片上私有缓存之间的一种协作形式。它包含了溢出和缓存到缓存共享两个方面。Speight 等人假设基线处理器具有私有 L2 缓存和一个作为牺牲缓存的共享片外 L3。该平台在一致性协议上有各种微妙之处，我们在此不予赘述。与本讨论相关的是，L2 与 L3 之间的一致性通过基于广播的监听协议来维护。由于访问其他片上私有 L2 比访问 L3 或内存更快，因此鼓励缓存到缓存共享。

当一个块（干净或脏）从私有 L2 中被驱逐时，会在共享总线上传播一个写回，以便该块最终能写入片外牺牲 L3。允许另一个 L2 托管该块，从而避免片外访问。Speight 等人将这一过程称为由其他（宿主）L2 进行的嗅探（snarfing）（在本书中通常称为溢出）。如果宿主 L2 有空闲空间，即新块只替换无效或共享的行，则允许嗅探。此外，仅当该块此前表现出重用潜力时才允许这样做。这是通过为每个 L2 缓存维护一个表来估计的，该表跟踪 L2 最近驱逐的所有行以及它们是否被再次重用。这两个事件都通过监听总线来观察。当这样的行再次被驱逐时，在广播期间会设置一个位，允许其他 L2 缓存嗅探该块。如果多个 L2 缓存自愿作为宿主，则只指定一个作为宿主。一致性协议在总线上有一个中心实体，负责收集监听响应并决定哪个缓存托管被驱逐的行。

Speight 等人 [49] 和 Chang 与 Sohi [48] 的方案在精神上相似。实现上的差异主要源于前者使用监听，而后者使用目录。为了触发溢出，Chang 和 Sohi 努力识别单件（甚至在替换策略中使用此信息），而 Speight 等人则使用表来预测重用潜力。

ASR，Beckmann 等人，MICRO’06

Chang 和 Sohi 的协作缓存机制使用运行时参数 N 和 M 来支配副本驱逐和单件溢出的程度。但他们没有考虑在运行时估计最优的 N 或 M。Beckmann 等人 [20] 引入了运行时方案来选择合适的复制级别，这些方案可应用于任何已提出的复制机制（牺牲复制、CMP-NuRAPID、协作缓存）。在许多缓存创新中，我们必须回答如下问题：当一个块从 L1 中被驱逐时，我们必须选择是否将其保留在本地 L2 体中（牺牲复制）；或者，当从 L2 中驱逐一个块时，我们必须选择是驱逐一个 LRU 单件还是一个非 LRU 副本

（协作缓存）。Beckmann 等人引入了选择性概率复制（Selective Probabilistic Replication，SPR），如果伪随机数生成器产生的值小于 R，则倾向于创建副本。他们用一种机

制来增强这种简单的概率方案，以确定 R 的最优值，称为自适应选择性复制（Adaptive Selective Replication，ASR）。该论文的关键贡献在于设计了一些结构，有助于估计如果 R 增大或减小，本地命中率和内存访问将如何变化。无论我们考虑增大还是减小 R，都有相关的成本和收益，以下四种机制有助于这种估计：

- 增加复制的收益：影子标签结构在假设更高复制级别的情况下跟踪缓存的内容，从而估计这将带来的额外副本命中。

- 增加复制的成本：跟踪对缓存中 LRU 块的命中，以确定增加复制级别可能产生的新缺失。

- 减少复制的收益：另一个结构跟踪最近被驱逐的块，以估计高复制是否正在驱逐可能有用的块。

- 降低复制度的代价：缓存会跟踪对可能未在较低复制级别创建的副本的命中，从而估算迁移到较低复制级别的代价。

上述估算有助于判断是否必须增加或减少 R，以在更高的本地命中率和更高的有效容量之间找到最佳平衡。

分布式与弹性协作缓存，Herrero 等，PACT'08 和 ISCA'10

Chang 和 Sohi 的协作缓存方案 [48] 的一个重要缺点是，需要一个复杂的集中式目录，每次 LLC 缺失时都可能被多次访问。这个集中式目录复制了每个片上 L1 和 L2 的标签；它在大多数块移动时更新；它负责识别单副本/副本，并识别缓存协作的机会。在查找一个块时，必须搜索大量标签，因为该块可能位于众多 L1 和 L2 缓存的任意一路中。

Herrero 等人的分布式协作缓存（DCC）方案 [50] 试图缓解其中一些实现复杂性。首先，他们提出使用传统的组相联全局标签阵列来跟踪所有片上块。该标签阵列可以具有低相联度和大量组，与 Chang 和 Sohi 的协作缓存中维护的复制标签几乎没有相似之处。这个全局标签阵列预计会跟踪每个片上块的共享状态（每个标签为每个私有 LLC 一位）。由于该标签阵列的相联度低，某些块的标签可能不得不从该标签阵列中逐出，即使它可能仍存在于某个片上缓存中——为了防止这种不一致，该块会从所有片上缓存中逐出。希望通过在标签阵列中设置许多组，可以使这种事件相对不频繁。这种逐出策略是对全局

标签阵列（由所有核心共享）强制执行 LRU 的结果，因此可以比协作缓存中本地做出的逐出决策产生更好的命中率。新组织在能效上要高得多，因为每次访问只需查找有限数量的标签。

Herrero 等人还提出，将该标签阵列分块并分布在芯片上，采用将连续组放置在相邻块中的交错方式。这分散了网络负载，并消除了协作缓存中发现的集中式瓶颈。

请注意，所提出的解决方案并未缓解协作缓存的所有实现低效问题。每次块逐出（甚至干净块逐出）时都必须交换若干消息，以更新标签阵列并修改单副本/副本状态。

Herrero 等人通过一种在私有块和共享块之间分配缓存块的机制 [51] 增强了他们

的设计。每个缓存块跟踪私有路中 LRU 块的命中次数以及共享路中 LRU 块的命中次数。在每个周期间隔，每个缓存块通知其他核心已分配给共享路的路数。当块被逐出时，如果保留该块预计具有高效用，则按轮询顺序将其溢出到全局共享路之一。

动态溢出-接收，Qureshi，HPCA'09

在最近的工作中，Qureshi 提出了一种基于协作缓存框架的优雅设计。除了 ASR 策略 [20] 之外，迄今为止提出的大多数协作缓存策略 [48, 49, 50] 都没有估算将逐出行溢出到相邻缓存的成本/收益。更值得注意的是，相邻的接收缓存甚至没有选择权，必须接受强加给它的块。Beckmann 等人的 ASR 策略 [20] 试图估算这种成本/收益并设置概率阈值以鼓励或抑制溢出。Qureshi 提出了一种简化成本/收益估算的解决方案，并且能够感知缓存接收其他缓存逐出块的能力。

在 Qureshi 的动态溢出-接收（DSR）设计 [52] 中，每个私有缓存被指定为溢出者或接收者（但不能两者兼有）。概率阈值被消除。唯一允许的协作是将块从溢出者缓存溢出到接收者缓存。溢出者/接收者的指定以最大化片上命中率的方式进行。每个私有缓存基于一种称为组决斗 [53] 的技术（在第 3.1.1 节中进一步解释）独立地将自己指定为溢出者或接收者。每个缓存始终允许对其少数组（例如，1024 组中的 32 组）进行溢出，并始终允许对其少数组进行接收；这些组被称为“始终溢出”组和“始终接收”组。每个缓存一个计数器跟踪哪组产生更多缺失，并动态地为缓存中所有其他组选择更好的策略（溢出或接收）。请注意，该计数器还跟踪这些组在所有其他缓存中的缺失（在基于侦听的协议中，通过关注总线很容易计算）。因此，缓存的溢出者/接收者状态也是与其共同调度的其他应用的函数。每个私有缓存使用不同的决斗组来决定其溢出者/接收者指定。

上述过程开销非常低，每个缓存仅需一个计数器。Chang 和 Sohi 的一次机会转发方案需要每个块一位来表示是否已使用一次机会。不再需要该位，因为接收者缓存不允许溢出，并且块几乎总是只溢出一次。接收者缓存可能随时间变为溢出者缓存，从而允许块多次溢出。在决斗组上收集的统计信息可以估算应用是否因其接收者状态而受损，并在必要时通过切换到溢出者状态来实施 QoS。

CloudCache 与 StimulusCache，Lee 等人，HPCA'11 与 HPCA'10

Lee 等人 [54] 假设处理器的 LLC 由私有缓存组成。他们提出 CloudCache 设计，其中每个私有缓存的边界在设计时并不固定。一个核心可能使用多个缓存 bank 中的路来组成其私有缓存。根据这些路与核心的距离，形成优先级链，同时也兼作替换决策所用的优先级列表。当块被访问并获得更高优先级时，块会在 bank 之间移动以反映优先级顺序。因此，分配给一个核心的路（可能分布在多个 bank 中）本质上形成了 D-NUCA 组织。块搜索通过向分配给核心的所有路广播请求来完成。如果另一个核心通过目录访问同一个块，则广播被禁用。每个核心为少量采样组维护额外 32 路的标签。这允许核心确定获得更多路的效用。定期由集中式代理收集此信息，并在核心之间重新分配路。StimulusCache 设计 [55] 的大部分内容已被 CloudCache 设计 [54] 所涵盖。StimulusCache 设计的动机是观察到核心在制造过程中比缓存 bank 更可能出现故

障。该设计试图将属于故障核心的“多余”缓存 bank 分配给其他正常核心。除了私有 LLC 的常见复杂性（通过目录查找 LLC 块）之外，这些设计还承受 D-NUCA 带来的额外复杂性（块搜索和块迁移）。如前所述，Cho 和 Jin [41]、Awasthi 等人 [42] 以及 Chaudhuri [45] 的工作试图在共享 LLC 的背景下类似地在竞争核心之间分配缓存空间。

MorphCache，Srikantaiah 等人，HPCA’11

与 CloudCache 类似，MorphCache 设计 [56] 也从私有缓存组织开始，并允许重新配置以形成更大的缓存。根据对工作集需求的分析，私有 L2 或 L3 缓存切片要么合并形成更大的共享缓存切片，要么将先前合并的切片拆分为更小的私有切片。因此，每个缓存切片（L2 或 L3）既可以是私有缓存，也可以是共享于 2-16 个核心的更大缓存的一部分。如果 L2 缓存切片合并，相应的 L3 缓存切片也会合并。当相邻切片的利用率不平衡或高利用率切片处理相同共享数据时，会触发合并。合并后的缓存切片具有更高的 UCA 延迟。如下文所述，一个

优化的共享缓存基线通常能提供可重构私有缓存设计中许多单独选项所提供的优良特性。

2.3 讨论

我们在表 1.1 中比较了通用共享缓存和私有缓存的特性。最近大多数关于私有缓存组织的论文继续引用通用共享缓存的缺点，同时提倡私有缓存组织。这类论点往往具有误导性，因为它们对近年来在很大程度上缓解了共享缓存主要缺点的若干创新视而不见。例如，有几篇论文提到共享缓存效率低下，因为平均访问延迟高，而且访问共享缓存所用的网络负载更高。这些论点适用于早期采用组交错映射或实现物理连续共享缓存的共享缓存设计。它们不再适用于最近提出的设计。即使是一个简单的设计，采用分布式分块共享 S-NUCA 缓存、页到 bank 映射和首次触碰操作系统页着色，也能在很大程度上确保大多数 LLC 请求不离开本地 tile。

同样，关于共享缓存的论文经常强调私有缓存命中率差，却没有给予协作缓存技术应有的认可。此外，只有在 LLC 为包含式时，才能实现共享 LLC 中轻松查找数据的好处。

尽管如此，私有缓存组织在克服其缺陷方面可能面临更陡峭的攀登。一个主要缺陷是定位可能驻留在其他私有缓存中的行的复杂性。私有缓存在核心数较少的商业实现中已被使用；使用广播总线有助于查找数据。然而，随着核心数量的增加，这种方法不能很好地扩展。需要片上目录，且不易维护。关键区别在于：在共享 S-NUCA 缓存中，我们确切知道在哪里找到块；在私有缓存组织中，我们需要“搜索”块，因为它可能在任何地方。私有缓存当然将继续在现代多核中常见；如果它们最终由能够处理私有缓存之间一致性的片上共享包含式末级缓存作为后备，它们将最为有效。此类私有缓存可能不需要协作，因为被逐出的行无论如何都会在共享 LLC 中找到。因此，我们认为，第 2.1 节和下一章所讨论的那种共享缓存创新，可能具有更长期的影响。

第三章　影响缓存命中率的策略

上一章关注的是模糊共享缓存和私有缓存之间界限、并试图将数据拉近请求核心的策略。本章讨论的大多数论文对缓存访问延迟的非均匀特性不敏感/不可知。因此，这里的许多讨论将集中在命中率上。本章大部分内容将假设共享 LLC，并考察用于块插入、替换和获取的各种策略。这项工作与过去几十年大量缓存文献的区别在于强调多线程或多程序工作集。部分工作可以广泛归类为“缓存分区”，关注的指标通常是缓存命中率和服务质量（QoS）。虽然第 2 章讨论了一些部分属于缓存分区范畴的论文 [20, 22, 26, 27, 41, 42, 46, 47, 52]，但该章更侧重于非均匀延迟 bank 的集合（私有或共享）以及改善邻近性的数据放置策略。我们将继续提及一些先前讨论过的论文，以阐明它们对缓存分区文献的贡献。

3.1　面向吞吐量与服务质量的高速缓存划分

3.1.1　引言

基线平台　大多数关于缓存划分的论文都试图将组相联单片共享缓存的各条路（way）分配给多个竞争的线程或程序。对于小规模多核而言，这是一个合理的模型。如前所述，大规模多核很可能采用分块（tiled）结构，而共享的 LLC 也很可能是分布式的，比如每个核一个缓存体（bank）。当共享缓存是分布式的时候，我们可以分布组（set），也可以分布路，或者两者都分布。如果分布的是路，那么一个给定的缓存行可能驻留在多个缓存体之一，这就需要块搜索机制。这就是复杂的 D-NUCA 模型。正如第 2 章所论述的，一种更具可扩展性的模型采用 S-NUCA 架构，并借助基于操作系统的页着色来改善数据邻近性。S-NUCA 采用组划分，一个组的所有路通常都包含在单个缓存体中。缓存划分可以通过将组分配给核并结合缺失率感知的页着色来实现 [41, 42]。这一点已在 2.1.3 节讨论过，本节末尾也将简要总结。在这种模型下，一个给定的缓存体通常第 3 章

容纳某个应用程序大工作集的一部分，或者某个应用程序的小工作集，以及另外几个大型应用程序溢出（spill）的工作集。目前还没有人研究过针对后一种情况的路划分（或组划分）及其与页着色策略的交互。预期收益是否很高尚不清楚。即使我们假设复杂的 D-NUCA 是未来的发展方向，路划分也必须考虑到访问不同路的非均匀延迟，而大多数缓存划分论文并未考虑这一点（[57] 是唯一的例外）。

因此，本节的大部分工作适用于采用共享单片 LLC 的小规模系统，或者适用于 LLC 的每个分块缓存体由多个核共享的大规模系统。在这两种情况下，我们都预期有

少量核竞争可用的缓存空间。

缓存替换策略中的策略 缓存容量通常是在缓存替换过程中进行调整的。传统上, 大多数缓存都使用 LRU 块替换方法。这需要为每个组维护一个最近使用栈（recency stack），以指示该组中每个块最近一次访问的顺序。替换过程涉及以下三个主要策略:

1. 牺牲块选择：传统上，选择该组最近使用栈中的 LRU 块。
2. 块插入：当插入一个新块时，传统上将其作为最近使用栈中的 MRU 块插入。
3. 块提升：当一个块被访问时，将其移动到最近使用栈中的 MRU 位置。

大多数缓存划分论文都试图通过改变上述三个策略中的一个或多个来偏离基线 LRU 策略。

优化策略 同样值得注意的是，缓存划分可以有多个合理的性能目标。Hsu 等人 [58] 表明，最优划分会因所优化的指标不同而显著变化。他们确定了三种主要的性能目标类型：（i）共产主义式（Communist），确保每个应用程序达到相似的性能水平或性能下降程度；（ii）功利主义式（Utilitarian），试图通过将资源分配给能够最好地利用额外缓存空间的线程来最大化整体吞吐量；（iii）资本主义式（Capitalist），指对资源的无管制竞争，类似于基线 LRU 策略。Hsu 等人表明，资本主义式策略可能导致在整体吞吐量和公平性方面高度次优的划分。他们还表明，另外两个性能目标之间总体上具有良好的相关性，即除少数例外情况外，优化公平性的策略也能提供足够的整体吞吐量（反之亦然）。这些例外情况使得在设计缓存划分策略时必须明确性能目标。3.1.2 节主要关注功利主义式策略，而 3.1.3 节关注共产主义式策略。

显然，如果应用程序的工作集能够轻松地放入缓存，那么巧妙的缓存管理就没什么用武之地。当总工作集超过可用缓存空间时，这些策略就显得很重要。在这种情况下，策略必须决定哪些块可以留在缓存中，哪些块被逐出。先前的工作一直假设缓存替换采用 LRU，这是一种资本主义式策略。LRU 下的行为受应用程序发出数据请求的速率支配；如果一个应用程序访问大量数据，它就会被分配大量缓存空间。但这种额外的缓存空间未必能帮助这个需求旺盛的应用程序降低缓存未命中率。相反，它可能会伤害其他共同调度的应用程序，这些应用程序可能被剥夺了它们享受高命中率所需的少量缓存空间。这就是近期主张从资本主义式转向功利主义式或共产主义式策略的主要洞见：空间分配应基于预期收益或影响，而不仅仅是基于活动水平。

动态组采样 本章介绍了几种创新的缓存管理策略。很少有单一策略能在所有基准程序上都成为明显的赢家。而且，这些策略通常只有在针对每个程序优化其参数时才能发挥最佳效果。许多论文中的一个共同特点是使用一种称为动态组采样或组决斗监视器（set dueling monitors，SDMs）的机制，来帮助每个程序识别最优策略和参数。这是 Qureshi 等人 [53, 59] 引入的技术。在一个包含多个组的大型缓存中，可以用少数几个组来监视给定策略或参数的行为。例如，如果缓存有 4096 个组，可以将 16 个组硬连

线为使用策略 A，将 16 个组硬连线为使用策略 B。缺失计数器可以跟踪这 16 个组中的缺失次数，并判断策略 A 是否优于策略 B。然后可以将更好的策略应用于缓存的其余 4064 个组。因此，用于实验的缓存空间不到 1%；即使这个“实验实验室”中的某些策略高度次优，这里产生的额外缺失也应该在噪声范围内。当然，主要好处是可以在运行时为每个程序选择最优策略。在处理多核工作负载时，可能选项的数量有时会急剧增长，这可能需要使用更多的组样本。

3.1.2　吞吐量

我们首先介绍目前被认为是基于吞吐量的缓存分区技术中最先进的几项工作（UCP、TADIP、PIPP、$AGGRESSOR_{pr} - VT$）。然后简要描述该领域早期提供初步见解的一些先前工作。UCP 通过根据每个路提供的边际效用为应用程序分配缓存路来执行显式分区。TADIP 和 PIPP 基于最近的工作，这些工作声称插入策略创新在构建最优缓存种群方面更有效。因此，它们通过为每个线程提供不同的插入选项来努力实现隐式缓存分区，这一概念将很快展开。虽然这些方案是显式或隐式形式的路径分区，但一些论文也研究了显式集合分区，这种方法在某些情况下可以通过页着色 [46] 在现代硬件中直接实现。

UCP，Qureshi 与 Patt，MICRO’06

默认情况下，竞争线程之间的缓存分区由 LRU 替换算法完成。Qureshi 和 Patt[60] 观察到 LRU 忽略了更多缓存空间可以提供给给定线程的边际效用。他们基于效用的缓存分区（UCP）机制估计每个路对每个线程的边际效用，并选择最小化缓存未命中率的分区。关键贡献是设计了一种低开销的监控机制（UMON），可以高效地计算必要的边际效用。

每个核心都有一个 UMON 单元，它存储 L2 缓存的标签，并像 L2 缓存仅被此核心访问一样管理这些标签。这些“影子”标签使用 LRU 替换策略进行管理。如果一个请求命中（例如）影子标签中最近使用的第 7 路，那么只有当缓存分区策略为此核心分配至少七路时，该请求才会命中。跟踪第 7 路边际效用的计数器递增。每个 UMON 单元的每一路都维护一个这样的计数器。每个纪元（例如 500 万周期），检查这些计数器，选择最小化整体缺失率的路分区，并重置计数器（或减半以保留一些历史）。为了让替换策略在一个集合内强制执行路分区，每个块必须有一个几位标签来指示其“所有者”核心。以强制执行每核心路配额的方式选择要驱逐的块。公平地说，UCP 主要改变了受害者选择策略：一旦选择了受害者核心，基线 LRU 受害者选择、插入和提升策略将在分配给该核心的路内应用。

注意，选定的路分区适用于整个缓存，并且不是为每个单独的集合做出路分区决策。影子标签开销可以通过采样来减少；例如，可以对每第 32 个集合进行监控。这将 UMON 单元的存储开销降低到仅几千字节。但每个核心需要一个 UMON 单元，因此开销随核心数量线性增长。通过维护部分标签可以进一步减少存储开销。

虽然对于两个竞争核心，路分区决策是简单的，但随着更多核心竞争，必须考虑的可能性数量呈指数增长。如前所述，也许这不是主要问题，因为单片缓存单元可能只由几个核心共享。Qureshi 和 Patt 还提出了快速决策的启发式方法 [60]。他们从标准贪心算法开始，并对其进行改进，使算法不仅检查下一路的边际效用，而且意识到（在算法中）即将出现的边际效用大跳跃。如果访问同时涉及共享和私有数据块，决策过程也会变得复杂，但这在 [60] 中没有考虑。

基于边际效用的缓存分区的一些初步见解由 Suh 等人 [61] 的工作提供（稍后讨论）。前一章描述的其他工作也为每个核心分配了路 [26，27]，并使用运行时指标来估计额外路的边际效用 [27]。最近几篇论文指出，如果优化函数专注于最大化核心 IPC 而不是最小化整体 MPKI，则可以对 UCP 进行小的改进 [62，63]。

TADIP（Jaleel 等, PACT 2008）。 缓存通常采用 LRU 替换策略，传入块占据访问最近性栈中的 MRU 位置。为减少混淆，我们将基线块插入策略称为 MRU 插入策略（MIP）。在 ISCA'07 的一篇论文中，Qureshi 等人 [53] 假设这种策略在许多工作负载（流式或工作集非常大的工作负载）中无法保留相关块。对于此类应用，将传入块标记为 LRU 而不是 MRU 更有效。这被称为 LRU 插入策略（LIP）。因此，插入策略被改变，但受害者选择和块提升策略与基线 LRU 机制相同。此策略的改进是双峰插入策略（BIP），它以小概率将块放置在 MRU 位置。对于给定应用，最佳插入策略（BIP 或 MIP）通过集合决斗监视器（SDM）选择，该监视器在几个集合上采用每种策略并确定性能最佳的一种（见第 3.1.1 节）。这被称为动态插入策略（DIP），并在第 3.2.1 节中更详细地讨论。图 3.1 以图形方式表示了许多最近的插入策略。

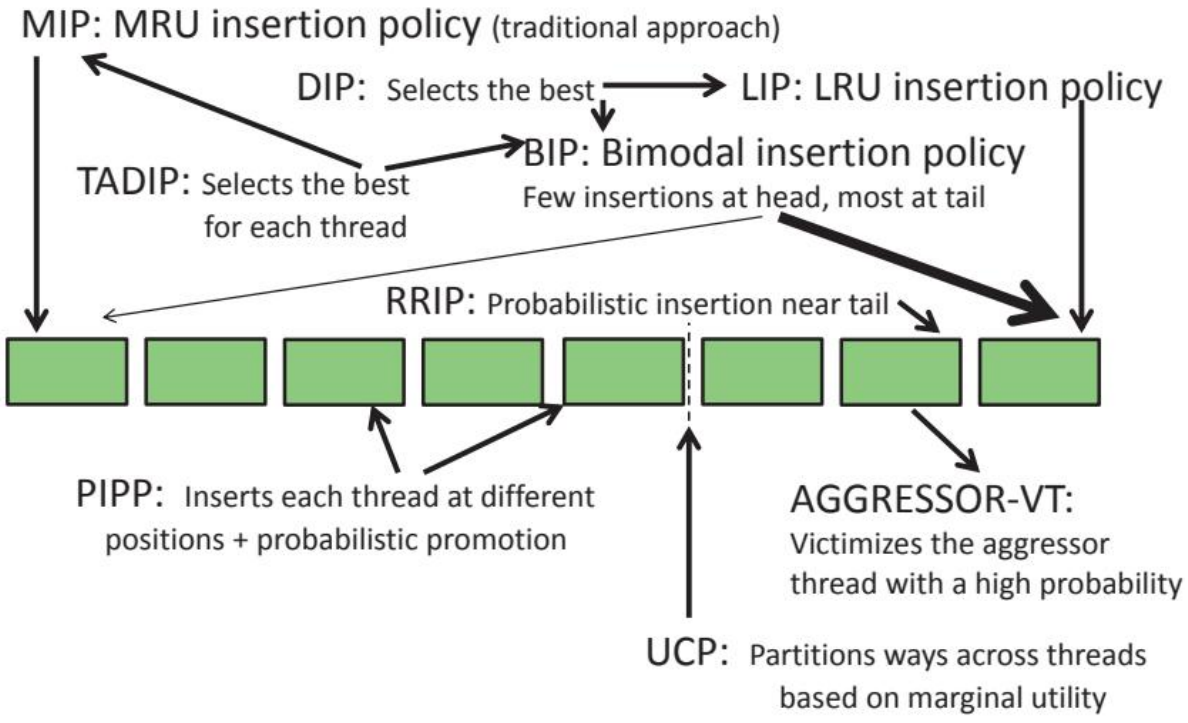


图 3.1 缓存块的插入、提升和牺牲块选择策略。图中箭头表示优先级的变化。

Jaleel 等人 [64] 首先表明在共享缓存中使用 DIP 是有效的。在这种情况下，共享缓存的所有应用程序要么使用 BIP,要么使用 MIP。然后作者提出了线程感知 DIP（TADIP），它为每个单独的线程选择 BIP 或 MIP。通过为每个线程选择适当的插入策略，每个缓存行的生命周期受到影响，从而实现缓存分区。边际效用的概念（如 UCP 论文中）

这一点在 TADIP 中有所体现，因为它决定了哪些线程无法从保留块中获益。这些线程被迫通过将其块插入 LRU 位置来占用更少的路。但 TADIP 比 UCP 更有效，因为即使工作集很大的应用也能在缓存中保留其工作集的一部分（局部性最高的部分），这得益于 BIP 偶尔以概率方式插入到 MRU 位置。

与 UCP 不同，TADIP 不需要影子标签来估计最优缓存分区。TADIP 要求我们为每个线程估计最优插入策略（BIP 或 MIP）。这同样通过 SDM 以两种不同策略完成。第一种策略称为 TADIP-Isolated，它试图在假设其他线程采用 MIP 的情况下确定每个线程的最优策略。一个 SDM 用于监视所有线程使用 MIP 的集合的行为。另外 N 个 SDM 用于每个 SDM 对一个线程使用 BIP 而对所有其他线程使用 MIP（N 是核心或线程数）。第二种策略 TADIP-Feedback 试图在了解其他线程所选策略的情况下确定每个线程的最优策略。总共使用 2N 个 SDM，其中每个线程使用两个 SDM。这两个 SDM 中一个对“所属”线程使用 BIP，另一个使用 MIP；对于所有其他线程，两个 SDM 都使用其他 SDM 所选的当前最优策略。这些策略的总额外存储开销相当小。但重要的是确保构成 SDM 的集合只占缓存中总集合的一小部分，即只能支持中等数量的核心。

PIPP，Xie 和 Loh，ISCA’09

TADIP 方法使用可变插入策略来实现缓存的伪分区；之所以说是伪分区，是因为没有严格指定或强制实施分区。在 ISCA’09 论文 [65] 中，Xie 和 Loh 引入了对插入策略的修改。此外，他们提出了一种替代的提升策略。这种组合被称为提升/插入伪分区（PIPP）。TADIP 影响插入策略；PIPP 影响插入和提升策略；两者都不影响牺牲块选择策略。注意，这些基线策略指的是块在最近使用栈中的位置。采用新策略后，栈不一定按访问最近性排序。因此，牺牲块选择过程驱逐的是传统上最近使用栈末尾的块；这个块不一定是 LRU。因此，将栈称为优先级栈比称为最近使用栈更合适。

PIPP 不是像 TADIP 那样将块插入 MRU（高优先级）或 LRU（低优先级）位置，而是允许插入到栈中的任意位置。如果某个应用被认为能从更高容量中获益并获得较大的缓存分区，那么它的所有块都会插入到栈中较高优先级的位置。这个位置的确定由操作系统完成，可能借助 UMON 电路的输入，该电路为每个核心估计边际效用曲线。

PIPP 不是在被访问时立即将块移到高优先级位置，而是允许块以指定概率向上移动一个位置。

很容易看出，可能存在偏向 TADIP 或 PIPP 的场景。如果一个块在短时间内被访问两次，TADIP 会将块提升到高优先级位置

并会不必要地延长块的缓存寿命。另一方面，如果一个块只被访问一次，TADIP 能更快地驱逐该块。为了更好地处理后一种情况，Xie 和 Loh 在检测到流式或抖动工作

负载时使用更激进的策略——这类工作负载的块被插入到低优先级位置，并且提升概率更低。通过使用概率性 BIP 策略，TADIP 可以保留抖动工作负载的一部分，而 PIPP 则将应用的每个块统一插入到所选位置。总体而言，Xie 和 Loh 的结果表明 PIPP 优于 TADIP 和 UCP。

除了 PIPP 策略外，Xie 和 Loh 还提出了一种新颖的监视电路，称为缓存内估计监视器（ICEmon）。与集合决斗类似，指定少数集合来监视每个核心的行为。对于这些集合，使用传统 LRU 来管理其分配核心的块。来自非分配核心的块只允许占据栈中的低优先级位置。计数器跟踪每个额外缓存路为分配核心提供的边际效用。一旦考虑低优先级路，这些估计就会有噪声，但这种噪声无关紧要。这种监视器消除了每个核心中采样影子标签的需要。

AGGRESSOR$_{pr}$-VT，Liu 和 Yeung，PACT'09

Liu 和 Yeung 最近的工作 [66] 保持插入和提升策略不变（与基线 LRU 相同），但与 UCP 类似，专注于牺牲块选择策略（TADIP 和 PIPP 未触及）。他们首先观察到，一个侵略性缓存抖动线程在 80% 的情况下被一个能预见未来以最小化缺失率的理想替换策略选为牺牲者。这引出了他们提出的替换策略：（i）如果侵略性线程恰好是集合中的 LRU 块，则它被牺牲；（ii）如果不是，则以 99% 的概率牺牲侵略性线程的 LRU 块。这强烈地使替换策略偏向于驱逐侵略性线程的块。他们还观察到，对于许多线程和工作负载，当侵略性线程使用 99% 或 50% 的牺牲概率时，整体吞吐量（以加权 IPC 衡量）达到最大。

因此，在运行时采用以下过程（称为 $AGGRESSOR_{pr} - VT$）。与 [67] 中的算法类似，每个 epoch 进行测量以检测阶段变化，阶段变化触发探索模式以确定每个线程要使用的策略。每个线程被依次考虑，并与其他线程一起各执行一个 epoch，分别作为非侵略者和侵略者，牺牲概率为 50% 和 99%。在考虑的许多组合中，采用吞吐量最高的组合，直到检测到下一个阶段变化。

有趣的是，对于他们的工作负载，作者表明相对于基准 LRU 策略几乎没有改进空间。即使采用最优替换策略，IPC 和缺失率相对于基准 LRU 的改进也仅约为 5-6%。在这个小空间内，作者表明 $AGGRESSOR_{pr} - VT$ 策略明显优于 UCP。虽然机制不同，但该策略力求实现

与 TADIP 和 PIPP 相同的行为——三者都试图通过将抖动工作负载插入低优先级位置或在其被选为牺牲者时给予优待，来限制它们占用的空间。

Sub 等人，《超级计算杂志》，2004 年

UCP 基本框架的一部分，包括边际效用方法和缓存分区机制，最初由 Suh 等人提出 [61]。UCP 的主要贡献是利用采样的影子标签对每个核心的边际效用进行低开销估计。Suh 等人使用每路计数器甚至每组集合的计数器来估计边际效用。还维护 LRU 信息以跟踪集合间访问的新近度。通过组合这些计数器，可以以比整个路更细的粒度估计额外缓存空间的边际效用。操作系统考虑这一点并为每个进程指定分区大小；这在

做出替换决策时被考虑。现在每个缓存块标签必须存储其所属线程的 id，并且有计数器来跟踪每个线程在缓存中的块数。虽然这里不需要 UCP 的影子标签，但边际效用估计要么需要应用程序在隔离环境中执行，要么可能因其他应用程序的干扰而产生虚假估计。Dropsho 等人 [68] 也在其 Accounting Cache 中采用每路计数器来估计缺失率曲线。这样做是为了通过禁用路来降低能耗，而不是为了分区缓存。

Yeh 和 Reinman，CASES 2005

Yeh 和 Reinman 提出了一种自适应共享缓存分区方案 [57]，该方案具有一些后来也被纳入其他分区方案的特征。例如，他们提出使用影子标签在运行时快速估计应用程序的缺失率曲线，这一特性也出现在后续工作 [60] 中。采用基于 epoch 的方案，每个 epoch 使用贪心算法为应用程序分配路，以最大化吞吐量，同时确保没有应用程序的性能显著差于其拥有私有缓存库的情况。与该领域大多数工作不同，Yeh 和 Reinman 考虑的是 D-NUCA 缓存，并在为核心分配路时考虑邻近性。与所有 D-NUCA 方案一样，该设计受到搜索和迁移复杂性的困扰。

Rafique 等人，PACT’06

Rafique 等人 [69] 提出了硬件机制和操作系统策略，以在竞争线程之间分配共享缓存容量（共享缓存具有统一的访问时间）。与 Suh 等人 [61] 的工作类似，存在以路或整体缓存空间来实施每进程配额的机制。后一种配额在容量分配上更为公平，但在处理某些热集合时可能表现不佳。操作系统采用各种策略来确定配额，并通过写入特殊硬件寄存器来设置这些配额。操作系统策略可以采用静态分配，也可以监控性能计数器并尝试均衡每线程的缺失率或将每线程的 IPC 与每线程优先级匹配。

MTP，Chang 和 Sohi，ICS’07

Chang 和 Sohi [70] 观察到，对于许多工作负载，分配容量的小幅增加远比分配容量减少导致的性能下降更有益。因此，对于这些工作负载，在任何时候实施不公平分区都更好，即一个应用程序获得大部分缓存资源，而其他应用程序获得一小部分资源。其他应用程序在后续 epoch 中尝试弥补部分性能损失，届时每个应用程序获得大部分缓存空间。Chang 和 Sohi 指出，循环使用这些有偏分区比在整个执行过程中使用相同的分区更有利于吞吐量和 QoS。该机制被称为多时间共享分区（MTP）。作者还将 MTP 与其协作缓存方案 [48] 结合，以便每个 epoch 根据工作负载使用两种策略中更好的那一种。

自适应 SET PINNING，Srikantaiah 等人，ASPLOS’08

Srikantaiah 等人 [71] 的工作并不完全是作为缓存分区机制，但它确实在竞争核心之间分配 LLC 空间。它主要设计用于通过减少缓存干扰来提高整体缓存命中率。它通过在核心之间分配集合来实现这一点（不同于大多数其他论文直接或间接地在核心之间分配路）。

该论文首先引入了处理器内和处理器间缺失的概念。这指定了一个块是被下一个访问它的同一处理器（处理器内）驱逐，还是被不同的处理器（处理器间）驱逐。这

显然是一种近似分类。它是对谁恰好将块推出 LRU 栈边缘的采样。但它并未编码将块推向驱逐边缘的事件。

该论文认为，少数热缓存块是大多数处理器间缺失的原因，最好将这些块隔离在访问核心私有的缓存区域中。因此，每个核心维护一个处理器拥有的私有（POP）缓存，该缓存被视为层次结构中 L2 级别的一部分，但大小与典型的 L1 缓存相似。此外，L2 中的每个集合都分配给第一个访问它的核心。这基于对集合的首次访问实现了一定程度的缓存分区。如果一个核心引入的块索引到属于另一个核心的集合，它被迫将该块放入自己的 POP 缓存中。类似地，在查找块时，我们不仅必须查找该块索引到的集合，还必须查找所有 POP 缓存。因此，整个 L2 缓存没有集合内干扰。

该机制被称为集合钉扎（Set Pinning）。为了防止因首次访问集合时可能发生的情况而导致次优的缓存分区，处理器在运行时被迫放弃对集合的所有权。每个集合维护一个计数器，用于跟踪该集合是否对非所有者核心产生比命中更多的缺失。如果是这种情况，所有者核心放弃对该集合的所有权。这被称为自适应集合钉扎（Adaptive Set Pinning）。

集合分区也可以通过页着色等替代方案实现，并且已在 NUCA 缓存 [41，42] 和 UCA 缓存 [72] 中采用。本节末尾将对此进行讨论。

ACCESS，Jiang 等人，HPCA 2011

在最近的一篇 HPCA'11 论文中，Jiang 等人 [73] 认为未来的多核处理器可能会采用一组异构的私有 LLC，其中每个 LLC 本身可能被多个核心共享。鉴于未来多核上工作负载的多样性，使用不同大小的 LLC 可以防止过度配置并提高每瓦性能。Jiang 等人描述了在此类架构上计算线程到资源映射以优化吞吐量所需的硬件和操作系统支持。

每个私有 LLC 为共享该 LLC 的每个核心以及系统中发现的每种 LLC 相联度维护影子标签。集合采样被用来大幅减少这些影子标签的大小。这种结构允许操作系统估计每个线程在每个 LLC 上单独执行时的缺失率。此信息在每个程序阶段开始时收集。为了计算给定调度的缺失率，操作系统执行各种计算。它首先根据从影子标签收集的少量数据点估计缺失率曲线的方程。然后，它根据所考虑调度中共享该私有 LLC 的线程的相对缺失率来估计每个核心的缓存分配。上述两个估计用于计算该调度的总体缺失率。由于可能的调度数量可能非常高，仅在任务进入/退出或线程进入新的程序阶段时才进行性能估计。当这种情况发生时，操作系统考虑几个增量调度，这些调度根据新线程的容纳方式而变化。最终，该架构展示了如何以接近最优的方式在多个核心之间划分整体 LLC 缓存空间。

改善缓存共享的软件技术

大多数缓存分区工作都假设存在专用硬件来估计最优分区。例如，Qureshi 和 Patt 的 UCP 工作 [60] 假设使用影子标签来估计程序的缺失率曲线（MRC）作为分配的缓存大小（相联度）的函数。更近期的工作，如 TADIP[64]，不需要估计 MRC，但它假设有额外的硬件用于集合对决监视器（SDM）。因此，虽然这些想法是未来处理器的绝

佳候选，但大部分工作无法在现有处理器中支持。

虽然基于硬件的技术假设了共享缓存的预定义工作负载，但操作系统也应该能够构建运行时工作负载或程序调度，以最小化（但可能无法消除）线程间干扰。例如，Chandra 等人 [74] 描述了两种启发式模型和一种分析模型，如果提供每个线程的栈距离剖面作为输入，这些模型可以预测干扰程度。假设此类信息可用，操作系统可以首先确保共同调度的线程尽可能少地相互干扰；然后可以运用前面描述的基于硬件的策略来进一步减少干扰。接下来我们描述两种最近的基于操作系统的技术，这些技术可以在现代处理器中实现，以改善缓存中的共享行为，并可能补充其他基于硬件的技术。

Tam 等人 [75] 展示了如何在现代处理器中以可接受的成本估计 MRC，从而使操作系统能够实现类似 UCP 的方案。在 Power5 处理器上，Tam 等人将每个 L1 数据缓存缺失的地址记录在寄存器中并引发异常。调用的系统调用随后将寄存器值记录到内存中的跟踪日志中。关键在于利用了硬件性能监控单元，避免了昂贵的二进制插桩。但由于频繁的异常，此过程也会产生不小的开销，导致应用程序速度减慢 4 倍。然而，该过程只需在程序开始时或每个程序阶段开始时调用。可以在几百毫秒内记录足够的跟踪。然后可以使用标准的栈距离算法来分析跟踪并估计 MRC（同样消耗几百毫秒）。由于跟踪包含 L1 数据缓存缺失，它独立于可能调度在其他核心上的其他线程。Tam 等人使用运行时生成的 MRC 来实现页着色策略 [41]，该策略在核心之间划分集合以最小化总体缺失率。一些相同作者还继续使用基于页着色的技术将有问题的页面隔离到 LLC 的一小部分区域。这项工作在第 3.2.1 节中更详细地描述。

Zhuravlev 等人 [76] 关注实用的操作系统调度机制以改善缓存共享。他们关注具有多个共享缓存的处理器模型，并尝试将应用程序分配给核心，以最小化总体缓存干扰。他们提出的方法可能也适用于具有单个共享缓存的其他处理器模型。他们观察到，简单检查应用程序的缺失率就足以实现良好的应用程序分类。他们论证了不需要使用栈距离剖面的更复杂方法。每个应用程序的缺失率可以在运行时采样，甚至采样时其他共同调度的应用程序引入的噪声也不会导致糟糕的调度决策。一旦收集了这些近似缺失率，应用程序根据缺失率排序，并将应用程序分配给核心，使得每个共享缓存的总体缺失率大致均衡。这被称为分布式强度在线（DIO）机制。

集合划分方案

上一章着重于弥合共享缓存与私有缓存之间的差距，并降低 NUCA 缓存的延迟，而其中许多论文也在多个核心之间对 LLC 进行划分，无论是隐式还是显式。例如，Qureshi 的动态溢出-接收设计 [52] 将私有 LLC 指定为溢出者和接收者，从而隐式地将 LLC 划分为不断变化的不等配额。Beckmann 等人 [20] 通过引入硬件成本/收益度量并改变副本创建的概率，使这种划分稍微更加明确。

在上一章描述的工作中，最能体现显式缓存划分形式的是 Cho 和 Jin [41]、Lin 等人 [46, 47] 以及 Awasthi 等人 [42] 的研究。所有这些研究，甚至包括 Tam 等人 [75] 的研究，都假设采用共享 LLC 并在核心之间划分集合。这种划分通过页着色实现：强制每个核心将数据分配到映射到分配给该核心的集合的页面中。这给软件带来了更大的负

担；Lin 等人 [46] 和

Tam 等人 [75] 甚至表明，这种基于集合的缓存划分在现代硬件中是可以实现的。相比之下，本节讨论的显式和隐式路划分机制需要额外的硬件支持。当考虑 NUCA 缓存时，路划分与 D-NUCA 设计兼容，而集合划分与更简单的 S-NUCA 设计兼容。如上一章所述，通过引入成本函数来确定每个页面的首选颜色 [42]，可以使集合划分更加明确，并且可以通过以显著的硬件成本迁移页面来动态改变划分 [42，45]。

3.1.3 QoS 策略

上一小节关注的是最大化整体吞吐量的功利主义策略。正如 Hsu 等人 [58] 所描述的，还存在其他合理的性能目标，例如试图均衡所有共同调度线程的性能或性能下降的共产主义式策略。类似地，策略可以结合吞吐量和公平性，例如，Yeh 和 Reinman 尝试在最大化吞吐量的同时限制每个线程的下降 [57]。共产主义式策略的一种变体是精英式 [77] 策略，它对每个应用程序施加不同的优先级或不同的性能下降约束。如果系统期望为每个运行中的应用程序提供某些保证，例如在数据中心或其他向客户“出租”计算能力的平台中，则需要上述策略。本小节描述了有助于提供此类服务质量（QoS）保证的机制和最新创新。

Iyer 等人，SIGMETRICS 2007，ICS 2004

大多数关于 QoS 的先前工作都试图为应用程序调度 CPU 时间以满足性能约束。在多核 CPU 中，许多应用程序同时执行并共享 LLC 和内存带宽等资源。Iyer 等人 [77] 表明，分配缓存空间和内存带宽的策略对于调整单个应用程序的性能并允许每个应用程序满足其性能约束非常有帮助。该论文为一个尽力而为的 QoS 感知平台奠定了基本框架。如果必须提供比尽力而为更强的保证，那么系统必须能够在当前活动服务器无法满足所有性能保证时激活更多服务器。

首先，必须为每个应用程序指定最低性能约束（高优先级应用程序的性能约束也常被称为目标）。可以使用多种指标来衡量 QoS 策略的有效性，但应用程序执行时间或吞吐量是通常采用的指标。QoS 策略可以是静态的，即预先指定每个应用程序的资源配额；也可以是动态的，即预先指定性能约束，由硬件/操作系统负责在运行时监控行为并调整资源配额。资源配额影响分配给每个应用程序的共享 LLC 空间以及每个应用程序可用的内存带宽。内存带宽分配是 QoS 系统的重要组成部分，以防止优先级反转，即低优先级的应

用程序如果缓存配额很小，可能会经历多次缓存未命中，并对高优先级应用程序的内存总线可用性产生负面影响。

Iyer 等人的 QoS 感知内存架构分为三层。第一层是优先级分类，要求管理员指定优先级和约束。操作系统负责在每次应用程序上下文切换时将这一信息传播到硬件 QoS 寄存器。第二层是优先级分配，需要根据性能监控将操作系统指定的约束转换为实际的资源配额。配额保存在硬件 QoS 资源表中。CPU 发出的每次缓存访问在发送到 LLC

之前都会相应地标记为高优先级或低优先级。第三层是优先级执行，需要计数器来跟踪内存系统和缓存空间的使用情况。缓存配额执行需要一种了解每个应用程序使用情况的替换策略。内存带宽分配的执行要求允许高优先级应用程序在让出内存总线给其他较低优先级请求之前发出 N 个内存请求（如果存在于内存控制器队列中）。

在先前的工作中，Iyer [78] 关注优先级执行问题，并为 LLC 提出了不同的机制。第一种机制涉及路划分，要么为应用程序保留特定的路（称为静态），要么对应用程序可以占用的路数施加限制（称为动态）。第二种机制试图通过概率性地选择是否缓存获取的行来控制每个应用程序的缓存空间。第三种机制实现不同的缓存结构（全相联或直接映射缓存、流缓冲区、牺牲缓存等），并根据需求和优先级将它们映射到应用程序。单个缓存行也允许自我标记，因此根据优先级，它们总是被缓存或立即自我失效。在后续工作中，Iyer 等人专注于基于跟踪每个应用程序整体缓存空间使用情况的优先级执行方案。

在进一步的后续工作中，Varadarajan 等人 [79] 主张将缓存组织成许多小的（8-32 KB）直接映射存储体，称为分子。这使得能够为 QoS 进行细粒度的缓存资源分配。缓存以分子为粒度跨多个核心进行分区。这允许高灵活性和低功耗；尽管该论文忽略了访问许多小存储体的互连开销（Muralimanohar 等人 [7] 后来详细考虑了这方面）。分子缓存方法允许在组织分子以形成核心缓存时具有完全的灵活性，即某些组可能比其他组拥有更多的分子（因此具有更高的相联度）。对于许多应用来说，这种高灵活性可能不值得实现成本。

Guo 等人，MICRO 2007

Iyer 等人在 SIGMETRICS’07 上的论文之后，MICRO’07 上的一项研究 [80] 为 QoS 管理器提供了更大的灵活性。作者首先提出了以下指导原则。QoS 需求最好以易于量化的资源来指定，这样应该容易验证所请求资源的可用性。换句话说，以 IPC 或缺失率来指定 QoS 目标会使 QoS 管理器的工作更加困难，但以缓存容量来指定 QoS 目标则易于处理。如果服务器没有任务所请求的资源，

任务会被暂停（或者必须被分配给空闲服务器）。Guo 等人还认为，任务应该以不同的方式分类：用户应该能够指定他们的任务可以承受一定的性能下降（称为弹性执行模式），也许因此支付较低的服务费。

具备上述灵活性使得更容易最大化给定服务器上的整体吞吐量。如果每个应用都提出严格的资源请求，可能会发生资源碎片化。例如，服务器可能有一些备用资源永远得不到使用，因为每个新任务要求的资源都超过服务器上可用的资源。每个任务也可能使用的资源少于严格请求中指定的资源。使用弹性执行模式的任务可能能够放弃一些资源，以帮助在服务器上容纳更多任务并提高吞吐量。这可以通过两种方式实现。一个新任务（否则不可接受）可以降级到弹性执行模式，并尝试利用服务器上恰好可用的任何资源。或者，可以在不违反指定保证的情况下从服务器上的其他任务窃取资源。资源窃取需要诸如采样影子标签 [60] 之类的机制来检测任何违规。

在另一篇后续论文中，Zhao 等人 [81] 阐述了未来处理器中所需的缓存簿记结构

（CacheScouts）。本质上，重要的是要了解一个进程占用了多少缓存空间，以及进程如何在缓存中相互干扰（无论是建设性的还是破坏性的）。这有助于 QoS 管理器，帮助操作系统做出智能调度决策，帮助程序员调整应用程序，并帮助服务提供商向客户收取资源使用费用。为了计算上述指标，必须将进程 ID 标签与每个缓存行关联。开销主要可以通过仅采样少数组来减少。需要计数器数组来通过观察传入和逐出块的进程 ID 来估计干扰。

SHARP，Srikantaiah 等人，MICRO 2009

Srikantaiah 等人 [82] 为 Guo 等人 [80] 的基本框架带来了更正式的控制理论方法。使用数学公式将 IPC 表示为缓存未命中率的函数，从而允许将 IPC 指定为性能目标（不同于 Guo 等人的建议）。该框架还最小化了向应用分配缓存路数时的振荡。如果所需的路数多于或少于可用的路数，则重新分配路数以最大化吞吐量或保持分配的优先级。虽然当所需路数少于可用路数时，控制理论框架可以保证最大吞吐量，但如果所需路数超过可用路数，则只能提供尽力而为的 QoS。

Srikantaiah 等人，Supercomputing 2009

虽然多核 QoS 论文同时关注缓存和内存带宽分区（例如 [77]），但 Srikantaiah 等人 [83] 的工作首次考虑了缓存和处理器分区。如果多核工作负载由多个多线程应用组成，吞吐量是分配给每个应用的核心数以及

分配给每个应用的共享缓存资源的强函数。必须考虑大量的处理器和缓存分区搜索空间。Srikantaiah 等人通过动态构建基于回归的模型来修剪搜索空间，这些模型将 IPC 表示为处理器数量和缓存资源的函数。这用于迭代预测最优分区，并且随着更多配置被采样和回归模型被细化，分区会被调整。可以选择分区以优化吞吐量或提供 QoS 保证。

虚拟私有缓存，Nesbit 等人，ISCA 2007

Nesbit 等人 [84] 关注将带宽 QoS 感知地分区到共享缓存所需的机制。这些机制类似于网络中使用的机制。共享缓存为每个线程维护一个待处理请求的缓冲区。如果一个线程被分配了一半的带宽份额，并且一次共享缓存访问需要 20 个周期，则预期该请求将需要 40 个周期。设置一个每线程寄存器以指示该线程在 40 个周期过去之前不能发出其下一个请求。当该线程的下一个请求出现时（在第 40 个周期之前），其预期完成时间设置为 80 个周期。如果下一个请求在第 40 个周期之后出现，则预期完成时间设置为请求到达时间之后的 40 个周期。因此，计算每个线程下一个请求的预期完成时间，并且调度顺序基于最早估计完成时间。上述策略将任何多余的备用带宽优雅地分配给到那时为止最少利用缓存带宽的线程。它还当然确保一个线程获得大约其承诺的缓存带宽份额。在线程的缓冲区内，允许重新排序，例如，读操作可以优先于写操作。需要更高级的操作系统/反馈机制来估计适合每个线程的缓存带宽份额。如果没有这种 QoS 感知的调度策略，缓存密集型应用很容易在基线 FIFO 调度策略下使其他线程饥饿。

Kim 等，PACT 2004

在早期工作中，Kim 等人 [85] 研究了公平缓存划分问题，这是 QoS 问题的一个近亲。在公平缓存划分中，试图统一降低所有共同调度的应用程序的性能，其中性能降低是相对于每个应用程序在孤立执行时的性能来衡量的。Kim 等人基于缓存缺失指定了一些合理的目标指标来估计“性能”。他们引入了静态路划分策略，这些策略需要预先知道每个应用程序的栈距离分布。他们还引入了一种基于动态 epoch 的方案，该方案迭代地识别受影响最大和最小的应用程序，并将一个路从受影响最小的应用程序分配给受影响最大的应用程序（以努力均衡对所有共同调度应用程序的负面影响）。如果这种重新分配没有显著改变性能，则在下一个 epoch 后取消。Kim 等人表明，针对公平性进行优化通常会带来整体吞吐量的提升，而反之则不然，即专注于吞吐量优化的策略通常会导致不公平。

3.2　为大型共享缓存选择高有用性的群体

本节关注决定将什么带入缓存、保留什么以及驱逐什么的策略。这些尝试通过用更少的资源做更多的事情来最大化缓存效率。我们首先讨论替换/插入策略，这些策略决定当应用程序遭受许多容量缺失时保留哪些数据。然后我们讨论关联度的优化，主要针对冲突缺失。最后，我们研究块级优化，包括压缩、预取以及消除死块/无用块。

3.2.1　替换/插入策略

虽然替换策略在过去已被充分研究 [86, 87, 88, 89]，但大型共享 LLC 的出现导致了该领域近期大量的观察和创新。人们普遍认为，某种 LRU 策略的变体最适合在缓存中保留相关块。然而，来自多个应用程序的多个数据结构的访问，每个都经过 L1 缓存的过滤，被多路复用到 LLC 中。根据这些访问的性质，使用 LRU 可能非常次优，这是本小节中涵盖的许多近期工作的前提。

LRU 假设最近对块的访问是其在不久的将来被使用的最佳指标。然而，有几个块不符合这个描述。一些访问模式最好描述为扫描 [90]，其中大量块被访问一次，并在遥远的未来再次被引用。例如，当顺序读取大型数组的内容时，就会发生这种情况。抖动访问模式 [90] 涉及一组确实表现出重用的块，但重用距离超过了缓存的大小。两种访问模式及其混合在 LRU 下都会表现很差。此外，当使用非包含层次结构时，具有高时间局部性的块可能保留在 L1 或 L2 中，而块的 LLC 版本通常只被访问一次。对于所有这些访问模式，重要的是识别具有非常长重用距离的块，并且不将它们保留在 LLC 中。在即将描述的一些工作中，一个常见的主题是块在保留之前必须“证明其价值”的概念。这通常通过修改插入策略来完成。另一个常见的主题是基于使用频率对块进行优先级排序。还有另一类工作基于这样的观察：任何时候缓存中有一半的块已经完成了它们的最后一次使用，需要以某种方式被识别为死块。一些死块预测工作将在第 3.2.3.2 节中介绍。

LIN，Qureshi 等，ISCA 2006

在 ISCA'06 的一篇论文中，Qureshi 等人 [59] 认为替换策略不应只考虑最近性，还应考虑从内存获取块的“成本”。这个成本由检查缺失时的内存级并行性（MLP）来确定。本质上，孤立发生的缓存缺失成本很高，而与其他缓存缺失同时发生的缓存缺失成本较低。

3.2 为大型共享缓存选择高有用性的群体

缺失的成本很低。这个概念类似于之前探索过的缓存块关键性的概念 [91, 92]，但 Qureshi 等人使用了明确关注 MLP 的新指标。还有先前的工作将成本和最近性整合到替换策略中 [88]，尽管那项工作关注的是由于 NUMA 导致的成本变化。

Qureshi 等人首先在 MSHR 中为每个条目引入计数器，这些计数器根据 MSHR 中活动条目的数量每个周期递增。一旦缺失被服务，MSHR 条目计数器提供对该缺失服务所产生成本的估计，称为 mlp-cost。假设该块的下一次缺失将产生类似的 mlp-cost，因此，该块被认为或多或少珍贵。这个估计存储在该块的标签条目中。因此，该实现具有一个很好的特性：块的 mlp-cost 在块被获取时估计，不需要单独的预测器。替换策略使用块的最近性和其 mlp-cost 的线性组合，并驱逐这种组合最低的块。因此，该策略被称为线性（LIN）。然而，该策略有时表现不如 LRU，特别是如果块的 mlp-cost 在连续缺失之间不一致时。因此，Qureshi 等人采用了一种锦标赛风格的预测器，动态决定使用 LRU 还是 LIN。一组样本集确定使用 LRU 时的缺失成本，另一组样本集确定使用 LIN 时的缺失成本。更好的策略用于缓存的其余部分。这种集合决斗的初始形式被称为基于采样的自适应替换（SBAR）。

尽管 mlp-cost 方法有其优点且实现成本相对较低，但近年来受到的关注相对较少。我们认为，考察 mlp-cost 指标与其他较新替换策略的相互作用可能是值得的。

自适应缓存，Subramanian 等人，MICRO 2006

Subramanian 等人 [93] 推广了 Qureshi 等人自适应选择两种替换策略中更优者的思想。在他们的工作中，他们专注于在每个组的基础上选择 LRU 和 LFU 中更优者。这是通过使用单独的辅助标签结构来实现的，这些结构跟踪使用任一替换策略时的假设缓存内容。每个组还有一个位向量，用于跟踪最近的历史，即哪种策略可能命中而另一种可能未命中。使用这种滑动位向量为性能提供了一些理论界限，而基于饱和计数器的实现可能无法做到这一点。在每次驱逐时，该组更优的替换策略被用来确定必须驱逐的块。由于我们可能会频繁地在组的策略之间切换，缓存的主标签存储可能既不类似于任一辅助标签存储。辅助标签的开销可以通过使用部分标签或进行组采样来减少，而不会对性能产生显著影响。使用组采样将要求对缓存中的所有组应用统一的策略；尽管这似乎对研究的工作负载的性能也几乎没有影响。

DIP，Qureshi 等人，ISCA 2007

我们在第 3.1.2 节中描述了 Qureshi 等人 [53] 的 DIP 技术，并在此重新讨论。由于任何大型共享缓存中的基线块插入策略都将传入块放置在访问新近度栈的 MRU 位置，

我们将其称为 MRU 插入策略（MIP）。在许多工作负载中，将块插入 LRU 位置（LRU 插入策略，LIP）更好。这使块在缓存中短暂停留，是处理流式块时的适当方法。如果块在被驱逐之前再次被访问，它会被移动到 MRU 位置并在缓存中停留更长时间。在双峰插入策略（BIP）中，大多数块被放置在 LRU 位置，并以小概率将传入块放置在 MRU 位置。这能有效地将大型工作集中相对“热”的部分保留在缓存中。

最终，Qureshi 等人提倡一种动态插入策略（DIP），该策略对大部分缓存采用 BIP 或 MIP。这一决策是通过在少数组上使用 BIP 策略、在另一些组上使用 MIP 策略，并在所有其他组上使用更优策略来做出的。这种使用组样本在竞争策略中进行选择的方法被称为组决斗监视器（见第 3.1.1 节）。只需使用 32 或 64 个组进行这些采样，这些组中的未命中会使单个饱和计数器递增或递减。因此，除了用于自适应管理新近度栈并跟踪构成 SDM 的组的逻辑之外，几乎没有额外的存储开销。因此，Qureshi 等人的工作为一个经过充分研究的问题提供了新的见解，并提供了一种既不过于复杂又具有显著进步的方法。

重用距离预测，Keramidas 等人，ICCD 2007

如果替换策略驱逐未来最远才会被访问的块，它可以近似 Belady 的 OPT 替换算法 [86]。Keramidas 等人 [94] 尝试对块进行高置信度的重用距离预测。该替换策略结合了 LRU 和重用距离信息。在具有重用距离预测的块中，他们选择未来最远才会被访问的块。在没有重用距离预测的块中，他们选择过去最远被访问的块（LRU 块）。在下次预测访问或上次访问中距离最远的块被选为驱逐对象。Keramidas 等人使用基于指令 PC 的预测器来预测重用距离。当块被取入 L2 时，相应指令的 PC 被用于索引 CAM 和 RAM 结构，这些结构可能产生该块的高置信度重用距离预测。在更新和查找预测器时，必须导航多个预测结构。

牧羊犬缓存，Rajan 和 Govindarajan，MICRO 2007

在 2007 年的 MICRO 论文中，Rajan 和 Govindarajan [95] 提出了一个有趣的观察。他们表明，具有一半相联度但采用最优替换策略（Belady 的 OPT [86]）的缓存可以胜过基于 LRU 的基线缓存。基于这一观察，他们设计了一种

缓存组织，其中缓存被组织为两个逻辑分区（路分区），其中一个分区用于收集信息，使我们能够在另一个分区上模拟 OPT。

其中一个分区被称为牧羊犬缓存（SC），并使用 FIFO 替换策略。在大多数实验中，牧羊犬缓存有 4 路，主缓存（MC）有 12 路。在每次缓存未命中时，传入块 A 被放置在 SC 中。它在该组的接下来三次缓存未命中期间一直留在 SC 中；此时，块 A 位于 SC 的 FIFO 队列头部。在下一次缓存未命中时，我们必须确定是驱逐块 A，还是牺牲 MC 中的一个块来保留它。我们试图用类似 OPT 的替换策略来管理 MC。在某种意义上，当 A 被带入时，从 MC 中驱逐谁的决定本应在许多周期前做出，但被推迟到了此时。通过推迟决定，我们能够检查 A 的取入之后发生了什么；这种对“未来”的窥视使我们能够在 MC 内制定类似 OPT 的替换策略。此时，要么 A 被驱逐，要么 MC 中的一个块被驱逐。这个块是通过检查自 A 取入以来发生的访问来选择的；我们要么选择

自 A 取入以来最后被访问的块（完全匹配 OPT 对 MC 会做出的替换决定），要么在有多个候选者的情况下，回退到 LRU 策略。该组织需要一些复杂的结构来跟踪自 A 取入以来的访问以及这些访问如何影响替换候选者的选择。

对这一策略的另一种看法是：每个新进入的块都被迫至少再经历四次其他块的驱逐；到那时，我们根据最近的行为决定是保留该块还是放弃它。因此，Shepherd Cache 的组织方式可能有些类似于 PIPP 类策略 [65]（第 3.1.2 节），其中块被插入到距离优先级列表尾部四个位置的地方。

Extra-Lightweight Shepherd Cache，Zebchuk 等人，ICCD 2008

Zebchuk 等人 [96] 尝试了 Shepherd Cache 理念的实际实现。如同原始 Shepherd Cache 设计 [95] 中一样, 新进入的块被放置在几个指定为 Shepherd Cache（SC）的 FIFO 路中。Zebchuk 等人没有跟踪块下一次访问的顺序 [95]，而是简单地跟踪 SC 中的块是否被触碰过。当一个块 A 晋升到 SC FIFO 的头部时，如果它未被触碰过，则被驱逐。如果块 A 已被触碰过，则主缓存中的 LRU 块被驱逐，而 A 被插入到主缓存的 LRU 位置。这种实现复杂度低，使得设计看起来更像上一段提到的替代视角。Zebchuk 等人还通过考虑近似 LRU 的实际基线替换策略来降低复杂度。他们指出，伪 LRU [97] 的实际实现可能产生比真正 LRU 高 9% 的缺失率。

RRIP，Jaleel 等人，ISCA 2010 在 ISCA 2010 的一篇论文中，Jaleel 等人 [90] 提出了一种低复杂度的替换策略，它结合了大量先前工作的洞见。在深入探讨他们提出的设计之前，我们首先描述他们的观察以及一些先前的见解。

对于几乎任何替换策略，LLC 一个组中的块都被组织为一个优先级列表。假设高优先级的块比低优先级的块更可能被较早重新引用。因此，当我们将一个块放入优先级列表时，我们本质上是在做一个隐式的重引用间隔预测（RRIP）。在 LRU 中，我们假设新进入的块或最近访问的块具有近乎立即或非常短的重引用间隔。现代处理器中因其简单性而使用的另一种基线策略是未最近使用（NRU）。在该策略中，每个块关联一个位。每次触碰时，该位被设为 0 以指示最近使用。如果需要驱逐一个块，我们只需选择遇到的第一个位被设为 1 的块。如果所有块的位都被设为 0，则所有位都被设为 1。由于仅使用单个位来指示最近性，很难区分组中许多块的优先级。Jaleel 等人的一个主要贡献是使用每个块多个位来以更细的粒度跟踪优先级，以及在后续访问时初始化和更新这些优先级的机制。

Jaleel 等人 [90] 的 RRIP 策略与 NRU 最为相似，但以更细的粒度跟踪信息。每个块都有一个关联的 M 位计数器（设计默认为 $M = 1$ 时的 NRU）。计数器值是重引用间隔预测的隐式度量：低值意味着我们预期该块在不久的将来被重新引用，高值意味着我们预期该块在较远的将来被触碰。在静态 RRIP（SRRIP）策略中，新进入块的计数器被初始化为 $2^M - 2$，比计数器的最大值 $2^M - 1$ 小一。仅当块的计数器值为 $2^M - 1$ 时才选择驱逐该块。如果不存在这样的块，则该组中所有块的计数器递增，直到某个计数器饱和并且可以驱逐一个块。当块被触碰时，其计数器可以减一（这是 RRIP-频率优先策略，与 LFU [98] 非常相似）或直接设为零（RRIP-命中优先策略，更接近 LRU）。

作者表明后者表现更好，因为单次重引用表明该块不是扫描（无立即重用的流式访问）的一部分，应该被保留。除了刚才描述的静态 RRIP，作者还引入了双峰 RRIP，其中新进入块的计数器被初始化为 $2^M - 1$ 或（以低概率）$2^M - 2$。此外，SDM 可用于动态选择静态和双峰 RRIP 中更好的一个，从而产生动态 RRIP（DRRIP）策略。

RRIP 引入的主要复杂性是为缓存中的每个块维护 M 位计数器。需要相应的查找、比较、递增和递减操作。这种复杂性高于 NRU 所需的复杂性，略低于在 LRU 和 LFU 之间选择的动态策略所需的复杂性。SRRIP 策略比 LRU 需要更少的硬件，但性能优于 LRU。

本质上，RRIP 优于 DIP，因为新进入的块被给予更多时间来确定其重用潜力。DIP 将大多数新进入的块放在优先级列表的尾部，

而 RRIP 通过将块的计数器初始化为非最大值，将大多数新进入的块放在列表的中间位置。这与 Shepherd Cache [95] 的理念相似。RRIP 策略也可以看作 NRU 的推广，结合了中间插入策略。正如 Xie 和 Loh [65] 所指出的，块在优先级列表上的插入和提升方式有无限的可能性，并且在某种程度上，Jaleel 等人的工作试图捕捉许多成功的先前策略（DIP、LFU、NRU、Shepherd Cache）的最佳特性。造成问题的两种主要访问模式是扫描和抖动。扫描是对数据流的单次访问，在不久的将来没有重引用。抖动是对大部分数据的访问，其重用距离足够大，以至于块在重用之前就被驱逐。当这些访问模式与其他确实表现出时间局部性的访问模式交错时，尤其成问题。将块插入到靠近优先级列表尾部的替换策略家族（DIP、PIPP、RRIP 等）主要针对扫描和抖动的容忍度，在容忍程度上存在细微差异。

伪 LIFO，Chaudhuri，MICRO 2009

在 Chaudhuri 的 MICRO 2009 论文中 [99]，他引入了填充栈（fill stack）这一新颖概念。填充栈按照块到达的顺序跟踪一个集合中的所有块。一个给定的块最初位于栈顶，并逐渐下移，直到被逐出缓存（可能在其到达栈底之前）。鉴于块在填充栈中这种具有一定可预测性的移动，Chaudhuri 提出，块在填充栈中的位置可用于估计其存活性。Chaudhuri 表明，大多数缓存命中由位于填充栈顶部的块服务。换言之，即使位于填充栈顶部的块也可能已经发挥了其效用，应优先被牺牲。这激发了一类利用填充栈信息并信奉后进先出（LIFO）优先级的替换策略。即使大多数命中并不局限于栈顶，也很容易识别出“命中曲线的拐点”——即填充栈中超过该位置后，产生命中的块数量急剧下降的位置。

在使用填充栈信息之前，应用首先在每个阶段的开始使用传统 LRU 执行，并收集关于填充栈位置平均产生命中的信息。该信息在每个 epoch 收集，也用于检测阶段变化。在此命中曲线中识别出三个“拐点”——即填充栈中超过该位置后，产生命中的块数量下降的位置。考虑了三种策略，每个拐点对应一种。在每种策略中，如果块（i）在其当前填充栈位置尚未产生命中，且（ii）在高于拐点位置的同时最接近填充栈顶部，则该块被牺牲。使用集合对决机制从这三种策略和 LRU 中选择最佳策略。这种概率逃逸 LIFO（peLIFO）策略的主要开销在于，每个块现在必须跟踪其当前填充栈位置（每

次逐出时都必须更新），以及产生其上次命中的填充栈位置（以便估计命中曲线）。

Chaudhuri 还提出了两种替代策略。一种使用死块预测器，并在存在多个候选死块时使用填充栈来打破平局。第二种基于以下前提：如果（例如）75% 的命中产生于填充栈位置 X 之后，而 25% 的命中产生于

填充栈位置 Y 之后，那么 50% 的块应在位置 X 和 Y 之间被逐出。基于这些估计的概率，选择给定填充栈位置中的块进行牺牲。

与仅加速恰好被访问一次的块的逐出的 DIP 和 RRIP 不同，Chaudhuri 的 peLIFO 策略还可以加速被多次访问的块的逐出。在这个意义上，它们与先前关于死块预测的工作最为密切相关，但开销显著更低。死块预测的概念已被用于各种缓存替换策略 [100, 101，102]，并将在第 3.2.3.2 节中讨论。在 Chaudhuri 的设计中，填充栈的概念还被用于利用远距离时间局部性。额外的命中由那些下降到填充栈底部、保留在 LLC 中可视为“牺牲缓存”的区域、并能够在遥远的未来产生命中的块所导致。

Scavenger，Basu 等人，MICRO 2007

Chaudhuri 关于 peLIFO 的 MICRO’09 论文 [99] 指出了长期重用的存在。peLIFO 策略以概率方式保留一些可能暂时死亡的块，这些块后来产生了额外的命中。然而，peLIFO 并未明确尝试保留那些可能在遥远未来产生命中的块。Basu 等人 [103] 的一些先前工作正是试图做到这一点。

Basu 等人将他们的 LLC 划分为两个大区域——常规 L2 和牺牲文件（VF）。当块从 L2 逐出时，它不会像传统牺牲缓存 [104] 那样被不加区分地放入 VF。仅当块有可能在未来产生命中时，才将其放入 VF。使用计数布隆过滤器来估计该块的未命中次数，该指标用于确定未来的重用以及块在 VF 中保留的优先级。使用最小堆来高效跟踪 VF 中优先级最低的块；仅当另一个块的优先级高于此最低优先级块时，才将其保留在 VF 中。大型 VF 组织为直接映射哈希表，并使用双向链表维护映射到同一哈希的所有块。虽然这些创新有助于降低维护大型牺牲文件的复杂性，但商用处理器在实现基于指针的硬件数据结构方面几乎没有先例。在定义一种复杂度有效且能保留具有远距离重用块的技术方面，可能还有很大的改进空间。

NUcache，Manikantan 等人，HPCA 2011

Manikantan 等人 [105] 提出了一种新颖的方法来识别应保留在缓存中以在遥远未来产生命中的有用数据。首先，他们识别产生最多缓存未命中的 PC。这些 PC 被称为违规 PC（delinquent PC）。LLC 的路被划分为 MainWays 和 DeliWays。所有传入块被放置在 MainWays 中，由 LRU 替换策略管理。从 MainWays 逐出的块，如果是由一组选定的违规 PC 引入的，则保留在 DeliWays 中。DeliWays 遵循 FIFO 替换策略。“选定违规 PC”的选择最好通过一个例子来解释。如果一个违规 PC 是唯一被允许使用 4 个 DeliWays 的 PC，并占所有未命中的 50%，那么一个块将在每隔一次未命中时插入

到 DeliWays 中。当该集合经历 8 次未命中时，该块将被逐出 DeliWays。因此，块的下次使用距离可以告诉我们，将该块保留在 DeliWays 中是否有用。块的下次使用距

离定义为该集合在块被逐出与其下次使用之间经历的未命中次数。如果已知由违规 PC 引入的块的下次使用距离直方图，我们可以估计将违规 PC 添加到选定集合的收益。添加此违规 PC 的成本是它会将其他块更早地挤出 DeliWays。这些计算并非易事，选定集合通过增量选择收益最高的违规 PC 来贪婪地构建。

通过跟踪近期 PC 产生的缺失次数，可以识别出最严重的 PC。新进入的块被保存在另一张表中，当该块被逐出时，全局缺失计数被记录在表中。当该块再次被访问时，我们计算下次使用距离。这会更新对应问题 PC 的下次使用直方图。

污染缓冲区，Soares 等人，MICRO 2008

Soares 等人 [106] 提出了一种基于软件的页着色方案来提高命中率。他们认为，最好将高缺失率的页面隔离到 LLC 的一小块区域。此类页面被视为污染者，因为将它们保留在缓存中并不会为其缓存块带来许多命中，反而会逐出其他有用的块。因此，作者将 LLC 的 $1/16^{th}$ 指定为污染缓冲区，操作系统页着色引导污染者页面映射到污染缓冲区中的集合。这作为一种应用内缓存分区形式，以提高命中率。

Soares 等人使用硬件性能计数器和 Linux 内核模块，在运行时跟踪每个虚拟页的各种 L2 命中/缺失统计信息。在程序阶段开始时进行初始分析后，最不利于缓存的页面（即 L2 缺失率最高的页面）会逐渐经历页面迁移。探索阶段会复制不同数量的页面，直到确定页面到污染缓冲区的最佳分配。

管理包含性，Jaleel 等人，MICRO 2010

包含性与非包含性缓存之间的权衡是众所周知的。包含性缓存简化了缓存一致性，因为标签查找只需在较大的包含性缓存上执行。然而，包含性缓存浪费空间（尤其是当包含性缓存并不比层次结构的前几级大多少时），并导致“包含性受害者”。包含性受害者是具有高时间局部性且存在于 L1 缓存中的块；由于 L1 过滤掉了这些访问，这些块在 L2 中逐渐漂移到 LRU 位置，被逐出，并且为了保持包含性也从 L1 中移除。虽然这些问题及相应的解决方案已经为人所知数十年，但 Jaleel 等人最近的研究 [25] 再次将焦点放在现代多核背景下的这个问题上。这一点尤为重要，因为最近几乎所有关于 LLC 替换策略的论文都集中在非包含性层次结构上，并在很大程度上忽略了包含性层次结构中包含性受害者的作用。未来的论文还需要明确说明其模拟器中如何管理包含性；许多最近的论文甚至没有说明 L1-L2 层次结构是否具有包含性。

Jaleel 等人表明，虽然包含性与非包含性之间的差异对于单线程工作负载并不显著，但对于多核处理器和工作负载则很显著。他们还表明，包含性与非包含性之间的差异更多地归因于包含性受害者，而不是有效缓存层次结构容量的差异。如果 LLC 是层次结构前一级的八倍大，非包含性对性能的提升不大；当 LLC 仅大四倍或更少时，差异更为明显。这与最近英特尔（包含性）[107] 和 AMD 处理器（非包含性）[108] 的设计选择相符。当包含性确实遭受性能损失时，可以通过在 L2 逐出之前检查 L1 的替换策略来消除该损失（假设是 L1-L2 包含性层次结构）。在包含性层次结构中，当块从 L2 逐出时，L2 无论如何都会向 L1 发送消息以从 L1 中逐出该块。如果在 L1 中找到该块，L1 不会逐出该块，而是回复一个 NACK，强制 L2 寻找另一个受害者。这个过程

通常可以隐藏在从内存获取传入块的延迟之后。

3.2.2 关联性的新颖组织

本小节讨论的论文观察到，某些集合比其他集合承受更大压力，因此更容易发生冲突缺失。他们尝试通过基于硬件的方案来均衡各集合的负载。

在深入这些论文之前，我们描述几个值得考虑的其他替代方案。集合间负载不均衡的问题可以通过先前提出的硬件/软件技术来缓解。例如，页着色策略可以减少对集合访问的非均匀性，从而无需更复杂的硬件。Sherwood 等人 [72] 提出了这种方法以减少大型缓存中的缺失。当集合被确定为热集合时，最近访问的页面会从这些集合重新分配到不同的颜色。TLB 跟踪这种虚拟页的片上重命名。还提出了索引函数来平衡各集合的活动 [109, 110]。最终，减少冲突缺失的最简单方法是增加关联性。由于 LLC 通常采用串行标签和数据访问，更高的关联性只会影响标签访问的能耗。

V-Way 缓存，Qureshi 等人，ISCA 2005

在第 2.1 节中，我们讨论了 Chishti 等人 [24] 的 NuRAPID 组织。NuRAPID 设计试图通过允许缓存体中灵活的数据放置，在 NUCA 缓存中实现低访问延迟。为了实现这一点，数据存储和标签存储被解耦，即允许标签条目指向任何体中的任何数据块，而不是指向数据存储中的固定条目。管理这样的缓存需要在标签存储和数据存储中使用前向和后向指针来匹配块的标签和数据。Qureshi 等人 [28] 在 ISCA'05 论文中使用了类似的灵感来提高 LLC 的命中率。

在传统缓存中，块映射到唯一的集合，以及为该集合保留的固定数据存储条目集合。Qureshi 等人认为，LLC 数据放置的灵活性有利于

有两个原因：（i）它有助于处理缓存组之间的负载不均衡，（ii）它允许使用能够感知单个组之外数据使用情况的新颖替换策略。这种灵活性通过以下组织方式提供。

标签存储允许容纳比数据存储更多的条目。它在组织上类似于传统的标签存储，具有固定数量的组和路数；一个重要的区别是，每个标签存储条目维护一个前向指针，以指示对应于该标签的数据存储条目。如果标签存储的条目数是数据存储的两倍，那么一半的标签条目将是无效的。在某种意义上，标签是超额配置的，允许给定组容纳其公平份额两倍的条目。这有助于吸收每组负载的一些变化。数据存储只是缓存块的集合，与标签存储条目没有直接的固定对应关系。每个数据存储条目必须维护一个反向指针，指向其对应的标签存储条目。

当引入一个块时，如果该组没有空闲的标签条目，则使用 LRU 驱逐该组中的一个标签条目。传入的块被放置在与刚被驱逐的标签对应的数据存储条目中。当引入一个块时，如果该组存在空闲的标签条目（由于标签存储的超额配置，大多数情况下都是如此），我们必须接下来从数据存储中识别一个要替换的数据块。由于前向指针和反向指针的存在，传入的块可以替换数据存储中的任何块。替换策略不必只检查该组内的访问新近度，它可以驱逐缓存中任何可能已经完成其用途的块。这种全局替换策略的

实现如下。每个数据存储条目都有一个 2 位饱和计数器，每次访问时递增。替换引擎扫描这些计数器，寻找零计数器值；遇到的第一个计数器值为零的块被驱逐。在此扫描期间，每个检查的计数器值都会递减。当处理下一次替换时，扫描从上次离开的地方开始。

这种新的组织方式带来了更好的替换决策，并处理了组之间的负载不均衡。主要的代价是更大的标签存储、前向指针和反向指针的维护，以及替换引擎使用的逻辑和饱和计数器。

集合平衡缓存，Rolan 等人，MICRO 2009

Rolan 等人 [111] 提出，大型缓存中的组会经历非均匀的活动和压力。当这种情况发生时，经历高压力的组与经历低压力的组相关联。压力通过每个组关联的饱和计数器来测量，这些计数器在未命中/命中时递增/递减。关联可以是静态的（如果组仅在其索引的最高有效位上不同，则它们相关联）或动态的（高压力组与压力最小的组相关联）。当组关联时，从高压力组驱逐的块被放置为关联组中的 MRU 块。当采用这种关联时，块查找必须在两个关联组中顺序执行。这种集合平衡策略动态地为少数组提供更高的相联度。该领域的先前工作集中在直接映射基线缓存上，例如 Zhang[112] 的平衡缓存工作。最近，Khan 等人 [102]

通过引入死块预测的概念（在第 3.2.3.2 节中更详细地讨论）扩展了集合平衡缓存。

STEM，Zhan 等人，MICRO 2010

刚刚描述的集合平衡缓存在 Zhan 等人的另一篇近期论文中得到了扩展 [113]。首先，他们引入了一种更准确（但成本更高）的度量来确定组是否会从关联中受益。标签阵列中的每个组都用最近驱逐块的部 分（哈希）标签进行扩充。饱和计数器跟踪这些牺牲标签中的命中，并确定该组应该溢出还是接收块。每次未命中时也会检查饱和计数器，以确认接收组没有被压垮。其次，集合平衡思想与替换策略创新相结合。主标签和牺牲标签使用不同的替换策略（LRU 或 BIP[53]），并且为每个组使用更好的策略。

ZCache，Sanchez 和 Kozyrakis，MICRO 2010

Sanchez 和 Kozyrakis 在他们的 ZCache 设计 [29] 中解耦了“路数”和“相联度”的概念。路数表示在查找块时必须搜索的标签数量。他们将相联度称为可以为传入块腾出空间而驱逐的块数。ZCache 保持路数较小，但具有较大的相联度。然而，一个显著的缺点是，在大多数缓存未命中时，块必须被复制和重新排列。

ZCache 建立在 Seznec 的倾斜相联缓存设计 [109] 之上。在倾斜相联设计中，缓存的每一路使用不同的索引（哈希）函数。这导致比传统组相联设计更少的冲突。如果缓存有四路，块 A 可以驻留在四个固定位置；当块 A 被带入缓存时，它替换这四个位置之一中的块。ZCache 扩展了这种替换策略。这四个位置中的块可能驻留在其他不与块 A 冲突的位置。因此，例如，块 A 可以取代块 B 的位置，而 B 可以移动到其其他三个可能位置之一。类似地，这三个位置中的块也可以移动到其他地方。换句话说，如果

我们允许块重新定位到其替代位置之一，一个替换可能性的树就展开了。通过检查这棵树到一定深度，我们可以找到最合适的替换候选。Sanchez 和 Kozyrakis 使用块上的粗粒度时间戳在几个候选者中选择 LRU 块。传入的块随后在缓存内引起一系列复制，直到 LRU 块最终被驱逐。虽然替换过程不在任何延迟关键路径上，但就命中和未命中能量而言，它使 ZCache 比具有相同路数的基线更昂贵。总体而言，如果更好的替换决策能够显著减少片外访问次数，能量就可以降低。

3.2.3 块级优化

本节讨论试图优化被带入缓存的块的质量的论文。大致组织如下：（i）处理块预取的论文，（ii）处理死块预测的论文，以及（iii）识别无用字或执行压缩的论文。这些主题也与缓存替换策略的主题密切相关。

预取已经被积极研究了至少三十年，Chen 和 Baer [114]、Jouppi [104] 和 Smith [115] 做出了重要贡献。预取可以应用于大多数层次的缓存；当预取块到 LLC 时最为有效，因为片外延迟通常无法通过乱序执行来容忍。向多核的转变并没有成为预取领域的主要游戏规则改变者，尽管一些论文报告了多核中预取效果的下降 [116]，并且一些工作 [117] 考虑了多核上多个预取器的协调。预取论文继续与准确性、及时性、污染和内存访问争用的经典问题作斗争。预取是一个足够广阔的领域，我们无法在本书中公正地对待所有最近的工作——我们将很快讨论代表最新水平的样本。

预取概念也与死块预测的概念相关联。死块预测首先由 Lai 等人 [118] 引入（Wood 等人 [119] 也考虑过）。在那项工作中，他们表明可以通过构建访问该块的所有指令的跟踪来预测块效用的结束。一旦一个块被认为是死的，就作为驱逐该块并在其位置预取有用块的触发器。Hu 等人 [120] 也使用死块预测作为预取的触发器，但他们不是构建跟踪，而是使用一段不活动时间来判断块已死。这两项工作都集中在 L1 缓存上。在本小节中，我们描述了该领域的最新进展，这些进展集中在 L2 缓存的死块预测上。死块预测已被用于预取 [101, 118, 120, 121]、缓存绕过 [100, 101]、缓存替换策略 [100, 101, 102] 以及能量减少 [122, 123, 124]。

最后，本节还讨论了研究缓存块内字效用并尝试优化以减少能量或增加有效缓存容量的论文。通过压缩缓存块的正交优化也可以增加有效缓存容量。

3.2.3.1 预取 预取是一个积极研究的领域，并且通常与缓存层次结构本身正交。今天继续大量使用的基本预取设计包括 Jouppi 的流缓冲区 [104]、Hu 和 Martonosi 的标签相关预取器 [125] 以及 Nesbit 等人的全局历史缓冲区 [126]。本节仅涵盖该领域最近工作的代表性样本。鼓励感兴趣的读者查看该主题的其他近期论文 [127, 128, 129]。研究 Srinivasan 等人 [130] 引入的方法和分类法以分析每个预取操作的各种效果也是值得的。

不连续指令预取器，Spracklen 等人，HPCA 2005

大多数预取工作集中在数据块上。我们提出 Spracklen 等人 [131] 的工作作为指令

预取器的最新示例。该论文还提供了很好的

3. 影响缓存命中率的策略

指令预取现有技术的总结，组织为顺序（流式）、基于历史（预测器）和基于执行（类似 runahead）的预取。

Spracklen 等人表明，先前的指令预取方法对于商业工作负载（数据库、Web 服务器）不太有效，因为它们具有较大的指令工作集大小，具有许多直接和间接分支以及函数调用（占所有指令缓存未命中的 60%）。他们认为，如果只关注简单地识别指令取指中的不连续性，即对非连续缓存行的取指，就可以构建低复杂度的预测器。每次观察到这种情况时，不连续性和所取行的地址被记录在预测器中（如果有空间）。当取一个缓存行时，检查预测器，如果命中，则取预测的下一行。不连续性预测器与顺序下一 N 行预测器一起使用，因此顺序预取触发不连续性预测器的查找，并且当预取非连续行时，我们继续从新目标地址取顺序行。预测器是直接映射的；每个条目只跟踪上次历史。每个条目还有一个 2 位饱和计数器；计数器在有用预取时递增，如果另一个不连续性想要使用同一条目则递减。只有当饱和计数器为零时才能进行替换。

Spracklen 等人观察到，指令预取对 L2 不太有效，因为有用的数据块会被驱逐。因此，预取的指令块只放在 L1 中；只有当它们在从 L1 驱逐之前有用时，才会放在 L2 中。每个生成的预取还必须在将请求发送到下一级之前检查缓存标签；这会造成对缓存标签的争用和大的预取队列。因此，预取在放入队列之前被过滤。如果地址与最近的需求请求匹配，则预取被丢弃。这个和其他简单的优化允许 90% 的所有预取队列请求最终被发送到下一级。

时间内存流，Wenisch 等人，ISCA 2005

Wenisch 等人 [132] 设计了一种特别适合多处理器系统的预取机制。他们观察到共享数据的消费者通常访问与先前消费者相同的地址序列，称为时间地址相关性。这种潜在任意地址的序列称为流（不要与 Jouppi 的流缓冲区 [104] 混淆，后者指的是连续地址的序列）。由于这些对流访问频繁重复，该现象称为时间流局部性。因此，当多处理器系统中的节点 i 发生未命中时，它检查另一个节点 j 最近是否发生了相同的未命中。然后它预取与 j 后续未命中相同的地址序列。

其实现方式如下。每个节点在内存中的大型循环缓冲区中记录其缺失序列。当节点 j 在块 A 上发生缺失时，A 的地址被记录在循环缓冲区中地址 X 处。节点 j 将地址 X 发送给目录，以便将其记录为 A 目录状态的一部分。目录可以记录多个这样的流指针。当另一个节点 i 在块 A 上发生缺失时，它联系目录并获取 A 的最新值。它还会从目录收到信息：节点 j 在地址 X 处可以找到一个潜在的流。节点 i

向节点 j 发送流请求，节点 j 将流返回给 i。节点 i 最终可能收到源自地址 A 的多个可能流。然后节点 i 在这些流一致时沿这些流发出预取。预取的数据被放入预取缓冲区。当出现不一致时，节点 i 等待观察自身的需求缺失，以确定哪个流是正确的，然后沿该流继续。当流被消耗一半时，再次联系节点 j 以获取流中的后续条目。虽然在内存

中的循环缓冲区存储流会带来明显的开销，但对流的访问通常不在关键路径上。网络中也会引入更多消息。然而，预取的好处可能是巨大的，尤其是在商业应用中。流的概念并不局限：它能够处理顺序、不规则和指针追逐的数据结构。在后续工作中，Wenisch 等人 [133] 增加了扩展，使实现更加实用。

Ferdman 等人 [134] 还将时间流的概念应用于 L1 指令缓存。与时间数据流类似，指令缓存缺失序列被记录在内存中（并缓存在 L2 中）。L2 中一个块的标签存储指向该块在 L2 中出现位置的指针。这会启动将后续块预取到 L1 指令缓存中。

空间内存流，Somogyi 等人，ISCA 2006

在 2006 年的 ISCA 论文中，Somogyi 等人 [135] 引入了空间相关流的概念。在许多应用中，区域内的访问可能是稀疏的、非步长的，但可重复（因而可预测）。一个位向量可以跟踪区域内通常被触及的缓存块。当对区域的类似遍历再次开始时，位向量指导流的预取。

当一个区域（例如，操作系统页）被触及，硬件结构会跟踪该区域内被触及的块。得到的位向量构成流。流的触发点（或起点）被记录为首次访问该区域的 PC 以及该区域中第一个被触及块的偏移量。当缓存驱逐属于该区域的块时，该区域的流结束。触发点和位向量被记录在预测器中。在后续对区域的首次缺失时，查找预测器；命中时，使用预测的位向量预取该区域中的块。这甚至允许我们准确预取来自可能之前未被触及的区域的块，只要相同的 PC 在不同的区域上执行了类似的遍历。

时空内存流（STeMS），Somogyi 等人，ISCA 2009

空间内存流和时间内存流针对不同的缺失组（有一定重叠）。前者只能识别单个区域（操作系统页）内的块缺失。后者只能对之前访问过的流进行预测。Somogyi 等人 [136] 构建了一个统一的预取机制，可以处理两种访问模式。预测器的时间组件简单地跟踪对每个区域的首次访问以及其流中相应的 PC。流中的这些条目形成触发点，索引到空间组件，

预测器预测该区域内将被访问的所有块。这产生大量需要在不久的将来获取的块。为了不淹没内存系统，必须按适当的顺序向这一大组块发出预取。这要求时间流还记录每个条目之间的缺失次数；它还要求空间预测器不仅跟踪对该区域访问的位向量，还要跟踪确切的缺失序列以及对该区域连续缺失之间的间隔。有了所有这些信息，就可以重建确切的预期缺失序列并及时发出预取。

反馈导向预取，Srinath 等人，HPCA 2007

Srinath 等人 [137] 引入了硬件机制来估计预取的准确性、及时性和污染效应，并使用这些信息来限制预取的激进程度并决定预取块的管理。他们从模仿 IBM Power4 处理器 [138] 的基线流预取器开始。该预取器能够处理多个流，并且在检测到至少三个紧密相邻的缺失后，为流启动预取。流预取器试图保持在起始地址之前 P 个块。每次流中的条目被触及，起始地址前进 N 个块。P 被称为预取距离，N 被称为预取度。这两个杠杆决定了预取器的激进程度。Srinath 等人通过调整 P 和 N 的值采用了五个不同

级别的激进程度。

每个间隔，检查硬件指标以确定预取准确性、及时性和污染。准确性通过为每个预取块存储一个位来跟踪其使用情况来确定。及时性通过查看预取块是否仍在 MSHR 中时被访问来确定。污染通过在一个布隆过滤器风格的结构中跟踪最近被预取驱逐的块地址来确定。基于过去间隔（以及有限程度上的先前间隔）的这些指标，预取激进程度被增加、减少或保持不变。污染指标还用于修改缓存插入策略。根据预取造成的污染程度，预取块被放置在 LRU 优先级栈后半部分的三个位置之一。这些方案有效，并且仅带来少量的存储和复杂性开销。

Hur 和 Lin[139] 此前也考虑过类似的想法。他们同样调整流预取器的激进程度，但这是基于硬件来估计流的典型长度，并估计当前内存访问是否可能属于一个更长的流。该技术的优势在于，它能够先发出预取，然后即使对于非常短的流也能进行缩减。作者们进一步改进预取效果，方法包括使流检测器偏向于识别短流 [140] 等。

Ebrahimi 等人 [117] 进一步利用基于动态反馈信息的预取器节流概念。他们首先为链表数据结构设计了一种新颖的软硬件协同预取机制。然后将其与流预取器结合，并使用运行时准确性/覆盖率指标来节流效果较差的预取器。资源竞争不仅可能发生在使用多种类型的预取器时，还可能发生在处理器的多个核的预取流之间。Ebrahimi 等人 [141] 再次使用动态污

染/准确性/带宽指标来节流特定核的预取，以试图提高整体处理器吞吐量。

3.2.3.2 死块　IATAC，Abella 等人，ACM TACO 2005

Abella 等人 [123] 尝试通过关闭（门控-V_{DD}）被预测为死块的缓存行来降低 L2 的能耗。死块预测通过综合考虑时间和块访问计数来进行。他们首先表明，块命中之间的平均时间始终小于最后一次访问与块逐出之间的时间。他们还表明，这些时间随块在 L2 驻留期间被访问的次数而变化。这些观察结果启发了以下启发式方法，即每次访问计数的访问间隔时间（IATAC），用于死块预测和缓存块关闭。每个块跟踪对其进行的访问次数。给定此访问计数，预测命中之间的平均时间。该预测基于整个缓存的全局指标，并更重视最近被触及的行的行为。一旦块的预测时间过去，该块被视为死块并被禁用。使用此策略，任何时候都有超过一半的缓存被禁用，并且由于过早禁用块产生的额外缺失，性能略有下降。

访问计数预测器，Kharbutli 和 Solihin，IEEE TOC 2008

Kharbutli 和 Solihin[100] 构建了两个死块预测器，它们基于对 L2 中缓存块的访问计数。L2 中的每个块都有一个事件计数器，用于跟踪块当前在 L2 驻留期间的访问次数，以及一个保守地表示块在先前驻留期间行为的阈值。当事件计数器超过阈值时，该块被视为死块。然后，缓存替换策略会将其优先于其他块。阈值估计还用于确定块在其驻留期间是否从未被访问；在这种情况下，如果 L2 空间紧张，则下次取块时该块不会缓存在 L2 中（在许多先前工作中称为缓存绕过）。事件计数器可以采取两种形式。生

存时间预测器（LvP）跟踪块在 L2 驻留期间的访问次数。访问间隔预测器（AIP）跟踪在两次连续访问块之间对块所在集合的访问次数。块的过去行为存储在 L2 和下一级层次结构之间的表中。该表以导致缓存缺失的指令的 PC 和缺失块的地址为索引。表项存储该表项先前看到的最大事件计数器值，并在块被带入 L2 时作为阈值预测。

缓存突发，Liu 等人，MICRO 2008

Liu 等人 [101] 观察到，对块的单个引用进行计数可能导致不准确的预测，特别是对于不规则的数据访问模式。因此，访问以缓存突发的粒度进行计数。一个缓存突发是块在其

作为集合的 MRU 块的连续驻留期间的一系列引用。例如，在其 L2 驻留期间，一个块可能三次访问集合的 MRU 位置，每次访问服务若干次访问。在最后一次访问之后，块从 MRU 位置移动到 LRU 位置，并最终被逐出。在块的下一次 L2 驻留时，当块第三次离开 MRU 位置时，它被预测为死块。先前的死块预测器也使用访问块的指令的 PC 来形成踪迹 [118]；现在，踪迹由导致块移动到 MRU 位置的指令的 PC 形成。Liu 等人表明，这种新的块访问粒度在 L1 和 L2 的死块预测中带来了更高的准确性和覆盖率。预测器再次放置在 L2 和下一级层次结构之间 [100]。他们还表明，块最后一次离开 MRU 位置的时刻也是发出可替换死块的预取的好时机。除了预取触发之外，死块预测还用于在缓存替换策略中实现缓存绕过和优先级排序。Liu 等人的结果表明，不到三分之一的 L2 缓存包含有用的块；这表明 L2 缓存策略总体上还有很大的改进空间。

虚拟牺牲缓存，Khan 等人，PACT 2010

在最近的工作中，Khan 等人 [102] 也展示了 L2 缓存效率改进的类似空间。他们表明，一个缓存块在其 L2 驻留时间的 59% 内是死块。因此，L2 中预测死块的整个池被视为一个虚拟牺牲缓存，可以容纳从 L2 逐出的非死块。死块预测使用 Lai 等人的基于踪迹的预测器 [118]，并进行了一些变化以减少干扰。为了保持实现简单，只有两个集合是“伙伴”。当一个块从一个集合被逐出时，它被放置在其伙伴集合中。伙伴集合逐出一个死块或其 LRU 路。被逐出的块被放置在伙伴集合的 MRU 或 LRU 路中，取决于集合对决实现的结果（见第 3.1.1 节）。还防止块在其主集合和伙伴集合之间乒乓。当查找块时，顺序搜索主集合和伙伴集合。整体设计与 Rolan 等人的集合平衡缓存 [111] 最相似，不同之处在于使用死块预测而不是识别高压和低压集合。该概念也与 Shepherd 缓存 [95] 有相似之处，后者也使用一半的 LLC 作为牺牲文件。

采样死块预测，Khan 等人，MICRO 2010

在最近的一篇 MICRO 2010 论文中，Khan 等人 [142] 提出了对基本死块预测器的增强。与先前工作类似，预测被用于缓存替换和绕过。他们表明，由于前两级的过滤效应，基于迹的死块预测对于三级层次结构中的 LLC 并不那么准确。相反，通过简单地跟踪在块的 LLC 驻留期间最后访问该块的指令的 PC，可以实现高准确率。预测器复杂度通过将所有预测元数据保存在一个单独的标签阵列中来降低，该阵列维护所有 LLC 集合中一部分集合的部分标签。这个采样器结构用于训练一组饱和计数器（真正的预测器），这些计数器跟踪倾向于产生

对块的最后访问。换句话说，少数集合的访问模式被用来识别对整个 LLC 产生最后访问的 PC。当这样的 PC 访问 LLC 中的块时，与该块关联的一个位被置位以指示它是死的。事实上，如果替换策略经常选择死块进行驱逐，则可以从 LLC 中消除 LRU 的元数据（尽管它可能在较小的采样器结构中使用）。因此，总体而言，LLC 设计被大大简化，大部分复杂性被局部化到一个小型采样器结构中。

3.2.3.3 压缩与无用字　在缓存缺失时，整个块被带入缓存，假设访问流中存在空间局部性。然而，在许多工作负载中，一个块中只有一部分字（字通常最多八字节宽）被访问，其余从未被触及。通过仅获取缓存块中有用的字，（i）我们可以增加缓存的有效有用容量，（ii）减少缓存与下一级层次结构之间的带宽需求。类似地，块的压缩是一种正交技术，也可以用来实现同样的两个好处。早期有一些工作 [143，144，145，146，147，148] 研究了仅获取有用块的预测技术，主要针对 L1 缓存。同样，压缩已在内存系统（例如 [149，150，151]）和一些 L1 缓存设计 [152，153] 中得到广泛研究。我们接下来描述将这些概念应用于 L2 缓存的近期工作。

蒸馏缓存，Qureshi 等人，HPCA 2007

Qureshi 等人 [154] 设计了一种技术，与先前工作不同，它不依赖预测器，也不需要从下一级选择性获取块，就能提高性能。在缓存缺失时，整个块被获取并放入 L2。在块驻留 L2 期间，块中各个字的有用性被学习。一旦块漂移到接近 LRU 位置，他们得出结论：到目前为止被触及的字是有用的，其余是无用的。有用的字保留在缓存中，而无用的字被驱逐。这有助于提高缓存的有效容量并提升命中率。

L2 现在被组织成两个 bank——一个行组织缓存（LOC）和一个字组织缓存（WOC）。LOC 类似于传统的 L2 缓存；尽管它的路数较少。当一个数据块从 LOC 中被驱逐时，其有用的字被移动到 WOC。WOC 有许多路，每路包含单个字；因此它具有很高的标签存储开销。对于表现出高度空间局部性的工作负载，作者引入了一种基于集合决斗监视器（见第 3.1.1 节）的方案，以恢复回传统的缓存组织。从 LOC 到 WOC 的转移只发生在 L2 驱逐的一半行上——那些有用字百分比低的行。作者还尝试了一种对 WOC 中的字进行压缩的优化。

虽然该技术提高了缺失率和性能，但实现需要相当多的复杂性；因此，要实现识别无用字的潜力，可能还需要更多的研究。主要的存储和能量开销由 WOC 的标签存储引入。由于只有 L1 缓存看到对单个字的访问，L1 缓存必须跟踪

每个字的有用性，并将该信息合并到块的 L2 副本中。在 L1 和 L2 中需要额外的位来跟踪字的有用性，并且在 L2 和许多 L1 之间引入了不平凡的交互（特别是如果 L1-L2 层次结构是非包含的）。需要在 LOC 和 WOC 之间进行复制，并且必须遵守某些对齐约束。WOC 中的命中导致 L1 内出现部分块，需要使用分扇区的 L1 缓存 [146]。恢复回传统组织也需要额外的标签存储。

自适应缓存压缩，Alameldeen 和 Wood，ISCA 2004

数据块压缩引入了一个基本的容量/延迟权衡：压缩块增加了容量，但由于必须解压缩，它们导致更高的读取延迟。为了在这种权衡中取得平衡，Alameldeen 和 Wood [155] 首先避免在延迟关键的 L1 缓存中进行压缩（类似于 Lee 等人 [156] 的方法）。在 L2 缓存中，只有当整个缓存似乎从压缩中受益时，才对传入块采用压缩。为了估计这种收益，在每次 L2 访问时，如果压缩可以避免一次缺失，则饱和计数器增加 L2 缺失代价；如果即使没有压缩该访问也会命中，则减少解压缩延迟。L2 缓存的组织方式使得最多可以存储八个标签，并且最多可以存储四个未压缩数据块。标签存储中的信息（八个块的标签以及每个块的可压缩性）允许我们计算增加/减少饱和计数器的条件。

因此，该技术能够提高工作集较大的应用的命中率，同时不会增加不需要高 L2 容量的应用的命中时间。然而，还必须考虑一些实现复杂性。更大的标签存储使 L2 面积增加约 7%。需要压缩和解压缩流水线，Alameldeen 和 Wood 表明这可以在五个周期内完成。压缩算法识别出七种不同的可压缩数据类别，并为块中的每个字附加一个 3 位前缀。在 L2 中移除或替换一行可能有些复杂，因为一个集合中的所有数据元素必须连续，有时可能需要逐出多个条目。

带压缩的间接索引缓存（IIC-C），Hallnor 和 Reinhardt，HPCA 2005

在 2005 年的 HPCA 论文中，Hallnor 和 Reinhardt [157] 首先提出，压缩应应用于 LLC、内存总线和内存中。这会在每一级带来改进，并减少在各层级之间传输时的压缩和解压缩开销。其次，他们基于 Lee 等人 [156] 的见解，将压缩限制在 L3，并针对低延迟优化更高层级的缓存。第三，他们扩展了自己先前的间接索引缓存（IIC）设计 [158]，以优雅地处理压缩块。

IIC 将块的标签放置在少数可能位置之一。标签携带指向数据数组中块位置的指针。因此，数据数组是完全关联的，代价是更昂贵的标签数组。引入压缩后（IIC-C 设计），一个块被分散到

四个子块；未压缩的行使用全部四个子块，而压缩块使用 0、1 或 2 个子块。标签现在必须存储指向每个子块的指针。因此，标签存储变得更加昂贵（它还要求条目数是数据数组的两倍）。对于 64 字节的块，这种开销可能高达 25%，因此，这些技术仅对较大尺寸的块和子块有效。然而，数据放置的灵活性消除了 Alameldeen 和 Wood 的设计 [155] 引入的替换复杂性。

为了进一步降低 L3 中解压缩的延迟成本，Hallnor 和 Reinhardt 增强了 IIC 的分代替换策略。数据数组被组织为按优先级排列的 FIFO 池。在未命中时，块必须离开其池；根据它是否被访问过，它会移动到优先级更高或更低的池。当 L3 中的块被访问时，它会被解压缩。解压缩后的版本被放回 L3。如果池中没有未命中，该块以未压缩形式保留在那里，其访问可以快速得到服务。如果池遭受许多未命中，未压缩的块最终将移动到不同的池；此时，它会被再次压缩。因此，工作集较小的应用不会因解压缩延迟而受到太大影响，而工作集较大的应用会频繁压缩其块以腾出空间。这以完全不同的方式实现了与 Alameldeen 和 Wood 的自适应缓存压缩类似的目标，但它确实需要许多块复制和压缩/解压缩周期。

压缩与预取，Alameldeen 和 Wood，HPCA 2007

本节中的一些主题可以产生协同交互。我们之前提到了 Qureshi 等人对压缩和有用字技术组合的评估 [154]。接下来，我们描述 Alameldeen 和 Wood 对压缩和预取技术组合的评估 [116]。本质上，这两种技术是互补的，同时使用两种技术的加速比大于每种单独技术加速比的乘积。这是因为预取受到带宽压力和缓存污染的困扰，而压缩可以缓解这两个问题；同样，解压缩的延迟成本可以通过预取到 L1 来隐藏。Alameldeen 和 Wood 的一个关键贡献是一种自适应预取方案，它根据预取块的成本/收益来限制预取。收益通过跟踪预取块的使用情况来估计。成本通过利用尚未完全压缩的集合中存在额外标签来估计（缓存组织基于他们之前描述的工作 [155]）。这些额外标签跟踪最近被逐出的块；这有助于跟踪预取块逐出其他有用块的情况。上述事件通过一个饱和计数器进行跟踪，当计数器在一个方向上饱和时禁用预取。这在多核芯片中尤为重要，因为在这些芯片中预取被证明不那么有效。

CHOP，Jiang 等人，HPCA 2010

我们以一项最接近块级优化精神的技术来结束本节。本质上，该技术采用非常大的块，并且仅当块中的元素表现出高度的空间和时间局部性时才选择缓存该块。Jiang 等人 [159] 考虑使用

一个由嵌入式 DRAM 组织的大型 LLC，可能作为 3D 堆叠。如果数据以典型缓存块（最多 128 字节）的粒度组织在这个大型 LLC 中，标签的开销将极高。相反，如果数据以操作系统页面的粒度组织，页面内的许多块将不会被使用，内存带宽将被浪费。因此，Jiang 等人主张大型 LLC 用于以页面粒度存储数据，并设计过滤机制，仅获取预测为“热”的页面。热页面定义为访问次数最多的页面，这些页面合计占所有访问的 80%，并且对于所选工作负载，恰好占所有页面的 25%。过滤缓存是一种片上结构，用于监视对尚未成为 LLC 一部分的页面的片外访问。默认情况下，页面在首次接触时不会分配到片外 LLC 中。随着时间的推移，如果过滤缓存中的页面被认为是热的，则将其分配到 LLC 中。当页面从 LLC 中被逐出时（基于 LFU 替换策略），它会以某个初始计数器值重新放回过滤缓存中。当页面从过滤缓存中被逐出时，其计数器值可以保存在主存中，以便以后无需从头重新计算其活动水平。Jiang 等人还采用了一种自适应策略，如果内存带宽利用率低，则偶尔禁用过滤机制的使用。

3.3　小结

本章首先探讨了用于缓存分区的不同技术，以最大化吞吐量并提供服务质量（QoS）。基于效用的路分区已被频繁采用，并且可以以相对较低的开销实现。缺失率曲线可以通过采样的影子标签构建，其开销仅为几 KB 量级。但最终，末级缓存（LLC）的替换策略和分区似乎正朝着创造性插入机制的方向发展。将一个块插入到集合优先级列表的末尾附近（RRIP）是一种低开销的方法，仅在块被证明有用后才保留它。同样，插入点也可用于让一个应用优先于另一个应用，从而实现隐式缓存分区（PIPP）。

插入点机制需要非常少的逻辑和存储，因为它们可以通过少量集合样本和计数器来评估竞争策略。除了为工作负载计算最优缓存分区外，操作系统还必须尝试创建一种调度，使互补的应用能够同时共享缓存。最近的一些论文使用缺失率曲线来构建此类调度（ACCESS、DIO）。QoS 机制同样需要缺失率曲线和各种运行时统计数据来计算资源分配，并监控 QoS 保证是否未被满足。许多 QoS 论文的区别主要在于所分配的资源（缓存、带宽、路数、分子等）以及这如何影响程序 IPC。

对于替换策略，新颖的插入机制似乎因其简单性而成为未来的方向。虽然一些研究尝试了近似 Belady 的 OPT 算法的创造性替换策略，但它们不可避免地会在存储和逻辑复杂性方面产生更高的开销。同样，许多提高相联度的策略会带来不小的复杂性。它们可能不得不在商业采用上面临一场艰苦的战斗，尤其是因为软件方案（页着色或调度）可以帮助平衡各集合之间的负载。

大多数商业处理器主要使用基于步长或流的预取器。最近的一些工作提出了通过检查行为来控制步长预取引擎参数的策略。这些策略看起来足够简单，可以在商业上采用。关于流（地址中没有明显模式的数据块）的预取也有广泛的研究。虽然这些技术有效，但它们经常要求将流存储在内存中，这一特性可能会遇到可扩展性挑战。死块预测器已取得重大进展，但它们主要在为逐出确定块优先级时有用。Khan 等人的采样死块预测器可能足够简单，可以与其他基于插入的替换策略竞争。一些块级优化（压缩、过滤）会导致可变大小的块。支持可变大小块的缓存组织在标签和数据放置方面会带来显著的复杂性。最终，最近在缓存策略方面的研究也将在层次结构的较低级别具有潜力，CHOP 就是一个应用于片外 DRAM 缓存的策略示例。

3.3 小结

第四章　大型缓存内的互连网络

几十年来，SRAM 缓存中的存储单元变化相对较小。SRAM 存储单元仍然是传统的 6 晶体管设计，尽管未来正在考虑新颖的抗变异替代方案（见第 5.5 节）。

另一方面，用于连接缓存子阵列的互连一直是多项创新的焦点。十多年来，人们已经知道，当工艺技术缩小时，片上导线的缩放效果不如晶体管逻辑 [160]。对于全局长距离导线尤其如此。由于多年来缓存占用的面积没有太大变化（缓存继续占据约 50% 的芯片面积），缓存内的导线长度相对不变，但它们在延迟和能耗方面代表了更大的瓶颈（稍后将量化）。本章描述了缓解大型缓存层次结构中布线瓶颈的几种策略。

4.1　基本大型缓存设计

4.1.1　缓存阵列设计

一个简单的缓存由一组 SRAM 单元组成，这些单元被组织成多个二维阵列以存储数据和标签。图 4.1 展示了一个具有单个数据阵列和标签阵列的缓存的组织结构。每个存储单元都配有促进读或写操作的访问晶体管，访问晶体管通过称为字线的水平布线进行控制。集中式解码器接收地址请求，并在数据阵列和标签阵列中识别出相应的字线。为了减少解码器的面积开销和布线复杂性，一条字线由一整行单元共享。每次访问时，解码器对输入地址进行解码，并激活与地址匹配的相应字线。一整行单元的内容被放置到位线上，对于写操作，这些内容被写入单元；对于读操作，则被传递到输出。尽管放置到位线上的数据可以直接驱动小型阵列的逻辑，但由于位线电容很大，即使对于中等大小的阵列，这种设计的访问延迟也可能非常高。为了减少访问时间，采用灵敏放大器来检测位线信号的微小变化，并将其放大到逻辑摆幅。灵敏放大器还限制了位线中的电压摆幅，从而限制了其能耗。

读写 SRAM 单元本身的能量和延迟非常小；读取一个 SRAM 单元只需要 CPU 周期的一小部分，而 SRAM 单元中的动态能耗

微不足道。缓存访问延迟主要受与解码器、字线、位线和输出总线相关的导线延迟支配。同样，读或写期间激活的位线数量以及输出总线的长度决定了缓存的动态功耗。

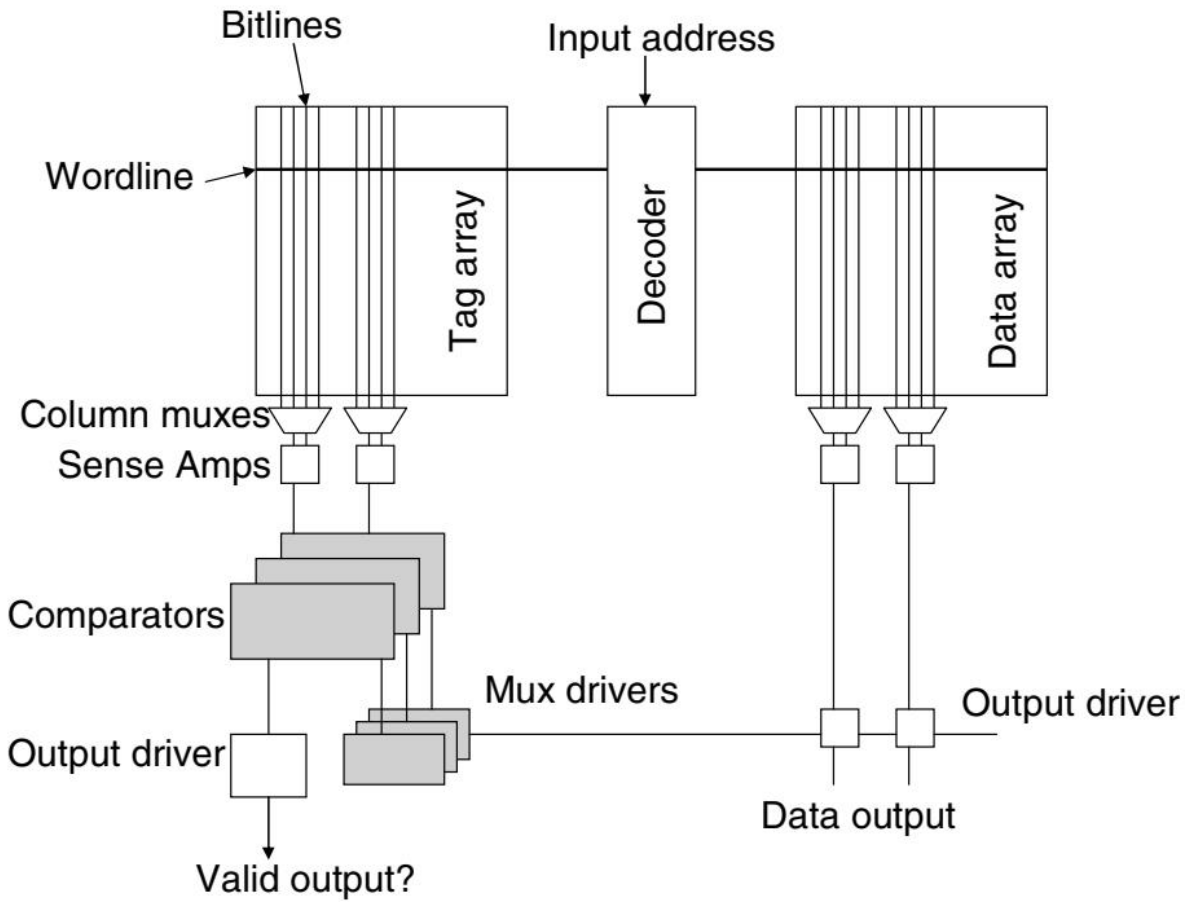


图 4.1 缓存的逻辑组织。

4.1.2 缓存互连

图 4.1 所示的单片模型无法随阵列尺寸扩展。由于位线采用低摆幅信号，且与中继器、字线和位线相关的硅面积开销，它们无法以固定间隔重复，导致其延迟随阵列尺寸呈二次方增长。此外，RAM 的吞吐量是字线和位线延迟的函数，单个阵列的带宽会随着阵列尺寸增大而迅速恶化。为减轻这种二次方延迟影响，大阵列通常被划分为多个子阵列。许多子阵列构成一个 bank。一个 bank 内的所有子阵列共享输入地址信号和输入/输出数据信号。大型缓存 bank 几乎从不采用多端口设计。如果采用流水线，一个 bank 可以同时处理多个请求。大型 L2 或 L3 缓存通常被划分为多个 bank，每个 bank 独立运行。每个 bank 可以在同一周期内同时接收/服务单个请求。bank 内部需要互连系统来连接子阵列。还需要更复杂的互连系统来连接 bank、核心和其他组件。bank 内系统通常是 H 树网络（如前面图 1.7 所示）。H 树网络为每个子阵列提供均匀延迟，极大简化了必要的调度逻辑。在 4.2.2 节中，我们将讨论 H 树网络的作用及潜在优化。本章大部分内容将聚焦于问题更多的 bank 间网络。

在规模相对较小的多核中，bank 间网络由交叉开关构成。例如，Piranha 芯片（约 2000 年）有八个核心和一个 1 MB 的 L2 缓存，组织为八个独立 bank [21]。一个芯片内交换机（ICS），本质上是一个交叉开关，用于将缓存 bank 连接到核心。因此，一个

缓存请求必须经过几根长导线，经历输出端口的仲裁，如果请求未立即获准可能还需缓冲，并通过交叉开关逻辑才能到达缓存 bank。Sun 的 Niagara 处理器也是如此，它使用交叉开关将八个核心连接到 3 MB L2 缓存的四个 bank [161]。Kumar 等人 [162] 描述并分析了连接中等数量核心和缓存 bank 可能需要的互连。

随着核心和缓存 bank 数量增加，交叉开关的复杂性和必要的布线会急剧增加。穿越芯片半程到达集中式交叉开关也是低效的。通常认为，数十个核心和缓存 bank 必须用不包含集中式共享组件的可扩展网络连接。事实上，未来大型缓存将分布在芯片上，页着色等技术将确保线程的数据很可能驻留在附近的缓存 bank 中。因此，附近组件必须能在不经过集中式结构的情况下通信。可扩展网络可以有多种形式，但近期文献中采用的主要是分组交换路由网络 [163, 164]。这种网络包含许多路由器，独立链路将路由器和组件连接到路由器，分组通过多个路由器从源发送到目的地。这些网络更具可扩展性，因为多个分组同时在不同链路上传输，交叉开关的集中式功能现在分布到多个路由器上，并且如果核心倾向于访问附近缓存 bank 中的数据，平均传输距离较短。

4.1.3　分组交换路由网络

接下来我们将简要介绍分组交换路由网络的设计。有兴趣的读者可以查阅《片上网络》综合讲座 [11] 了解更多细节。

拓扑任何片上网络最基本的属性是其拓扑，它描述了连接所有路由器和组件所需的边。常见的片上拓扑按边数（以及性能和成本）递增的顺序包括一维阵列、环、二维阵列或网格、环面、扁平蝶形和超立方体。网格是迄今为止最常用的拓扑，其中组件和路由器布置在二维阵列中，每个路由器连接到相邻组件以及北、南、东、西四个相邻路由器。除了

网络外围的路由器外，每个路由器的度为五，即每个路由器有五个输入端口和五个输出端口。

路由算法路由算法通常分为 oblivious（oblivious）或 adaptive（自适应）两类。Oblivious 路由算法使用的路径仅取决于源和目的地，而不取决于网络中的事件或负载。Oblivious 路由算法的一个重要子集是确定性路由算法。确定性路由为每个源-目的地对使用唯一的预定路径。确定性路由的一个常见例子是维序路由，其中消息首先沿第一拓扑维度路由，直到在该维度上到达与目的地相同的坐标；然后消息沿第二维度路由，依此类推。例如，在二维阵列中，如果源表示为（3，5），目的地为（7，9），消息首先沿 X 维度路由直到到达（7，5），然后沿 Y 维度行进直到到达（7，9）。另一方面，自适应路由可以将预定路径作为指导，但消息可以偏离此路径以避开拥塞的路由器。自适应路由算法的一个子集是最小路由算法。最小路由允许根据负载动态确定路径，但确保每一跳的方向都使消息更接近目的地。

缓冲、交叉开关、通道每个路由器都有一个交叉开关，允许任何输入端口的数据传

输到任何输出端口。如果多个传入消息希望前往同一个输出端口，仲裁单元必须将输出端口授予其中一个消息。输出端口的授予等同于授予连接到该输出端口的链路（也常称为通道）。失败的消息随后在其输入端口被缓冲。每个输入端口都有一组缓冲区。每个消息为了向前推进，必须竞争并获取两种主要物理资源：一个是用于下一跳的物理通道，另一个是它所访问的下一个路由器输入端口的空闲缓冲区条目。例如，如果一个消息从路由器 A 到路由器 B 再到路由器 C，它必须竞争获取连接 A 和 B 的通道，并且还必须确认 B 的相应输入端口有空闲缓冲区条目来容纳此消息，以防消息不能立即跳到 C（因为可能同时到达 B 其他输入端口的其他消息的竞争）。术语“流控”指的是消息在网络中传播时用于预留资源（缓冲区和通道）的策略。

数据包在我们讨论流控选项之前，让我们首先描述消息可以划分成的各种粒度。一个消息被划分为多个数据包。每个数据包都有一个头部，允许接收节点重新构建原始消息，即使数据包是乱序接收的。由于数据包是一个自给自足的单位，每个数据包可以在网络上独立路由，一个消息的不同数据包可能遵循不同的路径。这可以在使用自适应路由算法的网络中实现更好的负载分布。在片上网络中，消息通常很短，很少超过缓存行的大小。因此，消息很少被分成更小的数据包，这两个术语是同义的。常见的数据包类别包括：

数据（64 B 缓存行）、请求（4 或 8 B 地址）、控制（一致性消息，可能只有几位，如果包含地址则最多 10 B）。

微片数据包进一步划分为微片。一个微片通常与通道宽度大小相同，因此代表资源分配的最小粒度。如果通道宽 64 位，并且正在传输一个 64 字节（缓存行）的数据包，则该数据包被组织成八个微片，在八个周期内在通道上传输。微片不携带额外的头部。一个数据包的微片不能分离，否则接收方将不知道如何重建数据包。因此，一个数据包的所有微片遵循相同的路径，并严格按照顺序传输。头微片携带数据包的头部信息，路由器使用它来决定数据包的输出端口。一旦做出这个决定，数据包的所有后续微片都被发送到同一个输出端口，直到遇到尾微片。通过这样的架构，数据包可以尽可能大（以减少头部开销），但资源可以以微片的更细粒度进行分配，从而提高整体网络资源利用率。为了理解这一点，考虑以下类比：如果我们试图用沙子而不是岩石填充桶，桶的容积会得到更好的利用，即一个实体如果需求更少的资源，就有更好的机会在网络中传播自身。值得指出的是，虽然通道可能宽 64 位，但它将有额外的线（多达 20% 更多的线）用于控制和 ECC 信号。数据包头部还可能包括校验和或 ECC 码。偶尔，如果物理通道宽度小于一个微片，一个微片可能通过通道在多个周期内传输；每个周期的传输称为一个 phit。

流控流控策略在缓冲区和通道分配的粒度上有所不同。这些策略首先分为无缓冲和有缓冲两类。

无缓冲流控无缓冲流控可能具有能效优势，因为它消除了网络中的缓冲区。研究将片上网络总功耗的大约三分之一归因于路由器缓冲区 [165, 166, 167, 168]。在所谓的“热土豆”或“偏转”路由中，如果期望的输出端口忙，传入的微片被发送到另一个空

闲的输出端口。如果输入端口数量与输出端口数量匹配，应该总是可能将所有传入微片转发到某个空闲输出端口。这增加了平均消息所需的跳数和延迟，但简化了路由器。这种设计正被强烈考虑用于现代能量受限的多核 [169]。电路交换网络是无缓冲流控的另一个例子。当必须发送消息时，首先在网络上发送一个短的探测消息。这个探测消息找到一条一直到达目的地的路径，预留它经过的所有通道。一个确认被发送回发送节点。然后实际消息被发送到目的地，中间路由器不需要缓冲，因为所有需要的通道（输出端口）已经被预留。消息的尾微片在通过时释放通道。由于初始探测消息的往返延迟，这种流控机制有明显的延迟开销。这种开销可以在大消息传输中摊销，但正如我们已经指出的，在片上流量中消息往往很短。虽然

这种技术不需要为消息提供缓冲，但它确实需要为探测消息提供一些有限的缓冲，因为探测消息在等待所需通道腾空。

缓冲流控缓冲流控可以在粗粒度上通过分组缓冲流控实现，此时信道和缓冲区按分组分配。当一个分组想要从路由器 A 移动到相邻的路由器 B 时，它必须确保 B 的输入端口有足够的空闲缓冲空间来容纳整个分组。然后，从 A 到 B 的信道被保留给该分组。在接下来的许多周期里，整个分组从 A 传输到 B，不受任何其他分组的干扰。这种流控的两个常见例子是存储转发和直通。在前者中，整个分组必须被 B 接收后，头微片才能开始向下一路由器 C 传输。在直通中，头微片可以在等待整个分组被 B 接收之前继续向 C 传输。直通通常导致更低的延迟，但它可能更加资源密集，因为传输中的分组分散在多个路由器上，并且在每个路由器上保留了分组大小的缓冲区。虫洞流控通过按微片分配缓冲区，改进了直通中的存储低效问题。这是一种微片缓冲流控形式，但信道仍然按分组保留，即一旦一个分组开始在一个信道上传输，其他分组就不能占用该信道，直到第一个分组的尾微片被传输完毕。使用虫洞流控时，一个分组可能会停滞，其各个微片分散在多个相邻路由器中。当一个分组以这种方式停滞时，它会阻止相关信道被其他可能更容易向前推进的分组使用。这就引出了缓冲流控的最后一种形式——虚拟信道流控，它也按微片分配信道。

虚拟信道流控在虚拟信道流控中，假设多个分组可以同时在一个链路或物理信道上传输。我们之前提到，微片没有头部，一个分组的微片必须在信道上顺序传输。为了让多个分组共享一个物理信道，每个微片必须携带几位头部信息，以标识它所属的传输中的分组。当一个路由器接收到一个微片时，使用这几位头部信息将微片导向其所属的适当分组。因此，就好像两个路由器之间存在多个虚拟信道，每个分组在自己的虚拟信道上传输，并在另一端由自己的一组缓冲区接收。实际上，可能只有一个物理信道和一组共享缓冲区，但虚拟资源被多路复用到物理资源上。如果我们假设有四个虚拟信道（VC），那么一个物理信道上可以有四个分组在传输。每个周期，属于四个分组之一的一个微片（由 2 位头部标识）可以在物理信道上传输。这种在单个物理信道上对多个虚拟信道的时间分复用允许其他分组在有些分组可能停滞时向前推进。在每个路由器的输入端口，需要为四个分组中的每一个维护状态。该状态必须跟踪每个分组的头微片为下一跳选择的输出端口和虚拟信道；该分组的后续微片直接转发到该

输出端口和虚拟信道。

虚拟信道流控机制被认为是片上网络的黄金标准。它不仅产生最高的性能，虚拟信道还用于避免死锁（稍后讨论）。对于这样的架构，在分组可以转发到下一个路由器之前，必须获取各种资源。

虚拟信道分配首先，头微片必须在需要遍历的链路上获取一个空闲的虚拟信道。虚拟信道状态位于接收路由器的输入端口。发送路由器一旦在该虚拟信道上发送尾微片，就意识到虚拟信道可用。即使使用该虚拟信道的上一个分组被缓冲在接收路由器中，下一个分组也可以在同一虚拟信道上开始传输；下一个分组的微片只是被缓冲在较早的分组之后，并将在第一个分组被转发后处理。分组的其余主体微片不需要竞争获取虚拟信道。

缓冲区管理其次，每个微片必须确保在下一个路由器中存在空闲的缓冲条目，然后才能跳转过去。这通常通过两种方法之一来确定。在基于信用的方案中，每当一个路由器转发一个微片时，会向先前（上游）路由器发送一个信号，通知它一个缓冲条目已被释放。因此，路由器知道所有相邻（下游）路由器的空闲缓冲区数量，但有轻微的时间延迟。在开/关方案中，当下游路由器的缓冲区接近满时，它会向上游路由器发送信号。上游路由器停止发送微片，直到下游路由器停用该信号。激活和停用信号的阈值是兩個路由器之间链路延迟的函数。后一种方案减少了上游信令和计数器，但浪费了稍多的缓冲空间。

物理信道分配第三，每个微片必须竞争物理信道。如果我们假设每个物理信道有四个虚拟信道与之关联，那么在一个周期内最多可能有四个微片准备好传输，但只有一个可以在物理信道上传输。这是在该路由器做出的本地决策。

如果我们以一个 5x5 路由器（5 个输入端口和 5 个输出端口）为例，每个物理信道有四个虚拟信道，那么可能有高达 16 个传入分组（在 4 个输入端口和每个端口上的 4 个虚拟信道上）都希望前往同一个输出端口。这 16 个分组的头微片将竞争所需输出端口上的四个可用虚拟信道。假设下游路由器中存在空闲缓冲区，来自四个获胜分组的微片将在每个周期竞争物理信道。

基本路由器流水线这引出了典型路由器流水线的设计。共有四个基本阶段：

—路由计算（RC）：当遇到头微片时，检查其头部以确定最终目的地。调用路由算法来确定该数据包应转发到的输出端口。自适应算法在做出此决策时会考虑各种负载指标。为每个输入端口上的每个 VC 维护状态，

4. 大型缓存内的互连网络

并在此记录输出端口。该数据包的后续体微片无需经过 RC 阶段；它们只需采用头微片计算出的输出端口。

—VC 分配（VA）：一旦确定了输出端口，头微片必须在选定的输出端口获取一个空闲 VC。如果 VC 未立即授予，头微片必须每周期继续竞争，直到分配到一个 VC。分配的输出端口 VC 也记录在输入端口 VC 为该数据包维护的状态中。同样，后续体微

片跳过此阶段，并采用头微片获取的 VC。

—开关分配（SA）：所有微片都经过此阶段。每个路由器的核心是一个交叉开关，允许任何输入端口上的微片发送到任何输出端口。数据包的前导微片必须与其他数据包竞争输出端口（物理通道）的访问权。SA 阶段将可用的输出端口分配给竞争微片的子集。

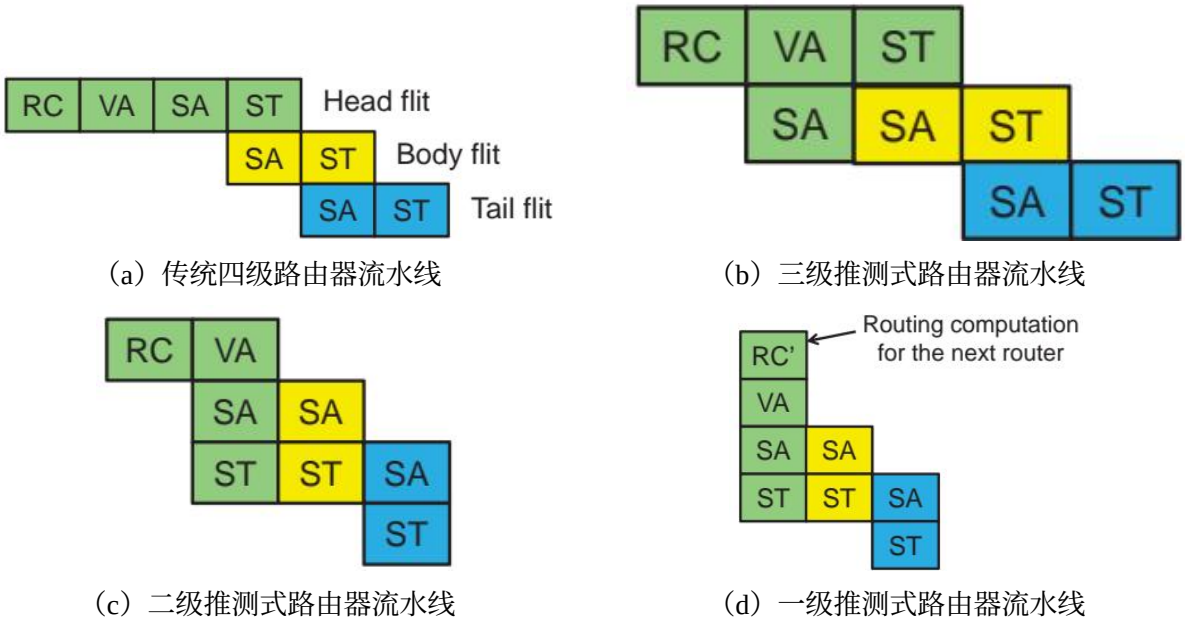
—开关遍历（ST）：被分配输出端口的微片在此阶段开始遍历。微片从其输入端口缓冲区读出并穿过交叉开关。此阶段之后是链路上的多个遍历阶段。例如，如果链路有 3 个周期的延迟，通常将其流水线化为 3 个锁存阶段。

图 4.2（a）展示了一个示例流水线及数据包通过流水线的遍历过程。我们提到，微片在跳转前必须获取 VC、物理通道和缓冲区——虽然 VA 和 SA 阶段分配 VC 和物理通道，但没有单独的阶段来分配缓冲区。对信用或开/关状态的快速检查作为 SA 阶段的一部分进行。

推测式路由器流水线刚才描述的路由器具有 4 级或 4 周期流水线。通过推测技术可以减少通过路由器的延迟 [163, 170]。图 4.2 还展示了示例推测式路由器流水线。在第一个示例（b）中，VA 和 SA 阶段并行执行。微片必须假设它将在 VA 阶段成功获取 VC，并在 SA 阶段推测性地竞争输出端口。仅当微片是头微片且 VC 竞争激烈时，推测才会失败。在低负载条件下，这种推测通常会成功并减少通过路由器的延迟。在高负载条件下，推测可能失败，此时流水线按顺序遍历。唯一的代价是输出端口可能已分配给未获取 VC 的微片，因此输出端口空闲一个周期。在高负载条件下，消息的瓶颈是缓冲区中的排队延迟，而不是路由器流水线延迟。因此幸运的是，推测在低负载条件下有帮助，此时路由器流水线延迟确实是瓶颈。

在图 4.2（c）的第二个推测式路由器示例中，VA、SA 和 ST 阶段可以并行发生。微片直接跳转到输出端口，假设它们将成功获取 VC 和输出端口。这种推测在低负载条件下同样具有高准确性。最后一个示例（d）实现了一个 1 周期流水线，其中所有 4 个阶段并行遍历。虽然可能预测 RC 阶段选择的输出端口，但这很可能是一个低准确性预测。因此，我们假设当微片到达路由器时，它附带几个比特，指定该微片的最佳输出端口。换句话说，路由是预先计算的。由于输出端口和下游路由器已知，RC 阶段现在尝试估计下游路由器应使用的输出端口。这个为下一个路由器预先计算的路由随微片一起发送到下游路由器。严格来说，路由器操作被流水线化为 2 个顺序周期，但分布在两个路由器上。这也称为前瞻路由。假设推测成功，给定路由器的延迟仅为一个周期。我们不知道有商业路由器采用如此高度的推测。显然，推测式路由器可能消耗更多能量并带来更高的复杂性。

死锁避免网络路由和流控制算法还必须包含死锁避免特性。当存在资源依赖循环时，就会产生死锁。考虑一个试图从路由器 A 跳转到路由器 B 的微片。该微片当前占用路由器 A 输入端口的一个缓冲区条目，并试图在路由器 B 的输入端口获取一个空闲缓冲区。路由器 A 中缓冲区资源的释放取决于路由器 B 中缓冲区资源的释放——这是一种资源依赖。如图 4.3（a）所示，四个试图转向的数据包可能最终导致资源死锁。通



（a）传统四级路由器流水线　（b）三级推测式路由器流水线

（c）二级推测式路由器流水线　（d）一级推测式路由器流水线

图 4.2　传统与推测式路由器流水线示例。

过四转向模型（图 4.3（b））可以最好地理解死锁。无限制的自适应路由算法可能导致死锁，因为所有四种可能的转向

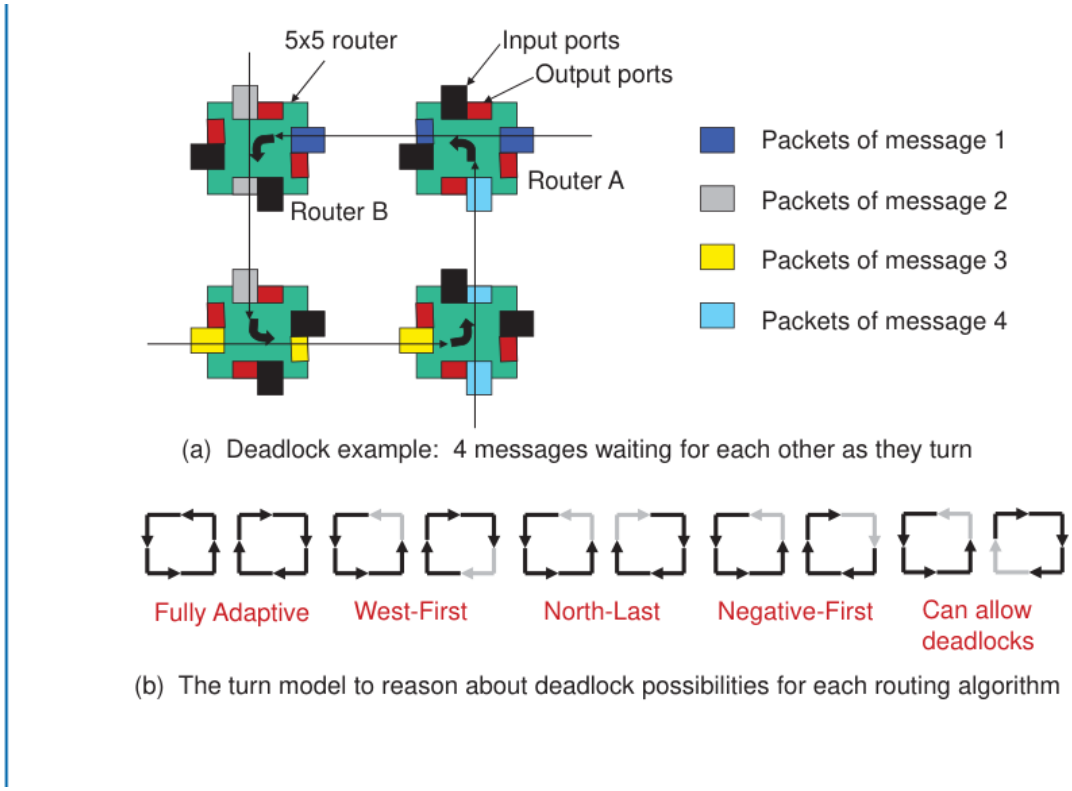


图 4.3　网络死锁示例及四转向模型。

（在顺时针和逆时针循环中）都是可能的，可能导致图 4.3（a）所示的死锁。维度顺序路由算法是无死锁的，因为它只允许循环所需的四种转向中的两种。虚拟通道也可

用于死锁避免。假设每个 VC 有一组与之关联的缓冲区，并且缓冲区编号。只要数据包按数字升序获取 VC 缓冲区，就允许任意转向。这保证不会发生资源依赖循环。当明显较高编号的 VC 缓冲区可能不可用时，数据包求助于无死锁路由算法，如维度顺序路由。

路由器瓶颈之所以如此详细地探讨路由器设计，是因为片上网络是访问大容量缓存不可或缺的一部分。从目前的讨论可以看出，可扩展网络可能需要具有复杂功能的路由器。虽然我们的大部分讨论集中在虚拟通道路由器上，但即使更简单的路由器也需要类似的功能。研究表明，片上网络在总芯片功耗中占据了相当大的比例（10-36% [165，168，171]）。因此，在考虑大容量缓存的性能和能耗优化时，不能将路由器和网络访问抽象掉。

4.2　互连设计对 NUCA 和 UCA 缓存的影响

路由器本身包含几个关键组件。尽管文献中的能耗分解有所不同，但可以公平地说，链路、缓冲区和交叉开关在网络总能耗中贡献了相当大的部分。交叉开关的能耗和面积复杂度随着端口数量和 flit 宽度的增加而显著增长。缓冲区能耗也随着端口数量、VC 数量和网络负载的增加而增长。仲裁器和路由逻辑消耗的面积不容忽视，但对整体能耗贡献不大。

路由器优化目前正在尝试解决这些瓶颈。许多近期的路由器优化针对交叉开关和缓冲区能耗 [167, 169, 172, 173]。链路能耗也通过低摆幅布线 [174] 得以降低。使用快速通道可以完全绕过路由器遍历。物理快速通道 [175] 是连接芯片区域的长距离链路。长距离传输的数据包大部分时间可以跳上物理快速通道，跳过许多中间路由器的遍历。缺点是某些路由器更复杂，因为它们必须同时容纳快速通道和常规通道。虚拟快速通道架构 [176] 使用常规拓扑，但某些虚拟通道保留用于长距离通信。在这种虚拟快速通道上的数据包可以绕过中间节点的整个路由器流水线。集中是另一种减少网络规模和平均所需跳数的技术，但它增加了路由器度数和复杂度 [177]。节能高性能网络和路由器的设计仍然是一个开放问题。这是一个活跃的研究领域，不是本书的主要焦点。上面列出的优化都是网络优化的例子，它们某种程度上与缓存架构无关，即它们适用于任何网络和几乎任何流量模式。我们将后续关于网络创新的讨论限制在与缓存层次结构组织密切相关的想法上。

4.2.1　NUCA 缓存

人们早就知道，单片大容量缓存会遭受长延迟和高能耗。大容量缓存不可避免地会被划分为更小的存储体，可能分布在芯片上，并通过可扩展网络连接。许多早期对此类 NUCA 架构的评估对片上可扩展网络做了简化假设。假设每一网络跳消耗一个周期，网络能耗很少被考虑。这导致了早期的 NUCA 设计，其中缓存被划分为 64-256 个存储体，并且需要许多网络跳才能到达相对较小的 64 KB 缓存存储体。在过去五年左

右的时间里，人们对路由器内部的复杂性及其延迟和能耗开销有了更深入的认识。虽然将缓存划分为多个存储体以改善缓存组件内的延迟和能耗可能是可取的，但这是以增加网络延迟和能耗为代价的。Muralimanohar 等人 [4, 7, 178] 通过构建缓存和网络的联合模型对此进行了量化。他们通过全面的设计空间探索表明，最优的大容量缓存组织是在缓存仅划分为少数几个存储体时实现的

此时更多的能量和延迟消耗在缓存存储体内，而片上网络内消耗的能量和延迟则少得多。

这是为任何大容量缓存研究建立基线参数的重要指导原则。设计空间探索在流行的公开分发的 CACTI 缓存建模工具 [4] 中建模。它还提供了网络和缓存组件内消耗的能量/延迟的分解，使研究人员能够对组件优化带来的整体收益进行 Amdahl 定律估算。

Muralimanohar 等人的大部分分析集中在物理上连续的大容量缓存上。这代表了 Beckmann 和 Wood [2] 以及 Huh 等人 [22] 的布局，其中缓存放置在芯片中心并被核心包围。这也可能代表一种缓存层次结构，其中大容量 L2 缓存分布在芯片上，并由同一芯片或 3D 堆叠芯片上的大容量物理连续 L3 缓存进一步支持。请注意，分布式缓存的布局不够通用，无法合理地纳入模型。因此，CACTI 6.0 模型侧重于物理连续的大容量缓存，但提供了手动建模分布式缓存属性所需的工具。

CACTI 6.0 对缓存存储体内的常见元素进行建模，如前文第 4.1.1 节所述。此外，它还建模片上网络的路由器和连线。考虑了不同类型的连线。考虑了具有不同宽度/间距和中继器尺寸/间距的传统基于 RC 的连线。每种连线类型在延迟、能耗和面积（固定金属面积预算下的带宽）方面提供了不同的权衡点。低摆幅差分连线 [179] 也被建模；这些连线提供更低的能耗，但带来延迟和面积损失。路由器模型基于 Orion [166]，主要量化缓冲区和交叉开关的能耗。路由器延迟模型是一个输入参数，因为路由器流水线级数是时钟速度和推测程度的函数。设计空间探索通过迭代缓存到存储体的所有合理分区来进行。假设每个存储体都与一个路由器相关联。对于每个这样的分区，估计每次访问的平均网络和缓存延迟。网络延迟包括路由器争用，这是根据若干基准经验估计并包含在 CACTI 模型中。设计空间探索还针对缓存的每个分区遍历所有互连选项和各种 VC/缓冲区路由器组织。最终，CACTI 提供优化某些用户定义指标的缓存组织。

图 4.4 最能说明设计空间探索所考虑的权衡。对于固定大小的缓存，随着 bank 数量的增加，bank 大小减小，bank 内的延迟也随之减小。同时，网络规模增大，虽然整体缓存面积和线长没有剧烈变化，但路由器延迟开始发挥重要作用。此外，随着网络中更多链路的加入，争用减少。最终结果是一条整体延迟曲线，当大型 32 MB 缓存被划分为八个 4 MB bank 时，该曲线达到最小。这使得网络规模远小于大多数先前 NUCA 缓存研究中所假设的规模。整体缓存能耗也存在类似的权衡曲线。

为了深入了解各组件对延迟和能耗的贡献，我们在图 4.5 和图 4.6 中展示了 NUCA 命中的饼图分解。这些结果由 CACTI 6.5 针对 32 nm 技术的 128 MB L2 缓存、16 核处理器生成。该配置的最优 bank 数量为 32 个 bank，组织为 8×4 网格。图 4.5 显示了影

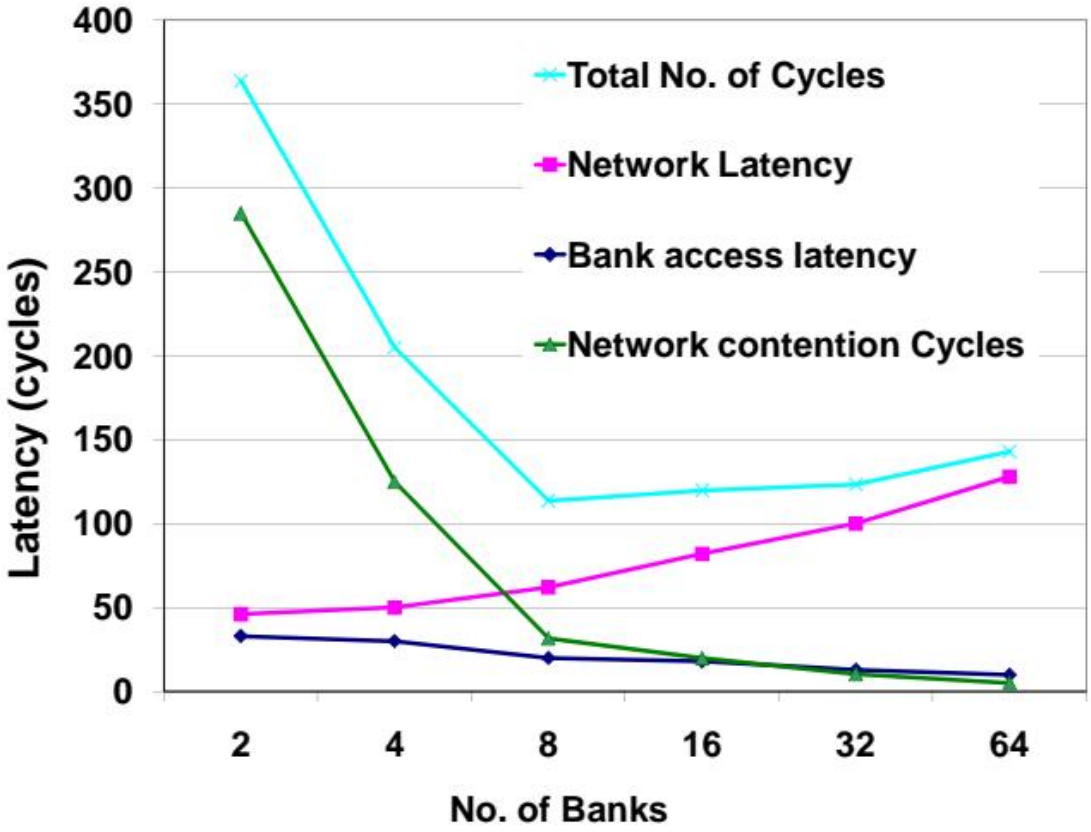


图 4.4　随着缓存存储体数量增多而变化的访问延迟（据文献 [4]）。

响 NUCA 命中延迟的关键组件。从图中可以得出一些有趣的观察。首先，在最优配置中，路由器延迟几乎与 bank 访问延迟相同。其次，网络中的争用对访问时间有显著贡献。有趣的是，当对 bank 的访问流水线化时（图中未显示），争用急剧下降（降幅超过 75%）。这表明，在 NUCA bank 内采用流水线 bank 或多个子 bank 是值得探索的有趣设计点。最后，即使存在如此高的争用，最优 bank 数量也仅为 32。图 4.6 显示了能耗分解。由于每个 bank 为 4 MB，bank 访问能耗在总能耗中占主导地位。请注意，并非所有访问都有类似的分解；如果访问未命中或请求的是目录信息，则只有标签阵列被激活，其访问能耗要小得多。在这些情况下，路由器组件的能耗将占主导。如果工作负载主要是单线程的，则降低 bank 能耗更为有利。另一方面，对于具有许多一致性事务的多线程工作负载，路由器和链路是能耗优化的良好目标。这表明，根据所执行工作负载的类型，可以进行智能的电压或频率调节。

4.2.2　UCA 缓存

基于上述讨论，很明显大型 NUCA 缓存将被分解为一到两个少量数 MB 的 bank，而不是数十个小 bank。这是摊销笨重路由器开销所必需的。瓦片架构可能会很常见，其中每个瓦片将容纳一个大型 L2 缓存 bank（要么是私有缓存，要么是大型共享缓存的一个切片）。因此，无论未来处理器采用私有/共享缓存、UCA/NUCA 架构，还是瓦片式/连续式 L2 缓存，很明显芯片上都会存在几个大型缓存 bank。因此，这种大型缓存

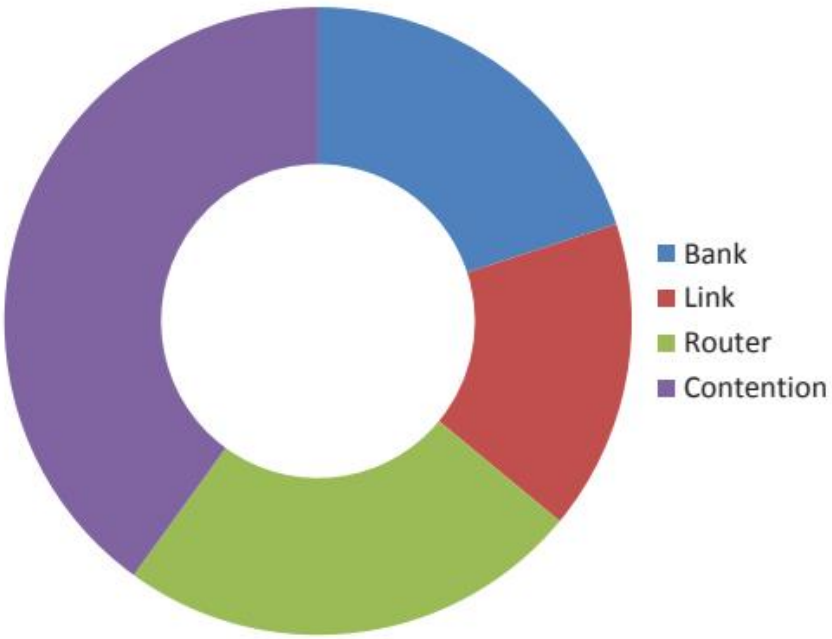


图 4.5 NUCA 缓存访问的延迟组成。

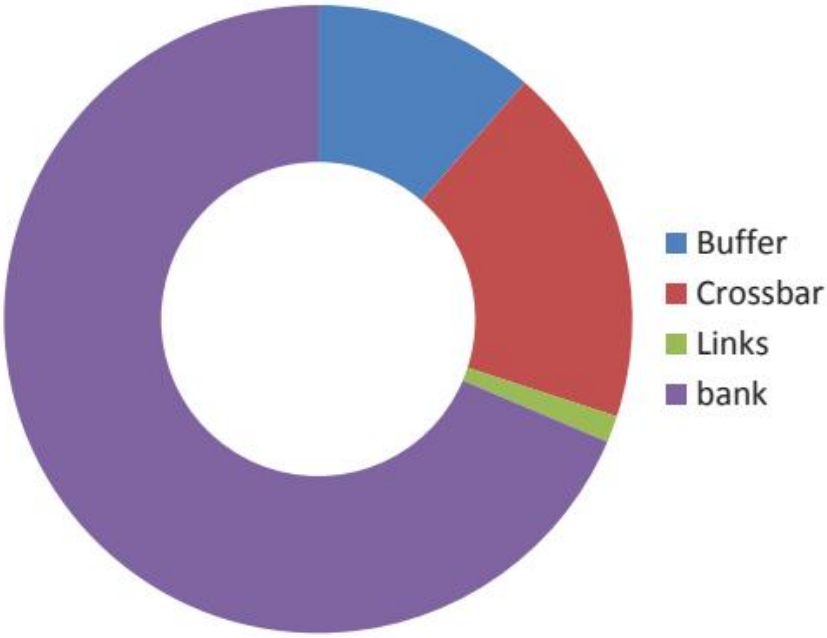


图 4.6 NUCA 缓存命中时的能量消耗组成。

bank 的优化很重要。我们预计这种 bank 的容量为几 MB，并提供统一的缓存访问。

图 4.7 展示了 CACTI 在 32 nm 技术下建模的各种大小 UCA 缓存 bank 中延迟和能耗耗散的饼图分解。H 树网络对缓存 bank 功耗的贡献在大容量缓存 bank 中明显是瓶颈。一个

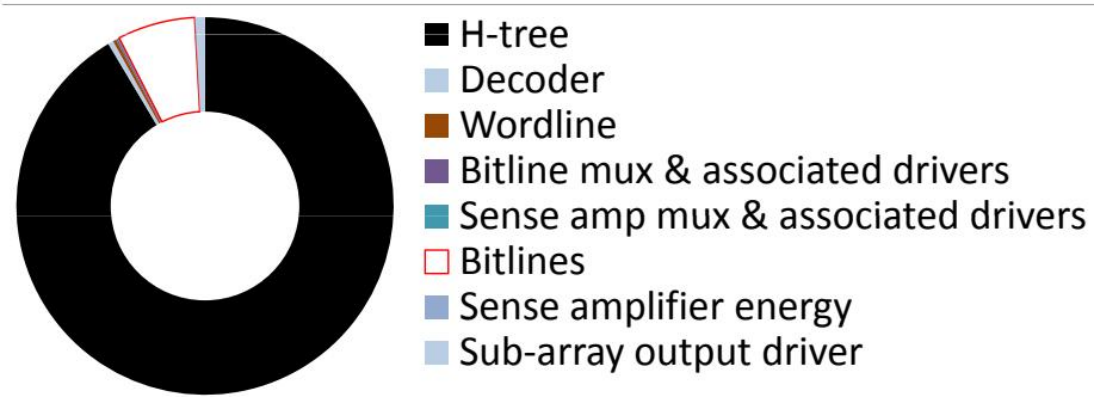


图 4.7　UCA 缓存访问时的能耗组成。

H 树网络用于将缓存 bank 的许多子阵列连接到其输入/输出端口。平衡的 H 树提供统一的流水线访问，无需复杂的开关电路或调度逻辑。在小容量缓存 bank 中，位线能耗往往占主导地位。随着缓存 bank 规模增大，H 树网络的贡献开始占主导地位。这主要是因为位线采用低摆幅差分信号，而 H 树网络使用基于全局 RC 的全摆幅布线。

上述观察表明，低摆幅差分信号 [174, 179] 也可用于连接缓存子阵列。降低全局导线上的电压摆幅可以线性降低能耗。使用单独的较小电压源可以实现二次方能耗降低。本质上，根据导线长度，低摆幅布线可以将能耗降低近一个数量级。低摆幅导线不需要中继器，但需要在其端点处配备接收器和发送器电路。因此，它们不易流水线化。当传输长距离时，它们还会带来更长的延迟惩罚。

Udipi 等人最近的一篇论文 [180] 考虑了在缓存 bank 内使用低摆幅布线来连接许多子阵列。虽然可以实现显著的能耗降低，但也可能带来明显的性能下降。因此，关键是要明智地使用适量的低摆幅布线，以平衡性能损失和节能。该论文提出了四个合理选项：（i）将 H 树实现为一组非流水线低摆幅导线，（ii）将 H 树实现为一组流水线低摆幅导线，（iii）实现多组并行低摆幅导线，以及（iv）用低摆幅主干增强基线全局导线 H 树，以访问两行子阵列。这些不同的设计落在能量-性能权衡曲线上的不同点。Udipi 等人认为，最后一种设计代表了一个最佳平衡点，特别是如果使用架构机制来增加从具有低摆幅链路的子阵列获取数据的可能性。然而，这些机制涉及不小的复杂性，因为流行或最近访问过的块必须迁移到具有低摆幅链路的子阵列。与 S-NUCA 与 D-NUCA 的讨论类似，可能存在替代的、复杂度较低的方法来利用缓存子阵列中的非均匀功耗访问。

阵列，但这仍是未来工作。该研究确实强调了需要认真考虑在缓存体（bank）内明智地使用一些低摆幅布线，以解决互连瓶颈。

4.3 面向大型缓存的创新网络架构

大多数架构研究在连接众多核心和缓存体时，通常假设采用基本的网格拓扑。一些研究还采用集中、快速通道或扁平蝶形拓扑。在本节中，我们将重点关注与缓存层次结构交互密切相关的网络创新。

传输线缓存，Beckmann 和 Wood，MICRO'03

传输线是一种与传统基于 RC 的导线不同的布线技术。在传输线中，延迟由电压波前在导线上传播和检测所需的时间决定。该延迟由 LC 时间常数和波前速度决定，而波前速度是互连周围介质中光速的函数。因此，传输线能够实现非常低的延迟。然而，也存在若干相关开销。导线必须具有非常大的宽度、厚度、水平和垂直间距、高信号频率、金属层上方和下方的参考平面，以及紧邻每条传输线的屏蔽电源线和地线。此外，两端都需要发射器和接收器电路（但不需要中继器）。由于这些高成本，传输线在各种芯片中仅被稀疏使用 [181，182，183，184，185]。在过去五年中，它们的普及度并未增长，部分原因是可以在 RC 领域通过粗导线提供低延迟，部分原因是人们更关注低能耗互连而非低延迟互连。尽管如此，它们仍然是一个值得关注的有趣设计点。由于面积/成本开销，它们可能仅在可接受低带宽的场景中被利用。

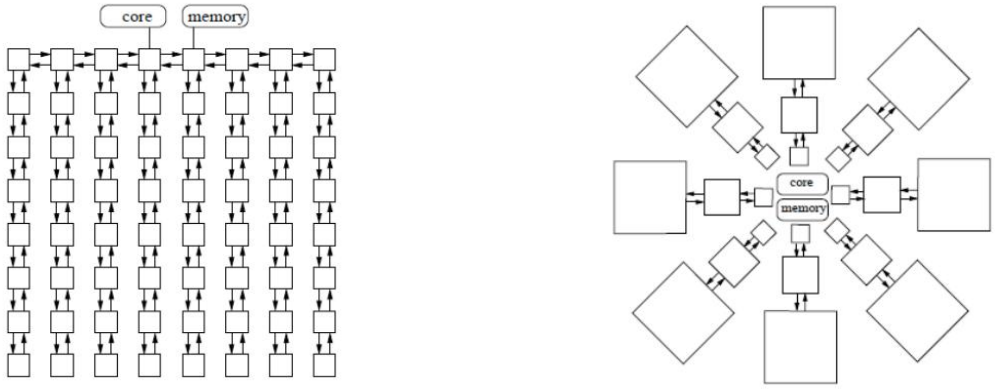
Beckmann 和 Wood 构建了一种大型缓存架构，假设传输线是首选的互连方式 [186]。L2 缓存被划分为大型缓存体，并放置在芯片外围。这些缓存体通过传输线链路连接到中央 L2 缓存控制器。由于传输线的高面积开销，这些链路必须很窄。采用了几项优化来处理这一带宽问题。只有部分地址被发送到缓存体，并在缓存体处进行 6 位部分标签比较。在加载时，标签的其余部分必须返回缓存控制器以进行完整标签比较。由于整个地址必须在缓存体和控制器之间交换，因此在加载情况下没有节省带宽。在 L1 写回（存储）的情况下，如果部分标签比较成功，该值将直接写入 L2——L1-L2 层次结构的包含性保证了部分标签命中对应于完整标签命中。因此，部分标签策略降低了对 L1 写回的带宽需求。第二项优化将单个缓存行分布到多个缓存体上。虽然这增加了总体带宽需求（地址必须发送到多个缓存体），但更窄的链路

可用于每个缓存体，因为缓存体仅提供部分缓存行。这与现代 DDRx DRAM 数据映射类似，其中通过将缓存行分布到多个 DRAM 芯片来缓解有限的 DRAM 芯片引脚带宽。由于传输线明显的低延迟优势，所得的 L2 缓存实现了高性能。

Halo 网络，Jin 等人，HPCA'07

Jin 等人 [5] 表明，专门针对 D-NUCA 流量的网络可以比通用的基于网格的网络简单得多。假设单个核心访问大型 D-NUCA 缓存。缓存被组织为二维缓存体阵列，给定块映射到唯一列，但可以驻留在该列的任何缓存体中，即集合的路（way）分布在该列的所有缓存体中。集合中路的放置基于 LRU 算法。因此，MRU 块被放置在离核心最近的路和缓存体中，而 LRU 块被放置在离核心最远的路和缓存体中。假设核心放置在缓存上方（如图 4.8（a）所示），缓存请求首先在 X 维度发送，直到到达正确的列。然后请求向下传播，直到块位于其中一个路（缓存体）中。当请求向下传播时，它伴随

着驻留在前一个缓存体（路）中的块——这允许在搜索缓存时将块按 LRU 顺序放置在缓存体中。当块位于某个缓存体或层次结构的下一级时，它被放置在最近的路中。当块被定位时，它沿列向上传播（在 Y 维度），直到到达最顶部的缓存体，然后在 X 维度传播，直到到达核心。



（a）简化网络的 NUCA 缓存　　（b）Halo 网络

图 4.8　面向大型 NUCA 缓存的 Halo 网络结构（据文献 [5]）。

鉴于这种请求和响应的路由方式，Jin 等人观察到该网络是无死锁的。不需要完整的网格网络，因为水平通信只发生在

顶行——所有其他行不需要水平链路。这大大简化了每个路由器的设计。通过重新组织缓存体使其围绕核心（图 4.8（b）），该组织得到进一步优化。请求和响应向外或向中心辐射。缓存体大小现在可以异构，最小的缓存体最靠近核心。虽然图中显示了对角线链路，但也可以采用锯齿形 XY 链路进行布局。

虽然该设计不易扩展到多核布局，并且专门针对 D-NUCA 流量进行了调整，但对于可能表现出非常特定流量模式的缓存组织来说，这是一个发人深省的想法。

Nahalal，Guz 等人，SPAA’08

Guz 等人 [6] 观察到，D-NUCA 缓存中对共享数据的处理非常不理想。共享块倾向于向一个重心位置聚集，而该位置并不靠近任何核心 [2，22]。他们尝试用一种新颖的布局/拓扑来解决这个问题，其灵感来源于设计合作村庄 Nahalal 时所使用的城市规划理念。虽然这篇论文并非专门讨论片上网络的设计，但它表明，巧妙的核心与缓存体布局和放置可以在缩短片上距离和减少网络流量方面发挥重要作用。

在所提出的组织方式中（如图 4.9 所示），一些缓存体被放置在芯片中心，它们被一圈核心所环绕，另一组缓存体则构成外环。这些缓存体共同组成一个共享的 L2 缓存，路数分布在各缓存体之间，块可以通过 D-NUCA 策略在路（缓存体）之间迁移。中心缓存体用于存储共享块；外围缓存体用于存储最靠近的核心私有的块。块最初被放置在请求核心的外围（私有）缓存体中；如果该块被其他核心访问（可通过计数器或检查目录协议的共享向量来检测），则将其迁移到中心的共享缓存体。因此，每个核心很可能在共享的中心缓存体或相邻的私有缓存体中找到数据——这也是搜索数据时查

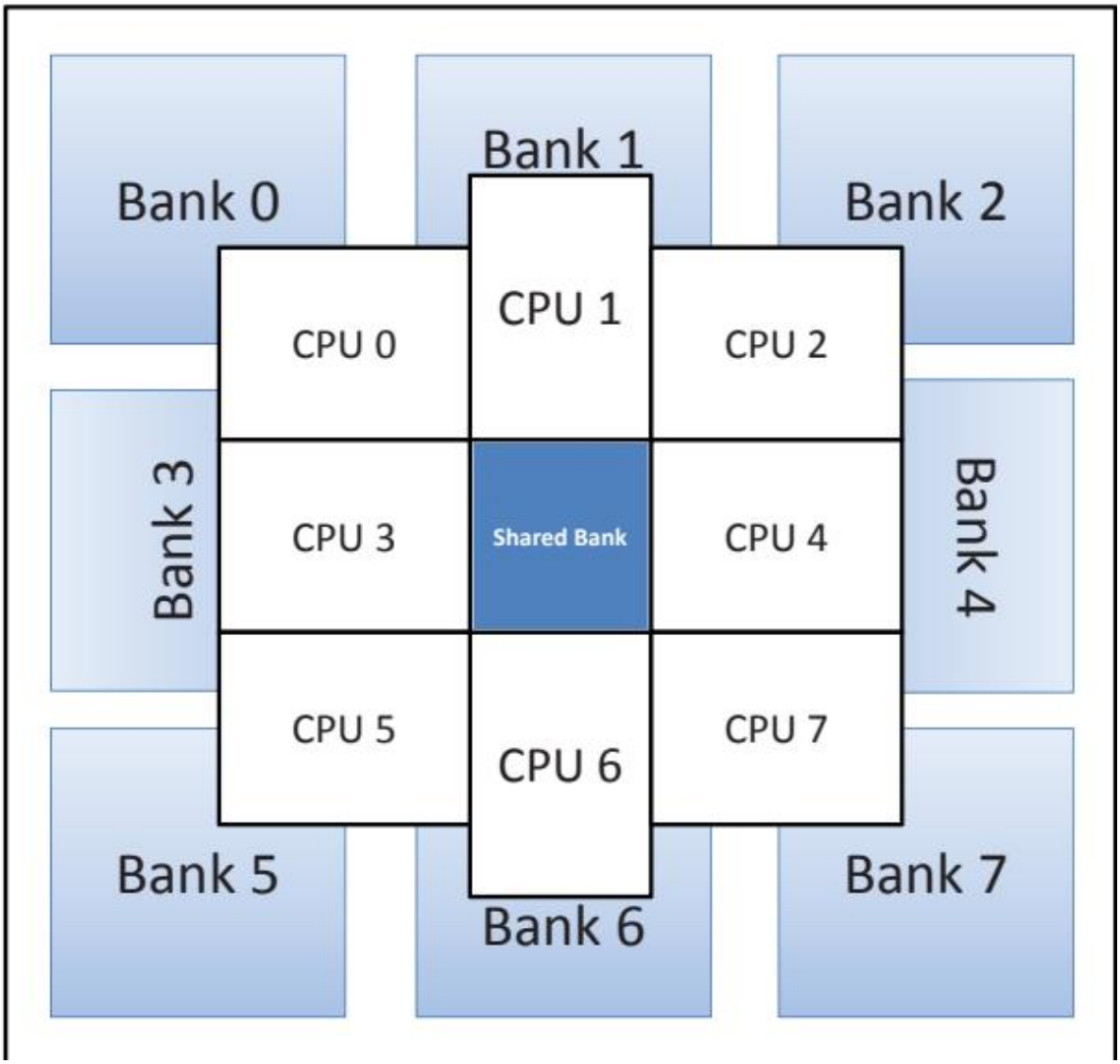


图 4.9 Nahalal 架构 [6]。

找缓存体的顺序。这里没有采用在共享和私有缓存体中并行查找的方式，而是使用一个预测器来确定两个区域中哪一个优先查找。鉴于核心与私有/共享缓存体的布局，绝大多数缓存查找只需经过相对较短的距离即可完成。虽然 Guz 等人关注的是 D-NUCA 架构，但这些概念也应该可以应用于其他 S-NUCA 或分块架构。

混合网络，Muralimanohar 和 Balasubramonian，ISCA'07

如前所述，导线可以通过多种方式实现。低延迟导线可以通过采用大宽度和大间距（粗导线）或传输线技术来设计，但这两种方法都会牺牲带宽，因为在固定的金属面积预算内能容纳的导线数量更少。Muralimanohar 和 Balasubramonian [7] 观察到，在大容量缓存网络中，请求和响应给网络带来的需求不同。响应需要传输缓存行，需要高带宽。而请求只包含地址，甚至可能只是部分地址。因此，他们建议对请求和响应使用不同的网络。响应采用传统的基于网格的拓扑结构，使用最小宽度导线以实现高带宽。请求则采用低延迟的粗导线。由于粗导线可以在一个周期内覆盖相对较长的距

离，网格拓扑可能过于复杂，因为它经常在数据传输中引入路由器流水线延迟。因此，请求网络提倡采用基于总线的广播网络。为了提高可扩展性，每一行缓存体都实现一条总线，这些总线通过一个常规的路由网络连接起来（图 4.10）。这项工作表明，缓存网络的导线感知设计和异构性可以带来性能和功耗的改善。

Manevich 等人 [187] 在其总线增强型 NoC（BENoC）架构中也提出了类似的论点，该架构在多级 NoC 中增加了一条简化的总线，该总线在特定操作（例如在 D-NUCA 架构中搜索块）上更为高效。

面向高效目录一致性的网络创新

Cheng 等人 [188] 观察到，一致性协议中的不同消息对网络的需求不同。数据消息需要高带宽，而大多数其他控制消息很短，需要低延迟。在需要多条消息（每条消息的跳数可能不同）的一致性事务中，有些消息不在关键路径上——这些非

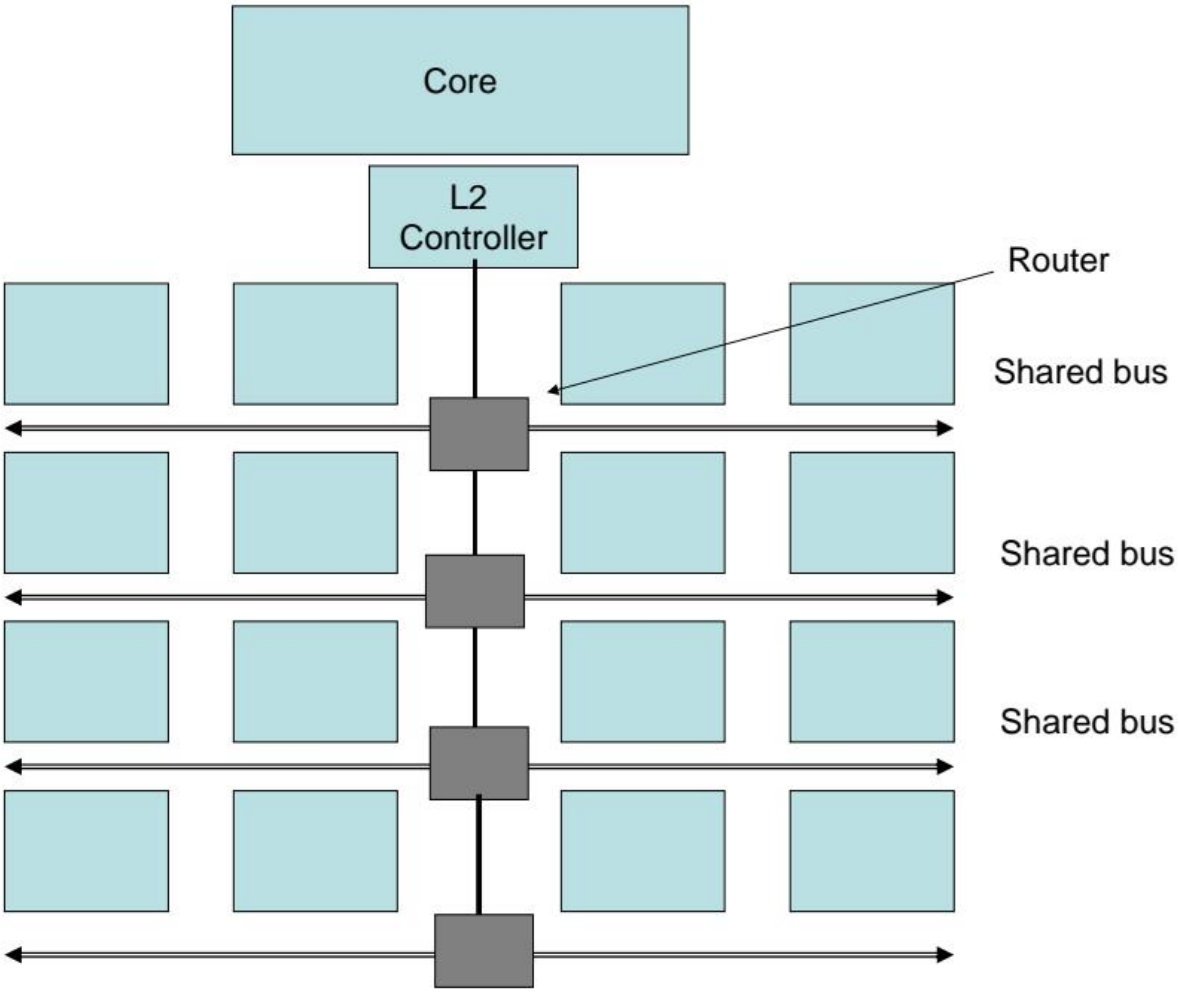


图 4.10　面向 NUCA 架构的混合网络 [7]。

关键消息可以在针对能耗而非延迟优化的链路上传输。Cheng 等人设计了一种网络，其中每条链路由不同的导线类型组成。每条消息被映射到在延迟、带宽和能耗方面最符合其需求的导线上。

Bolotin 等人 [189] 也观察到，一致性协议中的短控制消息应比长数据消息具有更

高的优先级。他们还设计了一种广播失效机制，该机制依赖路由器创建失效消息的副本，并快速将其传播到所有节点。这种技术也可用于加速 D-NUCA 缓存中的块搜索。或者，给定一组共享者，可以构建一个虚拟环，并将失效消息依次发送给所有共享者，直到向目录返回单个最终确认消息。

Eisley 等人 [190] 提出了一种在路由器内增加智能以加速一致性事务的机制。在典型的基于目录的缓存一致性协议中，必须先联系目录才能处理缓存缺失。而目录又可能需要联系其他节点，然后才能最终授予请求者访问权限。即使初始请求正在发往目录的途中，它也可能经过其他拥有足够信息来处理该请求的节点，但传统协议并未以任何方式利用这些信息。Eisley 等人通过构建虚拟树来建立分布式目录；一个块的所有共享者及其目录归属节点被组织成一棵树。树的边存储在每个相关路由器的表中。如果请求命中某个路由器

如果该请求恰好属于该块的虚拟树的一部分，则无需联系目录即可发起必要的连贯性操作。如果请求是读取，附近的共享者会提供数据，请求者被添加到虚拟树中。如果请求是写入，则沿虚拟树发送无效化消息，并解构该树。因此，在网络路由器中集成虚拟树允许中间路由器拦截并响应连贯性请求。如果表已满且必须逐出一个树条目，则树的一部分也必须被解构。除了每个路由器中引入的表之外，网络内连贯性机制还需要在属于该块虚拟树但没有该块缓存副本的节点上进行额外的缓存查找。Eisley 等人 [191] 也应用了相同的网络内智能理念，以指导将逐出的块放置在附近有空间的 bank 中（如协作缓存 [48]）。

高效基于侦听的连贯性的网络创新

通常认为基于侦听的连贯性无法扩展到大量核心。然而，基于侦听的协议具有一些理想特性：（i）无目录存储开销，（ii）响应请求时无需通过目录进行间接寻址，（iii）协议更简单且更易于验证。为了有意义地利用这些积极特性，必须克服基于侦听协议的一些消极特性，即集中式排序点和需要将每条消息发送到每个节点。

Agarwal 等人 [192] 在 HPCA'09 论文中尝试缓解第一个开销。他们假设采用分组交换网格（或任何其他可扩展拓扑）的无序网络，并在其上实现基于广播的侦听协议。每个注入的消息被分配唯一 ID，路由器负责确保节点按正确的数字顺序接收消息。在 MICRO'09 论文中，同一作者解决了第二个问题 [193]。他们在网络内实现过滤器，跟踪地址（或地址范围）在网络各个部分的存在。因此，在不需要看到该地址的网络部分中，该地址的广播被抑制。

Udipi 等人 [8] 采用了类似方法，但侧重于基于总线的网络。两个额外的观察推动了他们的设计：（i）总线优于路由网络，因为它们消除了昂贵的路由器；（ii）鉴于未来处理器中充足的金属面积预算，可以通过实现多条总线来实现可扩展性。片上网络由两级总线层次结构组成（图 4.11）。核心首先在其本地总线上广播请求。与本地总线关联的布隆过滤器确定该地址是否已在本地总线之外被访问，以及广播是否需要传播。对于具有局部性的应用，许多广播不会超出本地总线。如果广播必须传播，首先进行中央总线的仲裁，然后广播在中央总线上发生，并紧接着在其他本地总线上发生。

其他本地总线上的附加过滤器可能会抑制其他本地总线上的广播。初始本地总线广播直到中央总线广播（如果需要）完成才被视为完成——中央总线广播充当保持连贯性和顺序一致性的序列化点。每个本地总线需要单条目缓冲区，以容纳等待中央总线仲裁的请求。可以采用多个并行总线层次结构，每个层次结构处理不同的地址范围。相对于基线分组交换网格网络，所提出设计的能效是基线中链路与路由器能耗之比的函数。当采用高能效低摆幅链路时，所提出的设计尤其具有吸引力。

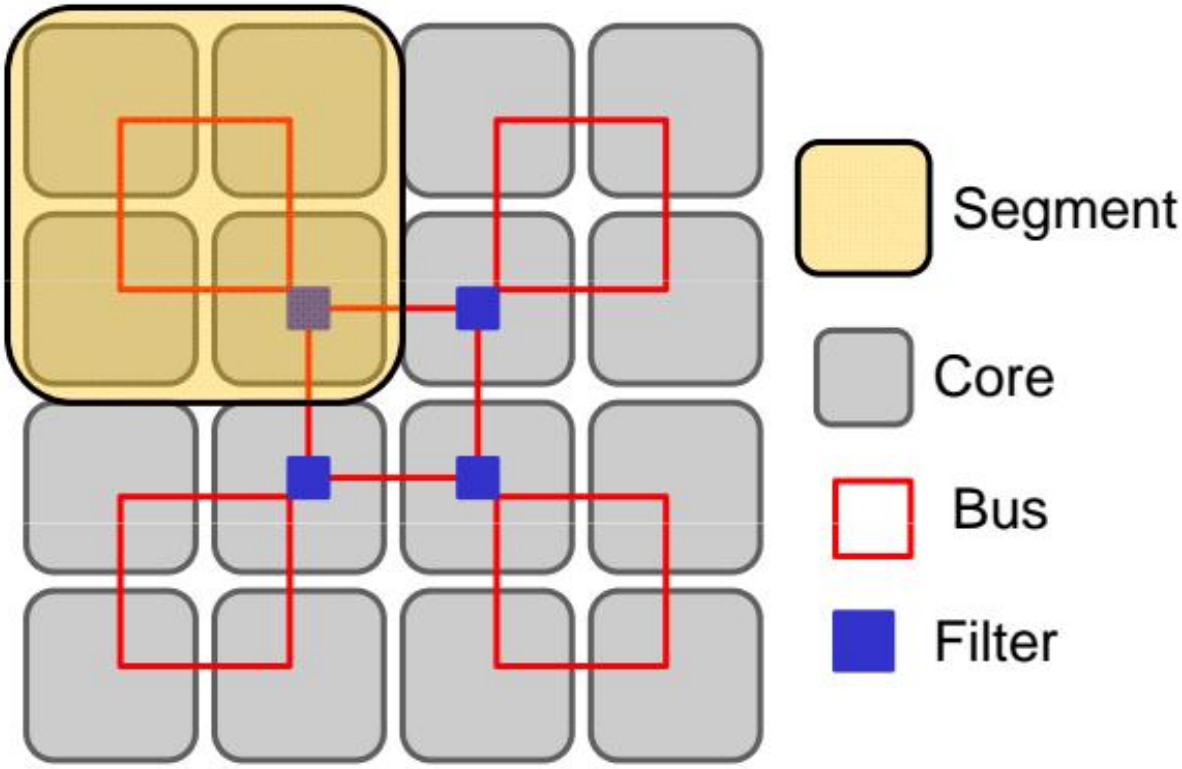


图 4.11　带有广播过滤器的层次总线设计 [8]。

Das 等人 [194] 的一些先前工作也主张在片上网络中使用层次结构以利用应用局部性（特别是当使用页面着色在 S-NUCA 缓存中进行智能数据放置时）。总线用于本地通信，分组交换网格拓扑用于连接众多总线。使用本地总线有助于消除附近通信的路由器导航。在层次结构的第二级使用目录协议和网格拓扑确保消息仅在存在远程共享者时才超出本地总线。

总结

显然，片上网络的许多方面都可以改进，例如布线、拓扑、路由器设计等。大多数讨论的论文大致可分类如下：

- 更好的布线：传输线 [186]、粗线 [7，188]、低摆幅线 [8]。

- 更好的拓扑：Halo [5]、Nahalal [6]、混合/层次拓扑 [7，8，187，194]。

- 更多/更少的路由器功能：针对连贯性操作进行优化 [189，190，191，192，193，194]。

注意，这些创新中的每一项都与缓存层次结构或连贯性协议的设计有特定联系。当

然，还有另一大批工作试图提高通用片上网络的效率 [11]。

第五章　存储技术

第 2 章和第 3 章主要关注数据选择和放置的逻辑策略。第 4 章考察了缓存的物理设计，特别强调网络；这是技术趋势如何显著影响缓存层次结构的一个例子。技术在不断进步，为缓存设计带来了新的机遇和挑战。因此，了解一些可能影响缓存设计的新兴技术现象非常重要。了解新兴的存储技术也很重要，因为片外存储层次结构可能会变得更深，众所周知的片上缓存原则将不得不适应存储层次结构的其他级别。片上缓存策略也必须改进，以解决片外存储器带来的问题。

本章首先讨论基本的 SRAM 单元。基本的缓存 SRAM 单元几十年来几乎没有变化。然而，工艺技术的进步正在为缓存设计者带来重大挑战和机遇。我们概述这些技术进步以及随之出现的相应缓存创新。最后，我们讨论新兴的存储技术。

5.1　静态 RAM 的局限性

二十多年来，静态 RAM（SRAM）技术一直是片上存储的理想选择。这不仅是因为其卓越的延迟和可扩展性，还因为逻辑组件中使用的相同 CMOS 晶体管也可用于 SRAM 缓存。SRAM 单元是一个简单的双稳态锁存器，其工作电压与处理器中其他组件相同。因此，它不需要单独的电压层或电荷泵，而其他存储类型通常需要这些。由于锁存器中的交叉耦合反相器可以自动纠正存储值中的任何电荷泄漏，该器件无需像动态 RAM（DRAM）那样定期刷新即可保持其值。在设计上，它输出差分信号，这更容易检测且更可靠。这些特性使 SRAM 多年来成为片上存储的最佳选择。

然而，随着我们进入多核时代，SRAM 面临两个关键挑战。首先，特征尺寸的缩小使得精确控制晶体管尺寸变得越来越困难，这增加了晶体管的漏电功耗和延迟变化。此外，SRAM 还容易受到辐射引起的软错误的影响，这可能导致随机位翻转。来自太空的高能粒子（如中子和 α 粒子）可能撞击存储单元，扰乱其电荷平衡，并可能翻转其中存储的位。对于大晶体管，位翻转的概率很低，简单的纠错码（ECC）足以识别和恢复少数故障位。

第 5 章

然而，随着晶体管变小，软错误预计将呈指数增长 [195]。因此，需要更复杂和昂贵的纠错方案来保证未来 SRAM 缓存的正确运行。其次，未来缓存受到 SRAM 高面积需求的限制。一个 SRAM 单元由六个晶体管组成，占用的面积是仅含一个简单电容器和一个访问晶体管的 DRAM 单元的 20 倍。随着处理器相对于主存的速度越来越快，有必要通过拥有更大的缓存来降低缓存未命中率。因此，SRAM 单元的高面积成本在

多核处理器中是一个严重的问题，因为在给定的芯片尺寸下，大缓存减少了核心的可用面积。这种严格的面积要求也使得难以采用替代的 SRAM 实现，这些实现以面积为代价来提高可靠性，尤其是对于大缓存。例如，八晶体管 SRAM 设计在单元内分离读端口和写端口以提高静态噪声容限 [196]。具有十个晶体管的 SRAM 单元通过使用额外的晶体管提供强大的负反馈来防止位翻转，进一步提高了噪声容限 [197]。然而，它们的应用将主要限于小型暂存 RAM 或 L1 缓存。

这些问题的长期解决方案可以尝试修改缓存的微架构以使其对故障更具鲁棒性，或者利用新兴的制造技术（如 3D 堆叠）来提高缓存的密度和弹性。目前也在努力寻找适合未来拥有数百个核心的处理器的新的存储技术。本章描述了针对这些 SRAM 挑战所考虑的技术。

5.2 参数变化

当制造纳米尺寸的晶体管时，在芯片上精确再现更精细光刻图案的能力会降低。光刻是制造过程中的关键步骤，其中电路图案被转移到硅片上。虽然摩尔定律继续缩小晶体管尺寸，但光刻过程中使用的光的波长难以缩小到 193 nm 以下。使用 193 nm 波长的光来产生亚波长图案（< 90 nm）会导致晶体管/导线尺寸和阈值电压（V_t）的缺陷。更具体地说，L_{eff}，即晶体管的有效栅极长度，具有最小的尺寸，受到的影响最大。工艺变化的另一个原因是无法精确控制晶体管中活动区域的掺杂浓度。这再次改变了晶体管的阈值电压（V_t）。

这些变化严重影响晶体管/导线延迟和漏电功耗。公式 5.1 显示了栅极长度、V_t 和延迟之间的关系。由于晶体管延迟是 V_t 和 L_{eff} 的函数，并且随 V_t 非线性变化，因此这种影响在晶体管中比在导线中更为明显。漏电功耗随 V_t 呈指数变化，如公式 5.2 所示。Borkar 等人 [198] 表明，工艺变化可使处理器的工作频率降低 30% 以上，并使漏电功耗增加 20 倍。

$$tr_{delay} \propto \frac{V_{dd} L_{eff}}{\mu (V_{dd} - V_t)^{\alpha}} \tag{5.1}$$

$$P_{leakage} \propto V_{dd} T^2 e_t^{-qV} / kT \tag{5.2}$$

控制晶体管漏电和延迟的一种简单方法是修改衬底电压以抵消 V_{th} 的变化，这称为体偏置。然而，体偏置会带来很大的面积开销。由于变化高度不可预测，很难在细粒度上提供足够的偏置电路来准确修复所有受影响的单元和电路。因此，需要架构解决方案来减轻工艺变化，同时不显著影响成本或性能。针对此问题的现有缓存微架构解决方案分为两类：隔离劣质缓存行以避免或限制其使用，或容忍受影响单元中的额外延迟。

工艺变化的影响通常在缓存中比在处理器中的其他逻辑组件中更为明显。由于大

量芯片面积专用于缓存，它们对总功耗的漏电贡献已经很高。由于变化，这种情况会显著恶化。此外，缓存有许多逻辑深度较浅的关键路径，使其更容易受到延迟变化的影响 [199]。由于一个字中的所有位是并行访问的，访问单个位的变化会减慢整个读或写操作。

在描述容忍工艺偏差的技术之前，我们将简要审视通常用于对缓存中的工艺偏差进行建模的方法。

5.2.1　建模方法

过去，偏差仅被视为晶圆上不同芯片之间的问题。晶圆上各芯片的工作频率和功耗特性存在差异，有些芯片的延迟或能耗特性优于其他芯片。然而，在 90 纳米以下，芯片内部的偏差已成为一个主要问题。因此，即使在单个处理器内部，某些组件也会显著更慢或消耗更多功耗。

在研究缓解慢速组件影响的体系结构技术之前，有必要估计组件之间功耗/延迟的偏差，并计算处理器受影响的比例。工艺偏差既可以是随机的，也可以是相关的。为了评估它们，应考虑以下五个参数：晶体管栅长、阈值电压、金属宽度、金属厚度以及层间介电层厚度。对于每一代工艺，ITRS [200] 都会给出每个参数的目标值。然后根据偏差分布、偏差百分比以及距离或范围因子对这些值进行调整。根据偏差分布的不同，偏差百分比可用于建模随机偏差，而范围因子有助于建模邻近组件之间偏差的相关性。

5.2.2　缓解工艺偏差的影响

如前所述，工艺偏差的两个主要副作用是器件和互连延迟的增加以及漏电功耗的增加。由于缓存组的访问延迟通常由最慢的组件决定，少数慢速晶体管和导线会对访问时间产生不可忽视的影响。

5.2 参数偏差

尽管所有 SRAM 缓存都受到偏差的影响，但它们对处理器功耗和性能的影响取决于缓存在层次结构中的位置。诸如 L1 数据缓存和 L1 指令缓存之类的小型高层缓存对性能的影响大于对功耗的影响，而大型低层缓存则主要导致高漏电功耗。接下来我们将讨论为容忍高层和低层缓存中的偏差而提出的体系结构技术。

良率感知缓存：Ozdemir 等人，MICRO 2006

Ozdemir 等人是最早研究 L1 缓存中偏差的学者之一，他们探索了微体系结构解决方案以缓解其影响 [201]。通过详细的 SPICE 模拟，Ozdemir 等人表明，16 KB 缓存中较差的晶体管和导线可使缓存访问延迟恶化高达 3 倍，并使漏电功耗增加 8 倍。这些不良特性最终会影响整体制造良率，因为微处理器即使能够运行，如果无法满足延迟和功耗约束，也会被丢弃。

Ozdemir 等人提出了三种提高处理器良率的方案：良率感知功耗降低（YAPD）、水

平 YAPD（H-YAPD）和可变延迟缓存架构（VLCA）。在这三种方案中，缓存中受影响的组件首先按路或阵列粒度进行隔离。随后，根据这些组件受影响的程度，要么永久关闭它们以实现尽可能最佳的访问时间和漏电，要么以低频率运行。YAPD 和 H-YAPD 通过关闭路来牺牲缓存容量以保持快速访问延迟，而 VLCA 则通过修改乱序核心架构以容忍性能较差的路来保留容量。

YAPD 是一种简单的解决方案，其功能类似于 Albonesi 的选择性缓存路 [202]。如果缓存的整体功耗或其访问延迟超过预定阈值，则关闭漏电功耗最高或关键路径最慢的路。然而，该方案的主要问题在于优化执行的粒度。由于位线由多个组共享，关闭受影响的单元及其相关的位线、预充电电路和敏感放大器需要减少许多组中的有效路数；在小型缓存中，所有组都会受到影响。此外，一个组中的多条路通常彼此靠近放置，它们的偏差高度相关。因此，一个组中两条路性能不佳的可能性非常高；在这种情况下，该技术会显著减少有效缓存大小。

YAPD 的一种简单扩展称为 H-YAPD，它通过关闭子阵列中的水平行而不是整列来解决这些问题。由于一行单元对应一个组，在传统缓存中实现 H-YAPD 会关闭整个组。为避免这种情况，缓存块的组织方式使得阵列中的每一行存储来自多个组的路。这样，即使整行被关闭，对单个组的容量影响也会减小。

与上述两种方案不同，VLCA 保留性能较差的缓存路，并根据其偏差因子支持对不同路的非均匀访问。VLCA 的特性与第 1.3 节讨论的 NUCA 架构非常相似。然而，这项工作的新颖之处在于将可变延迟设计扩展到 L1 缓存。例如，典型的乱序核心甚至在操作数就绪之前就分派指令，使得操作数正好在执行阶段之前直接转发到执行单元。为了支持可变延迟的加载，其依赖指令在没有操作数的情况下分派后需要延迟，直到加载值就绪。VLCA 为所有功能单元添加了加载旁路缓冲区，允许指令在功能单元中等待，直到延迟的加载到达。虽然 VLCA 不损害缓存容量，但在乱序核心中支持可变延迟加载对复杂性和性能有不可忽视的影响。

Ozdemir 等人还尝试了 VLCA 和 H-YAPD 的组合以最大化良率。然而，他们的结果表明，简单的 YAPD 设计即可显著提高良率。YAPD 可将良率损失降低多达 68%，而结合 VLCA 和 H-YAPD 的最复杂混合设计可将这一比例提高到 81%。

路优先化，Meng 和 Joseph，ISLPED 2006

Meng 和 Joseph 提出了路优先化（way prioritization）来容忍大型下层缓存（如 L2 或 L3）中的工艺偏差。与 L1 不同，L2 或 L3 的访问延迟问题较小。然而，由于它们的尺寸较大，漏电是一个关键问题。与之前讨论的 YAPD 类似，路优先化也依赖于 Albonesi [202] 提出的选择性缓存路（selective cache ways）来关闭慢速和漏电的单元。但是，他们不是永久关闭劣质缓存路，而是根据工作负载需求动态改变缓存大小。例如，如果一个程序从大缓存中获得了显著收益，那么只要处理器总能耗的降低足以补偿漏电单元的开销，就会启用漏电的路。这是通过为每个 bank 维护一个优先级寄存器来跟踪每路的漏电功耗来实现的。在运行时，根据工作负载需求，开启/关闭路，以使处理器的总能量延迟乘积最优。

替代缓存，Das 等人，MICRO 2008

迄今为止讨论的技术都需要修改缓存微架构来容忍偏差，并增加了设计的复杂性。Das 等人 [203] 表明，对于小缓存，通过冗余更容易保证均匀的快速访问时间。通常的做法是为存储阵列配备冗余的行和列存储单元，以便在发生硬错误时，将受影响的行/列重新映射到冗余行/列。然而，当处理偏差时，这种技术不可行，因为受影响的单元数量远大于总冗余行或列的数量。由于几个失败的位就会触发整个行或列的重新映射，因此增加冗余来修复所有受影响的位是浪费的。

为了解决这个问题，Das 等人提出增加一个称为替代缓存（Substitute Cache，SC）的独立结构。与之前讨论的方案不同，缓存的主数据阵列保持不变。但是，每个缓存路都增加了一个全相联缓存，其中包含几个数据条目及其对应的索引。数据条目小于缓存行（64 位），索引对应于地址的子集以识别字。在制造过程之后，执行内置自测试（Built-In-Self-Test）来识别访问时间超过目标延迟的关键字。这些字的索引

5. 技术

存储在非易失性区域中，并在启动时填充到 SC 中。每次访问缓存时，也会搜索其 SC。如果 SC 命中，则选择来自 SC 的数据而不是主数据阵列。由于替代缓存远小于主数据阵列，其访问时间保证小于数据阵列。

虽然 SC 在应对延迟偏差方面有效，但它没有解决偏差的第二个主要影响，即漏电功耗。此外，SC 仍然依赖原始标签阵列来确定命中或未命中状态，这意味着标签阵列的偏差仍然可能破坏延迟阈值。例如，在访问像 L1 这样的小缓存时，标签阵列和数据阵列是并行访问的。根据标签访问的结果，数据阵列的输出要么被发送出去，要么被丢弃。因此，缓存的净访问时间由标签阵列或数据阵列决定（取决于哪个更大）。对于大多数缓存，由于数据阵列尺寸较大，其访问时间占主导地位。在某些情况下，特别是对于小组相联缓存，标签阵列的访问时间可能更大。在这种情况下，有必要进一步扩展 SC 以包含完整标签，以便高延迟标签也可以从 SC 访问。

5.3　容忍硬错误和软错误

除了偏差之外，由软错误（由于粒子撞击）和硬错误（由于 V_{th} 退化、电子迁移等）引起的故障在未来技术中将变得更加突出。由于任何 MOS 晶体管都容易受到这些错误的影响，因此无论存储技术或制造工艺的进步如何，它们都可能存在。例如，DRAM 及其变体也容易受到辐射引起的软错误的影响。此外，缓存的大部分面积将继续使用 CMOS 逻辑作为外围电路，这些电路可能因硬错误和软错误而失效。本节描述了旨在提高缓存鲁棒性的通用技术，无论错误来源或采用的存储技术如何。

存储器最流行的容错技术是使用纠错码（ECC）来检测和纠正错误。ECC 为数据位增加冗余，其复杂度可能因错误覆盖范围的不同而有很大差异。通常的做法是对每个 64 位字采用单错纠正、双错检测（SECDED）。这需要 8 位来存储纠错码——存储开销为 12.5%。然而，当我们试图检测/纠正更多错误时，要么存储开销要么恢复过程的

复杂性会显著增加。Bose 和 Ray-Chaudhuri [204] 提出的 BCH 码可以纠正多达六个错误并检测七个错误。虽然其存储开销与 SECDED 相似，但恢复过程极其复杂。采用更强的 ECC，如 8 位纠正和 9 位检测（OECNED），会产生超过 89% 的存储开销——对于面积受限的片上缓存来说，这是不可行的。

ECC 的一个普遍问题是，无论缓存的错误率随时间如何变化，在设计时都必须预先支付恒定的存储开销以容忍最坏情况下的故障率。然而，多位故障的概率极其罕见，针对最坏情况的设计既浪费又低效。Kim 等人 [205] 和 Yoon 和 Erez [206] 利用了大多数读或写无错误的事实，提出了将错误检测和纠正解耦的技术，以低成本容忍更多故障。

二维错误编码，Kim 等人，MICRO 2007

在传统的阵列组织中，ECC 码与缓存行存储在同一行中。对缓存行的读取也会读取相关的 ECC，类似地，当更新缓存行时，数据和 ECC 都会更新。Kim 等人 [205] 认为，随着覆盖范围的增加，这种单行（或一维）ECC 的成本会显著增加。相反，他们提出了一种使用简单奇偶校验的二维编码方案。关键在于在行和列中每几个比特都有奇偶校验。与 ECC 不同，奇偶校验只能检测错误。然而，当行奇偶校验检测到错误时，所有包含受影响单元的列都会用列奇偶校验进行检查，以识别出错的单元。出错的比特可以通过简单地反转来纠正。这种技术在处理多位错误时也很高效。虽然从故障中恢复的代价很高（因为要纠正一个出错的比特，必须读取整列，这需要读取每一行并选取适当的比特），但在无错操作期间，读取的延迟开销几乎为零。但是，由于缓存中的每个比特都与水平和垂直奇偶校验相关联，写入必须同时更新垂直和水平奇偶校验。在传统的一维保护中，ECC 和数据同时更新。然而，在二维方案中，由于一次写入只更新每列中的一个比特，更新列奇偶校验需要读取旧内容来计算新的列奇偶校验。因此，每次写入之前都必须先读取以找到新的奇偶校验。尽管存在这些复杂性，但这是朝着构建更健壮缓存迈出的有希望的一步。

Yoon 等人，ISCA 2009

Yoon 等人 [206] 提出将数据和奇偶校验信息解耦，以减少复杂编码的存储开销。他们的动机也是基于这样一个事实：与无故障操作的数量相比，错误极其罕见。因此，他们不是为高开销编码在片上缓存中支付大量的存储开销，而是提出将缓存行的 ECC 码直接映射到主存。他们探索了一种两级保护方案，其中每个缓存行都由奇偶校验或类似于传统缓存实现的其他低开销编码保护。此外，在 DRAM 中分配一个区域来存储 ECC。每当第一级保护抛出错误时，就从内存中显式读取 ECC 以纠正错误。这样，就不必在芯片上存储所有 ECC 信息。由于代码大小不受片上存储的限制，根据错误率的类型，第二级可以使用更复杂的编码。因此，该方法能更好地扩展以满足不断上升的错误率。然而，写入仍然存在问题，因为可能需要更新多个层级。

5.4　利用 3D 堆叠解决 SRAM 问题

近年来，3D 芯片堆叠领域取得了许多技术进步。已经提出了许多架构创新来利用这项技术 [207] 解决 SRAM 的高面积开销问题。越来越明显的是，3D 堆叠中的多个芯片很可能用于缓存或内存存储，可能采用不同的技术 [9, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217]。在本小节中，我们触及一些与缓存策略和网络设计最密切相关的 3D 文献样本。

Li 等人，ISCA 2006

Li 等人 [209] 是最早详细阐述多核芯片中网络和缓存设计的团队之一（他们的工作之前还有其他一些相关努力 [208, 210, 212, 218]）。Li 等人没有采用 3D 路由器阵列来实现网络，而是提出在每个芯片上使用 2D 网格。这些平面网络然后通过实现为总线的芯片间连接进行链接。由于芯片间距离非常短，且一个堆叠预计由少于 10 个芯片组成，总线广播预计会很快。在芯片间传输期间要求经过路由器显然是过度的。这些总线（或 3D 柱）通过芯片间通孔实现，预计在未来技术中能支持相当高的带宽。虽然平面网格中的大多数路由器是 5x5 路由器，但一些路由器还必须支持往返柱的遍历，并实现为 6x6 路由器。核心分散在所有芯片上，使得没有两个核心堆叠在同一垂直平面上。这有助于减少热紧急情况。如果所有核心都放置在最靠近散热器的芯片上，热紧急情况可以进一步减少。缓存体也分散在所有芯片上，并假设采用 D-NUCA 架构。缓存块不在芯片之间迁移，因为芯片间延迟不是问题。Li 等人得出的结论是，3D 拓扑的性能优势优于 2D NUCA 架构中块迁移的优势。换句话说，从 2D S-NUCA 到 2D D-NUCA 的性能提升小于从 2D S-NUCA 到 3D S-NUCA 的性能提升。性能还显示出对网络中垂直柱数量的敏感性。

Madan 等人，HPCA 2009

在最近的工作中，Madan 等人 [9] 考虑了一种异构芯片堆叠。底部芯片（最靠近散热器）实现所有核心和 L1 缓存。第二芯片实现大型 SRAM 缓存体。第三芯片实现大型 DRAM 缓存体。每个缓存体的面积与底部芯片上的单个瓦片（核心加 L1 缓存）相似。片上网络实现在第二（SRAM）芯片上；底部和顶部芯片上的瓦片/体之间没有水平链路。假设每个瓦片有一个芯片间通孔柱，将核心直接连接到正上方的 SRAM 和 DRAM 缓存体。SRAM 缓存实现为 S-NUCA L2 缓存。当瓦片经历 L1 缓存未命中时，请求通过芯片间通孔发送到正上方的 SRAM 体。从这里，请求通过片上网络发送到唯一的体

缓存会处理该地址。Madan 等人实现了一种神谕式页着色方案，类似于第 2.1.3 节讨论的 S-NUCA 页着色方案，该方案将所有私有页放置在请求瓦片正上方的 SRAM 存储体中。共享页则放置在中央 SRAM 存储体中。这种策略确保通常只需一次垂直跳变即可服务 L2 请求。水平片上网络上的流量仅限于对共享数据的请求，并且这种流量倾向于从芯片中心辐射进出。如图 5.1 所示，树形拓扑非常适合这种

流量模式，并有助于降低网络中路由器的复杂性。最后，Madan 等人观察到页着色策略会导致缓存存储体压力不均衡，并提出了一种重构策略。如果某个 SRAM 缓存

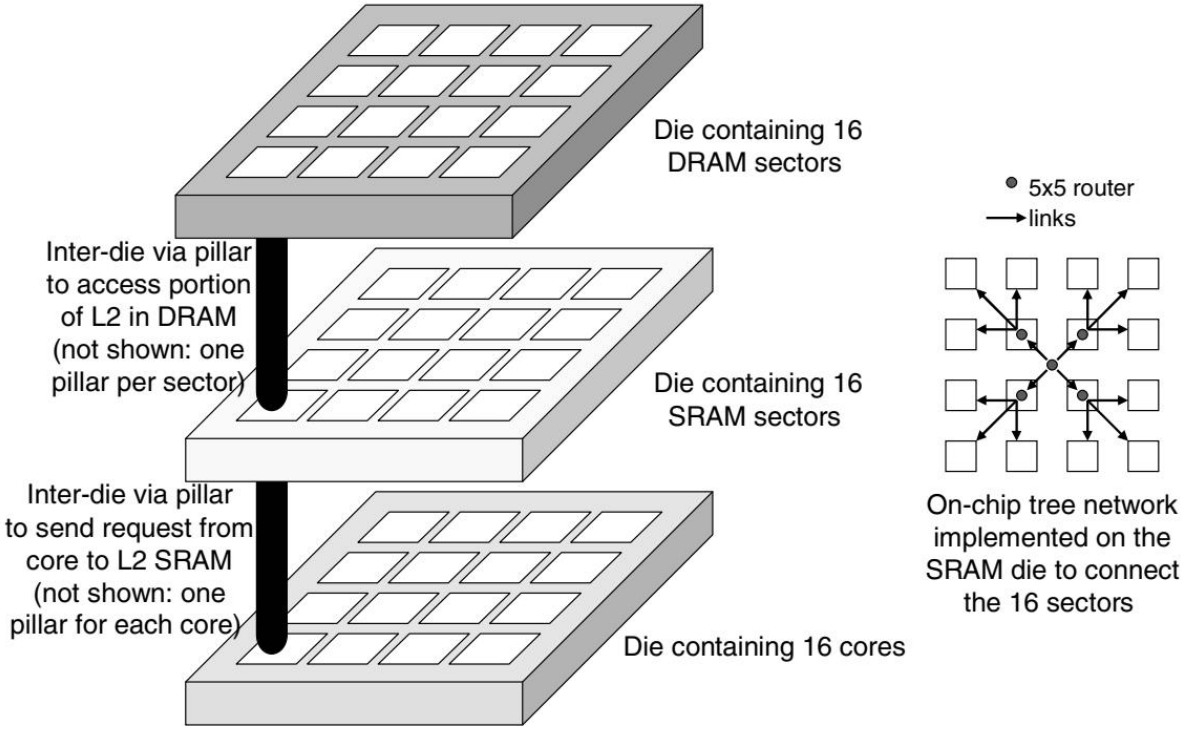


图 5.1 一种具有 SRAM 和 DRAM 裸片以及 SRAM 裸片上树形网络的 3D 可重构缓存 [9]。

存储体压力过大，则激活其上方的 DRAM 存储体，以选择性地增大 L2 缓存存储体的大小（相联度）。此时，部分 SRAM 缓存存储体用于为存储在 DRAM 存储体中的数据实现标签。因此，在 3D 堆叠中跨多个（可能是异构的）裸片组织缓存层次结构时，有许多值得考虑的选项。

5.5 新兴技术

尽管 3D 可以提高缓存容量，但它并未真正增加硅裸片中的位密度。3D 还受到硅成本、芯片的功耗和温度预算以及 TSV 带宽的限制。最后，即使采用 3D 处理器，所提供的额外面积最好通过平衡核心和缓存来利用，而不是完全分配给缓存。当权衡 3D 所需的额外制造步骤成本时，诸如 DRAM 或非易失性存储器（NVM）等替代存储技术——这些技术常因制造复杂性而在缓存应用中被忽视——可能显得颇具吸引力。通过更好的存储技术实现的指数级缩放，从长远来看可能具有更大的潜力。

过去几年中，为多核寻找合适存储技术的研究受到了极大关注。部分原因是 SRAM 相关的挑战，另外，诸如 NAND/NOR FLASH 等其他成熟技术在超越当前工艺技术时也面临缩放困难。尽管 FLASH 主要用于内存层次结构的较低层级，但在寻找 FLASH 的可扩展替代方案时，研究人员正试图设计一种可应用于内存层次结构多个层级的通用存储器。

在探索 SRAM 的替代方案时，应考虑以下重要因素，以最小化性能损失、功耗增加或过渡带来的其他有害影响。

面积

尺寸最小的单元需要占据 $4F^2$ 的面积，其中 F 是特征尺寸。这里，单元占据的面积实际上就是两条最小尺寸导线交叉点下方的面积。相比之下，一个 SRAM 单元的面积是 $120F^2$。随着多级单元的出现，通过在一个单元中存储多于一个比特，进一步有可能提高密度（或将每比特有效面积降至 $4F^2$ 以下）。虽然寻找单元面积接近或小于 $4F^2$ 的新技术具有挑战性，但找到比 SRAM 小得多的替代方案并不困难。几乎所有主流存储技术都比 SRAM 更密集。挑战在于如何高效地组织它们以获取最大收益。

访问时间

短访问时间对缓存至关重要。尽管处理器带宽的缩放并不理想，但片外导线可以接近处理器频率运行 [219]。因此，保持缓存访问延迟远快于主存以证明其存在的必要性至关重要。与其他任何存储技术相比，SRAM 单元具有最佳的延迟特性，并且读写延迟一致。也无需定期刷新。但是，由于其相对较大的尺寸，SRAM 阵列中的导线延迟主导了其访问时间。因此，在使用替代技术构建缓存时，必须利用其面积优势来保持低访问时间。

读写延迟之间的差异是许多新兴技术中的另一个主要问题。例如，在相变存储器（PCM）和磁阻 RAM（MRAM）——SRAM 的两种主要替代方案——中，写入所需时间比读取多一个数量级。这种额外延迟在单处理器中可能不是问题，因为写入通常被视为非关键操作。

但在多核中，写入对共享数据和内存模型实现的影响使得写入更为关键。此外，必须确保长写入不会阻塞关键读取。

其他基于 DRAM 的技术也可能因保持值所需的定期刷新而阻塞读取。刷新开销通常较低，但需要仔细调度刷新操作以避免性能损失。

耐久性

缓存在其生命周期内的写入次数比内存或存储磁盘高出几个数量级，而 SRAM 具有最佳的耐久性。例如，一个 SRAM 单元在 10^{17} 次写入后仍能工作，而大多数 NVM 单元在 10^8 次写入前就会失效。因此，当采用耐久性有限的 NVM 用于缓存时，需要特殊机制来延长单元寿命。这要么需要调整单元以提高耐久性（代价是能量、延迟、密度等其他关键参数），要么修改缓存架构以减少对 NVM 的写入。

字节可寻址性

缓存的访问粒度远小于主存或固态存储磁盘。因此，需要块访问的技术（如 NAND FLASH）在直接用于片上缓存时可能会遭受性能大幅下降。读取大量数据并从中选择几个字效率低下且需要高能量。

候选技术

未来系统中有几种前景看好的技术，包括自旋转移矩磁阻随机存取存储器（STT-MRAM）、忆阻器和相变存储器（PCM）。每种技术都有不同的权衡，并且必须克服重大障碍才能增强或取代 SRAM。作为近期目标，另一种成熟技术——DRAM 及其变体，

如嵌入式 DRAM（eDRAM）和三晶体管 DRAM（3T1D）——正被考虑用于片上缓存。事实上，eDRAM 已被证明在商业上可行，并已用于 IBM 的 Power 7 处理器中 [220]。本节介绍缓存存储技术的全景以及与之相关的现有工作。

5.5.1 3T1D RAM

除了 SRAM 之外，最广泛使用和理解的技术是 DRAM。DRAM 单元由一个简单的电容器和一个用于对电容器充电或放电的访问晶体管组成。由于其极简设计，它比 SRAM 的密度显著更高。然而，缓慢的充电和放电操作限制了其在存储器层次结构低层的应用。对延迟关键的 L1 需要更快的替代方案。为了解决这个问题，提出了三晶体管一二极管（3T1D）单元，这是一种类似于其他电容式存储器的技术。它依靠改变单元的电荷含量来存储逻辑 1 或 0 [221]。由于 3T1D 的面积开销相对高于其他新兴替代方案，它主要针对 L1 缓存。3T1D 有三个关键优势。首先，在设计上，3T1D 中的泄漏路径数量比 SRAM 少

即使考虑工艺偏差也是如此。其次，不是由电容器直接对长位线充电或放电，而是明智地使用二极管作为电压控制电容器来驱动下拉晶体管，进而驱动位线，从而实现与 SRAM 相似的访问速度。最后，尽管 3T1D 单元需要定期刷新，但与 DRAM 不同，读取不是破坏性的。

一般来说，3T1D 单元的刷新开销对性能的影响可以忽略不计。然而，Liang 等人 [222] 表明，在严重的工艺偏差下，一些单元需要比其他单元更频繁地刷新，而对整个缓存采用最坏情况的刷新周期会显著降低性能并增加刷新功耗。为了解决这个问题，他们提出了行级刷新策略以及针对 3T1D 缓存的新颖替换策略。关键思想是为每一行增加一个硬件计数器来跟踪保持时间，以便可以单独处理具有高刷新开销的行。同样，在存储新的缓存行时，也会考虑各个路的保持时间，以找到最佳的替换块。

然后，论文研究了三种不同复杂度的刷新策略。在简单方案中，一旦某行的保持时间低于某个阈值，该行要么被逐出，要么被无效。作为改进，在第二种方案中，刷新仅局限于那些刷新间隔低于阈值的行。因此，只有受工艺偏差影响的单元才会被刷新；其余行在需要刷新时被逐出。第三种方案根据保持值积极地刷新所有行。有趣的是，他们的研究表明，L1 中大多数缓存行的寿命远小于典型的刷新间隔。结合 3T1D 对非破坏性读取的支持，使其成为 L1 缓存中 SRAM 的有前景的替代方案。第二种方案（部分刷新）在消耗更少功耗的情况下实现了更好的性能；尽管即使存在严重的工艺偏差，不同策略之间的性能差异也小于 2%。

另一个影响刷新开销的因素是替换策略。典型的 L1 或 L2 缓存采用 LRU 策略来逐出缓存行。然而，LRU 并不适合那些有些块保持时间差、有些块完全失效（保持时间低于某个阈值的块）的缓存。例如，一个失效块在存储新块后很快会被无效，并且更频繁地显示为最近最少使用的块。为了解决这个问题，论文研究了不同的替换策略。在他们最简单的方案中，传统的 LRU 策略被扩展为避免在放置新缓存行时使用失效

块。其次，除了避免失效块之外，新行总是放置在保持时间最长的槽中。被替换的块被移动到集合内保持时间最长的下一个槽中，重复该过程直到保持时间最短的块被逐出。第三种方案与第二种类似，不同之处在于最近使用的块总是放置在最佳槽中。对这些技术的详细性能研究表明，避免失效块对于最大化性能和减少缺失至关重要。采用更复杂的替换方案仅在严重的工艺偏差下才有益。

5.5.2　嵌入式 DRAM

嵌入式 DRAM（或 eDRAM）是 IBM 的一项技术，专门旨在解决 SRAM 的密度问题而不牺牲速度。它类似于 DRAM，但使用基于高速逻辑的技术而不是传统的 DRAM 工艺实现。当考虑外围电路的开销时，一个 30 MB 的 eDRAM 缓存，在针对速度优化时，所需的面积不到同等大小 SRAM 的三分之一。它还具有优越的泄漏特性，仅消耗 SRAM 缓存泄漏的五分之一。IBM Power 7 是第一个利用该技术构建大规模 30 MB NUCA L3 缓存以供给其八个乱序核心的处理器 [220]。

任何基于 DRAM 的技术的问题在于其刷新开销，尤其是当考虑工艺偏差时。在 eDRAM 中使用高性能晶体管（低 V_t 但高泄漏的晶体管）加剧了这个问题，并将其刷新间隔比 DRAM 缩短了三个数量级。由于 eDRAM 缓存的大尺寸，如前所述在行粒度上进行微管理刷新是不可行的。为了解决这个问题，Wilkerson 等人 [223] 提出使用 ECC 来减少刷新开销。理想情况下，当一行被写入时，该行中的所有单元应保持其值直到下一个刷新周期。然而，由于工艺偏差，一些单元会提前很多失去其值。Wilkerson 等人提出采用强大的纠错码来容忍失效单元，而不是基于最坏情况保持时间刷新整行并避免失败。尽管大多数失效行可以通过简单的 SECDED 纠正，但它不能保证缓存中所有行的无故障运行。根据我们刷新缓存行的积极程度，多位错误可能不容忽视。此外，SECDED 无法识别或纠正的错误将导致应用程序崩溃或导致静默数据损坏。因此，采用了更强大的 BCH 码，它可以纠正最多五个错误并检测六个错误。BCH 的主要缺点是其纠正多位错误的高解码复杂度。为了避免长延迟惩罚，论文提出将检测过程分为两步：快速 ECC 和 Hi-ECC（或高延迟 ECC）。所有零个或一个失败位的行由快速 ECC 快速纠正，Hi-ECC 仅限于那些有多个失败的行。此外，容易发生多个失败的缓存行被禁用以进一步减少延迟惩罚。通过这些优化，与典型的 eDRAM 实现相比，他们实现了刷新功耗降低 93%。

5.5.3　非易失性存储器

大多数新兴技术都是非易失性的，并且不是基于电荷的。相反，它们依靠改变存储单元的电阻或磁特性来存储值。因此，不存在 DRAM 或 SRAM 中的漏电流问题。表 5.1 列出了一些有前景的存储器技术及其延迟、功耗、面积和耐久性特征。可以注意到，与 SRAM 或 DRAM 及其变体相比，它们中的大多数耐久性较差。此外，写入的开销（无论是在延迟还是功耗方面）都显著高于读取。延长寿命、隐藏写入延迟和降低写入

功耗的技术是活跃的研究领域。在这里，我们简要

提及最近几项利用潜在颠覆性技术用于片上缓存的研究工作。有兴趣的读者可以参阅 Qureshi 等人关于相变存储器的综合讲座，以了解该领域的最新工作 [14]。

表 5.1 不同存储技术的性能与物理特性比较（原书表 5.1）

参数	SRAM	DRAM	eDRAM	NAND 闪存	PCM	MRAM
单元面积	$120F^2$	$6-8F^2$	$16F^2$	$4-6F^2$	$4-16F^2$	$16-40F^2$
读取时间	极快	快	快于 DRAM	慢	慢于 DRAM	快
写入时间	极快	快	快于 DRAM	远慢于 PCM	极慢	慢
耐久次数	$>10^{16}$	$>10^{16}$	$>10^{16}$	10^5	10^8	10^5
非易失性	否	否	否	是	是	是
写入功耗	极低	低	低	极高	高	高

3D 堆叠 MRAM，Sun 等人，HPCA 2009

采用新技术的关键挑战之一是如何将其集成到现有制造工艺中的成本。磁阻 RAM（MRAM）通过改变每个单元的磁场来存储值。该技术需要特殊的磁板和不同于传统 CMOS 工艺的制造步骤。为了解决这个问题，Sun 等人 [215] 探索了一种 3D 堆叠设计，其中 MRAM 堆叠在传统 CMOS 处理器之上。研究还表明，MRAM 的高写入延迟和写入能量使其无法成为高级缓存（如 L2）的明显赢家，尤其是对于内存密集型应用。论文提出使用大型写缓冲区和特殊的调度策略（优先读取而非写入）来克服延迟问题。为了减少对 MRAM 的写入次数（从而降低功耗），他们提出了一种用于 L2 的混合缓存，由 SRAM 和 MRAM 存储体组成。频繁写入的缓存行要么放置在 SRAM 存储体中，要么迁移到 SRAM 存储体。他们的分析显示，对于单线程工作负载，结果很有前景。然而，如前所述，在 CMP 中，这种解决方案也可能导致性能损失，因为即使在 CMP 中，写入也常常处于关键路径上。此外，通过使用小型 L2 SRAM 缓存并将 MRAM 限制用于较低级别的 L3 缓存，可能实现混合缓存的效果。Wu 等人的下一篇论文 [214] 对混合缓存进行了详细的设计空间探索，阐明了其中一些选项。

混合缓存，Wu 等人，ISCA 2009

Wu 等人 [214] 展示了在混合缓存架构中不同技术之间的对比表现。他们对三种不同的缓存架构进行了详细的功耗和性能研究，并且对于每种架构，他们考虑了使用 SRAM、MRAM、PCM 和 eDRAM 构建的缓存。

他们的第一种架构（LHCA）模拟了传统的三级缓存层次结构：由 SRAM 构成的私有 L1、同样由 SRAM 构成的私有 L2，以及共享 L3，后者可以采用上述任何存储技术。L3 缓存的大小设置使得所有评估配置的有效面积相同。

他们的第二种架构（RHCA）模拟了一种混合 L2 缓存，其中原始设计中的 L2 和 L3 合并为一个多存储体缓存。每个存储体仍保留其单元类型，从而使缓存成为混合型。由于 SRAM 存储体总是更快，因此最大化由它们处理的请求数量是有益的。因此，论

文提出了一种迁移策略，将频繁使用的块从较慢的存储体移动到 SRAM 存储体。他们的策略与 D-NUCA 最为相似，但在存储体之间交换块的时机上有所不同。他们的映射方案类似于 D-NUCA，其中每个组分布在多个存储体（称为存储体组）上以便于交换。为了搜索一个块，请求被发送到存储体组中的所有存储体。慢速存储体中缓存行的关键性通过跟踪每个缓存行的访问次数来确定。这是通过为每个缓存行维护一个硬件饱和计数器来实现的。在快速 SRAM 存储体中，不是使用计数器，而是为每个缓存行维护一个粘滞位，每当缓存行被访问时该位被置位。一旦饱和计数器的最高有效位被置位，慢速存储体中的块就准备迁移。然而，只有当快速存储体中对应块的粘滞位被置零时，才允许交换。否则，不进行交换，而是将粘滞位重置为零。这缓解了两个热块在慢速和快速存储体之间频繁交换的问题。

第三种架构利用 3D 堆叠来大幅增加容量。在上述讨论的 CMP 架构中，一个专门为缓存设计的新芯片堆叠在处理器之上。由于堆叠缓存通常面向较低级别，快速访问时间不是关键因素，但高密度有助于降低缺失率。PCM 因其相对较低的面积需求，在他们所有的 3D 配置中被用作最后一级缓存。与前两种讨论的架构类似，考虑了采用 LHCA 和 RCHA 各种组合的 3D 堆叠设计。这包括两种 RHCA 组合：（1）由 SRAM 和 eDRAM 组成的混合 L2 缓存，以及用于 L3 的堆叠 PCM；（2）由 SRAM、eDRAM 和 PCM 组成的混合 L2 缓存。

以下是他们研究的一些关键要点。在 LHCA 设计中，在 L3 MRAM 缓存之前设置大型 L2 SRAM 缓存能有效过滤大量对 MRAM 的写入。尽管 MRAM 的写入功耗特性较差，但采用 MRAM 的 LHCA 设计最终获得了最佳的功耗值。在性能方面，使用 eDRAM L3 缓存比 MRAM 更有益，尽管它们之间的性能差异非常小。在 RHCA 和性能最佳的 LHCA 之间，RHCA 实现了更好的性能，但差异小于 2%。在 3D 设计中，RHCA L2（由 SRAM 和 eDRAM 组成）与 PCM L3 的组合实现了最佳性能。同样，3D LHCA 的性能也非常接近。很可能，与简单得多的 LHCA 设计相比，RHCA 所需的复杂搜索过程和簿记工作难以证明其合理性。

总结

面向新兴技术的缓存架构仍是一个活跃的研究领域，仍有很大的改进空间。例如，大多数即将出现的技术都是非易失性的，但现有的所有方案都忽略了这一点。许多以前需要操作系统或 BIOS 支持的可靠性技术，现在由于数据具有持久性，可以完全在处理器内部的硬件中实现。此外，调整新兴非易失性存储器（NVM）的特性，使其既适用于 CMP 缓存又适用于主存，是一个有趣的研究方向。尽管许多先前的研究已经探索了将存储器堆叠在处理器上的好处 [224]，但新兴技术对此类设计的影响仍未解决。

第六章　结束语

经过数十年的缓存研究，研究人员仍在不断取得重大进展，这令人惊叹。过去十年出现了重大创新：非均匀缓存访问的概念、模糊共享与私有边界的组织、缓存与网络的更紧密集成、插入、替换和分区的创造性策略。对更好策略的需求持续存在。最先进的缓存管理与最优缓存管理方案（例如 Belady 的 OPT 算法 [86]）的性能之间存在巨大差距。大型缓存中近一半的缓存块在驱逐前从未被访问过。因此，神谕式的标准设置得非常高，我们将继续朝着它迈出小步。关键挑战在于通过节能且不过于复杂的技术来实现这些小步。

在未来十年，我们不仅会继续看到片上缓存的改进，还会看到许多缓存技术被应用到内存层次结构的新层级。随着工艺技术的不断进步，我们将在内存层次结构中看到新的产物：新单元、新问题、3D 堆叠、包含 DRAM 和 NVM 的片外层次结构、非均匀的片外互连延迟、以及非均匀的片外排队延迟等等。这凸显了理解新兴技术现象的重要性，也是本书包含技术问题章节的原因。我们预计在片外层次结构内的数据管理领域将出现大量工作。虽然片外层次结构在过去相对简单，但未来的层次结构可能由不同技术的组合构成：某种提供高容量的 NVM 技术（PCM、STT-RAM），以及可能提供低延迟访问的类似 SRAM/DRAM 的技术。鉴于 DDRx 电气内存总线仅支持一个或两个接入点（DIMM），需要某种形式的缓冲片外网络（例如 FB-DIMM 的更好化身）来支持大规模多核多插槽系统的高内存容量需求。因此，片外层次结构将开始类似于本书重点关注的网络化 L1-L2 层次结构。它们还将受到这些新技术细微差别的约束，例如 PCM 有限的写入耐久性、行缓冲区的存在或排队延迟的显著成本。访问这些片外存储器预计将对整体系统延迟和能耗产生重大贡献。本书涵盖的许多逻辑缓存策略将需要适应片外数据管理工作。

甚至有可能设计出协调的片上缓存和片外访问机制，以降低整体系统延迟和能耗。最近的一篇论文尝试通过查看片上缓存中已死且可以急切驱逐的块来优化内存行缓冲区命中率 [225]。另一篇最近的论文尝试识别内存系统中的热 rank，并修改第 6 章

片上缓存替换策略，以降低对将被频繁访问的 DRAM 行（热行）的块的驱逐优先级 [226]。因此，我们预计这种协调缓存和内存访问机制的领域在未来几年将受到广泛关注。

显然，在大规模多核系统的片上与片外缓存领域，有趣的未来工作可能性并不缺乏。

参考文献

[1] C. Kim, D. Burger, and S. Keckler. An Adaptive, Non-Uniform Cache Structure for Wire-Dominated On-Chip Caches. In Proceedings of ASPLOS, 2002. DOI：10.1145/605397.605420

[2] B.M. Beckmann and D.A. Wood. Managing Wire Delay in Large Chip-Multiprocessor Caches. In Proceedings of MICRO-37, December 2004. DOI：10.1109/MICRO.2004.21

[3] N. Hardavellas, M. Ferdman, B. Falsafi, and A. Ailamaki. Reactive NUCA: Near-Optimal Block Placement And Replication In Distributed Caches. In Proceedings of ISCA, 2009. DOI：10.1145/1555754.1555779

[4] N. Muralimanohar, R. Balasubramonian, and N. Jouppi. Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0. In Proceedings of MICRO, 2007. DOI：10.1109/MICRO.2007.33

[5] Y. Jin, E. J. Kim, and K. H. Yum. A Domain-Specific On-Chip Network Design for Large Scale Cache Systems. In Proceedings of HPCA, 2007. DOI：10.1109/HPCA.2007.346209

[6] Z. Guz, I. Keidar, A. Kolodny, and U. Weiser. Utilizing Shared Data in Chip Multiprocessors with the Nahalal Architecture. In Proceedings of SPAA, June 2008. DOI：10.1145/1378533.1378535

[7] N. Muralimanohar and R. Balasubramonian. Interconnect Design Considerations for Large NUCA Caches. In Proceedings of ISCA, 2007. DOI：10.1145/1250662.1250708

[8] A. Udipi, N. Muralimanohar, and R. Balasubramonian. Towards Scalable, Energy-Efficient, Bus-Based On-Chip Networks. In Proceedings of HPCA, 2010. DOI：10.1109/HPCA.2010.5416639

[9] N. Madan, L. Zhao, N. Muralimanohar, A. Udipi, R. Balasubramonian, R. Iyer, S. Makineni, and D. Newell. Optimizing Communication and Capacity in a 3D Stacked Reconfigurable Cache Hierarchy. In Proceedings of HPCA, 2009. DOI：10.1109/HPCA.2009.4798261

[10] B. Jacob. The Memory System: You Can't Avoid It, You Can't Ignore It, You Can't Fake it. Morgan & Claypool Synthesis Lectures on Computer Architecture, 2009. DOI：10.2200/S00201ED1V01Y200907CAC007 DOI: 10.2200/S00201ED1V01Y200907CAC007

[11] N. Jerger and L. Peh. On-Chip Networks. Morgan & Claypool Synthesis Lectures on Computer Architecture, 2009. DOI：10.2200/S00209ED1V01Y200907CAC008

[12] A. Gonzalez, F. Latorre, and G. Magklis. Processor Microarchitecture: An Implementation Perspective. Morgan & Claypool Synthesis Lectures on Computer Architecture, 2010. DOI：10.2200/S00309ED1V01Y201011CAC012

[13] D. Sorin, M. Hill, and D. Wood. A Primer on Memory Consistency and Cache Coherence. Morgan & Claypool Synthesis Lectures on Computer Architecture, 2011. DOI：10.2200/S00346ED1V01Y201104CAC016

[14] M. Qureshi, S. Gurumurthi, and B. Rajendran. Phase Change Memory: From Devices to Systems.

Morgan & Claypool Synthesis Lectures on Computer Architecture, 2011. xiii,

[15] S. Kaxiras and M. Martonosi. Computer Architecture Techniques for Power-Efficiency. Morgan & Claypool Synthesis Lectures on Computer Architecture, 2008. DOI: 10.2200/S00119ED1V01Y200805CAC004

[16] David E. Culler and Jaswinder Pal Singh. Parallel Computer Architecture: A Hardware/Software Approach. Morgan Kaufmann Publishers, 1999.

[17] J. L. Hennessy and D. A. Patterson. Computer Architecture: A Quantitative Approach. Elsevier, 4th edition, 2007.

[18] A. Jaleel, M. Mattina, and B. Jacob. Last Level Cache (LLC) Performance of Data Mining Workloads on a CMP – A Case Study of Parallel Bioinformatics Workloads. In Proceedings of HPCA, 2006. DOI: 10.1109/HPCA.2006.1598115

[19] C. Bienia, S. Kumar, and K. Li. PARSEC vs. SPLASH-2: A Quantitative Comparison of Two Multithreaded Benchmark Suites on Chip-Multiprocessors. In Proceedings of IISWC, 2008. DOI: 10.1109/IISWC.2008.4636090

[20] B. Beckmann, M. Marty, and D. Wood. ASR: Adaptive Selective Replication for CMP Caches. In Proceedings of MICRO, 2006. DOI: 10.1109/MICRO.2006.10

[21] L. A. Barroso, K. Gharachorloo, R. McNamara, A. Nowatzyk, S. Qadeer, B. Sano, S. Smith, R. Stets, and B. Verghese. Piranha: A Scalable Architecture Based on Single-Chip Multiprocessing. In Proceedings of ISCA-27, pages 282–293, June 2000. DOI: 10.1145/342001.339696

[22] J. Huh, C. Kim, H. Shafi, L. Zhang, D. Burger, and S. Keckler. A NUCA Substrate for Flexible CMP Cache Sharing. In Proceedings of ICS-19, June 2005. DOI: 10.1109/TPDS.2007.1091

[23] Z. Chishti, M. Powell, and T.N. Vijaykumar. Optimizing Replication, Communication, and Capacity Allocation in CMPs. In Proceedings of ISCA-32, June 2005. DOI: 10.1145/1080695.1070001

[24] Z. Chishti, M. Powell, and T.N. Vijaykumar. Distance Associativity for High-Performance Energy-Efficient Non-Uniform Cache Architectures. In Proceedings of MICRO-36, December 2003. DOI: 10.1109/MICRO.2003.1253183

[25] A. Jaleel, E. Borch, M. Bhandaru, S. Steely, and J. Emer. Achieving Non-Inclusive Cache Performance with Inclusive Caches – Temporal Locality Aware (TLA) Cache Management Policies. In Proceedings of MICRO, 2010. DOI: 10.1109/MICRO.2010.52

[26] C. Liu, A. Sivasubramaniam, and M. Kandemir. Organizing the Last Line of Defense before Hitting the Memory Wall for CMPs. In Proceedings of HPCA, February 2004. DOI: 10.1109/HPCA.2004.10017

[27] H. Dybdahl and P. Stenstrom. An Adaptive Shared/Private NUCA Cache Partitioning Scheme for Chip Multiprocessors. In Proceedings of HPCA, 2007. DOI: 10.1109/HPCA.2007.346180 DOI: 10.1109/HPCA.2007.346180

[28] M. Qureshi, D. Thompson, and Y. Patt. The V-Way Cache: Demand Based Associativity via Global Replacement. In Proceedings of ISCA, 2005. DOI: 10.1145/1080695.1070015

[29] D. Sanchez and C. Kozyrakis. The ZCache: Decoupling Ways and Associativity. In Proceedings

of MICRO, 2010. DOI：10.1109/MICRO.2010.20

[30] R. Ricci, S. Barrus, D. Gebhardt, and R. Balasubramonian. Leveraging Bloom Filters for Smart Search Within NUCA Caches. In Proceedings of the 7th Workshop on Complexity-Effective Design, held in conjunction with ISCA-33, June 2006.

[31] Burton Bloom. Space/time trade-offs in hash coding with allowable errors, July 1970. DOI：10.1145/362686.362692

[32] J. Merino, V. Puente, P. Prieto, and J. Gregorio. SP-NUCA: A Cost Effective Dynamic Non-Uniform Cache Architecture. Computer Architecture News, 2008. DOI：10.1145/1399972.1399973

[33] J. Merino, V. Puente, and J. Gregorio. ESP-NUCA: A Low-Cost Adaptive Non-Uniform Cache Architecture. In Proceedings of HPCA, 2010. DOI：10.1109/HPCA.2010.5416641

[34] R. E. Kessler and Mark D. Hill. Page Placement Algorithms for Large Real-Indexed Caches. ACM Trans. Comput. Syst., 10(4), 1992. DOI：10.1145/138873.138876

[35] R. Chandra, S. Devine, B. Verghese, A. Gupta, and M. Rosenblum. Scheduling and Page Migration for Multiprocessor Compute Servers. In Proceedings of ASPLOS, 1994. DOI：10.1145/381792.195485

[36] J. Corbalan, X. Martorell, and J. Labarta. Page Migration with Dynamic Space-Sharing Scheduling Policies: The case of SGI 02000. International Journal of Parallel Programming, 32(4), 2004. DOI：10.1023/B:IJPP.0000035815.13969.ec

[37] B. Verghese, S. Devine, A. Gupta, and M. Rosenblum. Operating system support for improving data locality on CC-NUMA compute servers. SIGPLAN Not., 31(9), 1996. DOI：10.1145/248209.237205

[38] R.P. LaRowe, J.T. Wilkes, and C.S. Ellis. Exploiting Operating System Support for Dynamic Page Placement on a NUMA Shared Memory Multiprocessor. In Proceedings of PPOPP, 1991. DOI：10.1145/109626.109639

[39] R.P. LaRowe and C.S. Ellis. Experimental Comparison of Memory Management Policies for NUMA Multiprocessors. Technical report, 1990. DOI：10.1145/118544.118546

[40] R.P. LaRowe and C.S. Ellis. Page Placement policies for NUMA multiprocessors. J. Parallel Distrib. Comput., 11(2), 1991. DOI：10.1016/0743-7315(91)90117-R

[41] S. Cho and L. Jin. Managing Distributed, Shared L2 Caches through OS-Level Page Allocation. In Proceedings of MICRO, 2006. DOI：10.1109/MICRO.2006.31 DOI: 10.1109/MICRO.2006.31

[42] M. Awasthi, K. Sudan, R. Balasubramonian, and J. Carter. Dynamic Hardware-Assisted Software-Controlled Page Placement to Manage Capacity Allocation and Sharing within Large Caches. In Proceedings of HPCA, 2009.

[43] L. Jin and S. Cho. SOS: A Software-Oriented Distributed Shared Cache Management Approach for Chip Multiprocessors. In Proceedings of PACT, 2009. DOI：10.1109/PACT.2009.14

[44] M. Marty and M. Hill. Virtual Hierarchies to Support Server Consolidation. In Proceedings of ISCA, 2007. DOI：10.1145/1250662.1250670

[45] M. Chaudhuri. PageNUCA: Selected Policies For Page-Grain Locality Management In Large

Shared Chip-Multiprocessor Caches. In Proceedings of HPCA, 2009. DOI: 10.1109/HPCA.2009.4798258

[46] J. Lin, Q. Lu, X. Ding, Z. Zhang, X. Zhang, and P. Sadayappan. Gaining Insights into Multicore Cache Partitioning: Bridging the Gap between Simulation and Real Systems. In Proceedings of HPCA, 2008. DOI: 10.1109/HPCA.2008.4658653

[47] J. Lin, Q. Lu, X. Ding, Z. Zhang, X. Zhang, and P. Sadayappan. Enabling Software Management for Multicore Caches with a Lightweight Hardware Support. In Proceedings of Supercomputing, 2009. DOI: 10.1145/1654059.1654074

[48] J. Chang and G. Sohi. Co-Operative Caching for Chip Multiprocessors. In Proceedings of ISCA, 2006. DOI: 10.1145/1150019.1136509

[49] E. Speight, H. Shafi, L. Zhang, and R. Rajamony. Adaptive Mechanisms and Policies for Managing Cache Hierarchies in Chip Multiprocessors. In Proceedings of ISCA, 2005. DOI: 10.1109/ISCA.2005.8

[50] E. Herrero, J. Gonzalez, and R. Canal. Distributed Cooperative Caching. In Proceedings of PACT, 2008. DOI: 10.1145/1454115.1454136

[51] E. Herrero, J. Gonzalez, and R. Canal. Elastic Cooperative Caching: An Autonomous Dynamically Adaptive Memory Hierarchy for Chip Multiprocessors. In Proceedings of ISCA, 2010. DOI: 10.1145/1816038.1816018

[52] M. K. Qureshi. Adaptive Spill-Receive for Robust High-Performance Caching in CMPs. In Proceedings of HPCA, 2009.

[53] Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely, and Joel Emer. Adaptive Insertion Policies for High Performance Caching. In Proceedings of ISCA, 2007. DOI: 10.1145/1250662.1250709

[54] H. Lee, S. Cho, and B. Childers. CloudCache: Expanding and Shrinking Private Caches. In Proceedings of HPCA, 2011. DOI: 10.1109/HPCA.2011.5749731

[55] H. Lee, S. Cho, and B. Childers. StimulusCache: Boosting Performance of Chip Multiprocessors with Excess Cache. In Proceedings of HPCA, 2010. DOI: 10.1109/HPCA.2010.5416644

[56] S. Srikantaiah, E. Kultursay, T. Zhang, M. Kandemir, M. Irwin, and Y. Xie. MorphCache: A Reconfigurable Adaptive Multi-Level Cache Hierarchy for CMPs. DOI: 10.1109/HPCA.2011.5749732

[57] T. Yeh and G. Reinman. Fast and Fair: Data-Stream Quality of Service. In Proceedings of CASES, 2005. DOI: 10.1145/1086297.1086328

[58] L. Hsu, S. Reinhardt, R. Iyer, and S. Makineni. Communist, Utilitarian, and Capitalist Cache Policies on CMPs: Caches as a Shared Resource. In Proceedings of PACT, 2006. DOI: 10.1145/1152154.1152161

[59] M. Qureshi, D. Lynch, O. Mutlu, and Y. Patt. A Case for MLP-Aware Cache Replacement. In Proceedings of ISCA, 2006. DOI: 10.1109/ISCA.2006.5

[60] M. Qureshi and Y. Patt. Utility-Based Cache Partitioning: A Low-Overhead, High-Performance, Runtime Mechanism to Partition Shared Caches. In Proceedings of MICRO, 2006. DOI: 10.110

9/MICRO.2006.49

[61] G.E. Suh, L. Rudolph, and S. Devadas. Dynamic Partitioning of Shared Cache Memory. J. Supercomput., 28(1), 2004. DOI：10.1023/B:SUPE.0000014800.27383.8f

[62] X. Lin and R. Balasubramonian. Refining the Utility Metric for Utility-Based Cache Partitioning. In Proceedings of WDDD, 2011.

[63] G. Suo, X. Yang, G. Liu, J. Wu, K. Zeng, B. Zhang, and Y. Lin. IPC-Based Cache Partitioning: An IPC-Oriented Dynamic Shared Cache Partitioning Mechanism. In Proceedings of ICHIT, 2008. DOI：10.1109/ICHIT.2008.164

[64] A. Jaleel, W. Hasenplaugh, M. Qureshi, J. Sebot, Jr. S. Steely, and J. Emer. Adaptive Insertion Policies For Managing Shared Caches. In Proceedings of PACT, 2008. DOI：10.1145/1454115.1454145

[65] Y. Xie and G. Loh. PIPP: Promotion/Insertion Pseudo-Partitioning of Multi-Core Shared Caches. In Proceedings of ISCA, 2009. DOI：10.1145/1555815.1555778

[66] W. Liu and D. Yeung. Using Aggressor Thread Information to Improve Shared Cache Management for CMPs. In Proceedings of PACT, 2009. DOI：10.1109/PACT.2009.13

[67] R. Balasubramonian, S. Dwarkadas, and D.H. Albonesi. Dynamically Managing the Communication-Parallelism Trade-Off in Future Clustered Processors. In Proceedings of ISCA-30, pages 275–286, June 2003. DOI：10.1145/871656.859650

[68] S. Dropsho, A. Buyuktosunoglu, R. Balasubramonian, D. H. Albonesi, S. Dwarkadas, G. Semeraro, G. Magklis, and M. L. Scott. Integrating Adaptive On-Chip Storage Structures for Reduced Dynamic Power. In Proceedings of the 11th International Conference on Parallel Architectures and Compilation Techniques (PACT), pages 141–152, September 2002. DOI：10.1109/PACT.2002.1106013

[69] N. Rafique, W. Lim, and M. Thottethodi. Architectural Support for Operating System Driven CMP Cache Management. In Proceedings of PACT, 2006. DOI：10.1145/1152154.1152160

[70] J. Chang and G. Sohi. Co-Operative Cache Partitioning for Chip Multiprocessors. In Proceedings of ICS, 2007. DOI：10.1145/1274971.1275005

[71] S. Srikantaiah, M. Kandemir, and M. Irwin. Adaptive Set Pinning: Managing Shared Caches in Chip Multiprocessors. In Proceedings of ASPLOS, 2008. DOI：10.1145/1353534.1346299

[72] T. Sherwood, B. Calder, and J. Emer. Reducing Cache Misses Using Hardware and Software Page Placement. In Proceedings of SC, 1999. DOI：10.1145/305138.305189

[73] X. Jiang, A. Mishra, L. Zhao, R. Iyer, Z. Fang, S. Srinivasan, S. Makineni, P. Brett, and C. Das. ACCESS: Smart Scheduling for Asymmetric Cache CMPs. In Proceedings of HPCA, 2011. DOI：10.1109/HPCA.2011.5749757

[74] D. Chandra, F. Guo, S. Kim, and Y. Solihin. Predicting inter-thread cache contention on a chip-multiprocessor architecture. In Proceedings of HPCA-11, February 2005. DOI：10.1109/HPCA.2005.27

[75] D. Tam, R. Azimi, L. Soares, and M. Stumm. RapidMRC: Approximating L2 Miss Rate Curves on Commodity Systems for Online Optimizations. In Proceedings of ASPLOS, 2009. DOI：10.1

145/1508284.1508259

[76] S. Zhuravlev, S. Blagodurov, and A. Fedorova. Addressing Shared Resource Contention in Multicore Processors via Scheduling. In Proceedings of ASPLOS, 2010. DOI：10.1145/1735971.1736036

[77] R. Iyer, L. Zhao, F. Guo, R. Illikkal, D. Newell, Y. Solihin, L. Hsu, and S. Reinhardt. QoS Policies and Architecture for Cache/Memory in CMP Platforms. In Proceedings of SIGMETRICS, 2007. DOI：10.1145/1254882.1254886

[78] R. Iyer. CQoS: A Framework for Enabling QoS in Shared Caches of CMP Platforms. In Proceedings of ICS, 2004. DOI：10.1145/1006209.1006246

[79] K. Varadarajan, S. Nandy, V. Sharda, A. Bharadwaj, R. Iyer, S. Makineni, and D. Newell. Molecular Caches: A Caching Structure for Dynamic Creation of Application-Specific Heterogeneous Cache Regions. In Proceedings of MICRO, 2006. DOI：10.1109/MICRO.2006.38

[80] F. Guo, Y. Solihin, L. Zhao, and R. Iyer. A Framework for Providing Quality of Service in Chip Multi-Processors. In Proceedings of MICRO, 2007. DOI：10.1109/MICRO.2007.6

[81] Li Zhao, Ravi Iyer, Ramesh Illikkal, Jaideep Moses, Srihari Makineni, and Don Newell. CacheScouts: Fine-Grain Monitoring of Shared Caches in CMP Platforms. In Proceedings of PACT, 2007. DOI：10.1109/PACT.2007.19

[82] S. Srikantaiah, M. Kandemir, and Q. Wang. SHARP Control: Controlled Shared Cache Management in Chip Multiprocessors. In Proceedings of MICRO, 2009. DOI：10.1145/1669112.1669177

[83] S. Srikantaiah, R. Das, A. Mishra, C. Das, and M. Kandemir. A Case for Integrated Processor-Cache Partitioning in Chip Multiprocessors. In Proceedings of Supercomputing, 2009. DOI：10.1145/1654059.1654066

[84] K. Nesbit, J. Laudon, and J. E. Smith. Virtual private caches. In Proceedings ISCA, 2007. DOI：10.1145/1250662.1250671

[85] S. Kim, D. Chandra, and Y. Solihin. Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture. In Proceedings PACT, 2004. DOI：10.1109/PACT.2004.1342546

[86] L. Belady. A Study of Replacement Algorithms for a Virtual-Storage Computer. IBM Systems Journal, 1966. DOI：10.1147/sj.52.0078

[87] J. Jeong and M. Dubois. Optimal Replacements in Caches with Two Miss Costs. In Proceedings of SPAA, 1999. DOI：10.1145/305619.305636

[88] J. Jeong and M. Dubois. Cost-Sensitive Cache Replacement Algorithms. In Proceedings of HPCA, 2003. DOI：10.1109/HPCA.2003.1183550

[89] W. Wong and J. Baer. Modified LRU Policies for Improving Second-Level Cache Behavior. In Proceedings of HPCA, 2000. DOI：10.1109/HPCA.2000.824338

[90] A. Jaleel, K. Theobald, S. Steely, and J. Emer. High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). In Proceedings of ISCA, 2010.

[91] S. Srinivasan, R. Ju, A. Lebeck, and C. Wilkerson. Locality vs. Criticality. In Proceedings of ISCA-28, pages 132–143, July 2001. DOI：10.1109/ISCA.2001.937442

[92] R. Balasubramonian, V. Srinivasan, and S. Dwarkadas. Hot-and-Cold: Using Criticality in the Design of Energy-Efficient Caches. In Workshop on Power-Aware Computer Systems, in conjunction with MICRO-36, December 2003.

[93] R. Subramanian, Y. Smaragdakis, and G. Loh. Adaptive Caches: Effective Shaping of Cache Behavior to Workloads. In Proceedings of MICRO-39, December 2006. DOI：10.1109/MICRO.2006.7

[94] G. Keramidas, P. Petoumenos, and S. Kaxiras. Cache Replacement based on Reuse-Distance Prediction. In Proceedings of ICCD, 2007. DOI：10.1109/ICCD.2007.4601909

[95] K. Rajan and R. Govindarajan. Emulating Optimal Replacement with a Shepherd Cache. In Proceedings of MICRO, 2007. DOI：10.1109/MICRO.2007.14

[96] J. Zebchuk, S. Makineni, and D. Newell. Re-examining Cache Replacement Policies. In Proceedings of ICCD, 2008. DOI：10.1109/ICCD.2008.4751933

[97] K. So and R. Rechtshaffen. Cache Operations by MRU Change. IEEE Transactions on Computers, 37(6), June 1988. DOI：10.1109/12.2208

[98] D. Lee, J. Choi, J. Kim, S. Noh, S. Min, Y. Cho, and C. Kim. LRFU: A Spectrum of Policies that Subsumes the Least Recently Used and Least Frequently Used Policies. IEEE Trans. on Computers, 50(12), 2001. DOI：10.1109/TC.2001.970573

[99] M. Chaudhuri. Pseudo-LIFO: The Foundation of a New Family of Replacement Policies for Last-level Caches. In Proceedings of MICRO, 2009. DOI：10.1145/1669112.1669164

[100] M. Kharbutli and Y. Solihin. Counter-Based Cache Replacement and Bypassing Algorithms. IEEE Trans. on Computers, 2008. DOI：10.1109/TC.2007.70816

[101] H. Liu, M. Ferdman, J. Huh, and D. Burger. Cache Bursts: A New Approach for Eliminating Dead Blocks and Increasing Cache Efficiency. In Proceedings of MICRO, 2008. DOI：10.1109/MICRO.2008.4771793

[102] S. Khan, D. Jimenez, D. Burger, and B. Falsafi. Using Dead Blocks as a Virtual Victim Cache. In Proceedings of PACT, 2010. DOI：10.1145/1854273.1854333

[103] A. Basu, N. Kirman, M. Kirman, M. Chaudhuri, and J. Martinez. Scavenger: A New Last Level Cache Architecture with Global Block Priority. In Proceedings of MICRO, 2007. DOI：10.1109/MICRO.2007.36

[104] N. Jouppi. Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers. In Proceedings of ISCA-17, pages 364–373, May 1990. DOI：10.1109/ISCA.1990.134547

[105] R. Manikantan, K. Rajan, and R. Govindarajan. NUcache: An Efficient Multicore Cache Organization Based on Next-Use Distance. In Proceedings of HPCA, 2011. DOI：10.1145/1854273.1854356

[106] L. Soares, D. Tam, and M. Stumm. Reducing the Harmful Effects of Last-Level Cache Polluters with an OS-Level, Software-Only Pollute Buffer. In Proceedings of MICRO, 2008. DOI：10.1109/MICRO.2008.4771796

[107] Intel. Intel Core i7 Processor. http://www.intel.com/products/processor/corei7/spec

ifications.htm.

[108] AMD. AMD Athlon Processor and AMD Duron Processor with Full-Speed On-die L2 Cache, 2000.

[109] A. Seznec. A Case for Two-Way Skewed-Associative Caches. In Proceedings of ISCA, 1993. DOI: 10.1145/173682.165152

[110] M. Kharbutli, K. Irwin, Y. Solihin, and J. Lee. Using Prime Numbers for Cache Indexing to Eliminate Conflict Misses. In Proceedings of HPCA, 2004. DOI: 10.1109/HPCA.2004.10015

[111] D. Rolan, B. Fraguela, and R. Doallo. Adaptive Line Placement with the Set Balancing Cache. In Proceedings of MICRO, 2009. DOI: 10.1145/1669112.1669178

[112] C. Zhang. Balanced Cache: Reducing Conflict Misses of Direct-Mapped Caches. In Proceedings of ISCA, 2006. DOI: 10.1145/1150019.1136499

[113] D. Zhan, H. Jiang, and S. Seth. STEM: Spatiotemporal Management of Capacity for Intra-Core Last Level Caches. In Proceedings of MICRO, 2010. DOI: 10.1109/MICRO.2010.31

[114] T. Chen and J. Baer. Effective Hardware Based Data Prefetching for High Performance Processors. IEEE Transactions on Computers, 44(5):609–623, May 1995. DOI: 10.1109/12.381947

[115] A.J. Smith. Cache Memories. Computing Surveys, 14(4), 1982. DOI: 10.1145/356887.356892

[116] Alaa Alameldeen and David Wood. Interactions Between Compression and Prefetching in Chip Multiprocessors. In Proceedings of HPCA, 2007. DOI: 10.1109/HPCA.2007.346200

[117] E. Ebrahimi, O. Mutlu, and Y. Patt. Techniques for Bandwidth-Efficient Prefetching of Linked Data Structures in Hybrid Prefetching Systems. In Proceedings of HPCA, 2009.

[118] A. Lai, C. Fide, and B. Falsafi. Dead-Block Prediction and Dead-Block Correlating Prefetchers. In Proceedings of ISCA, 2001. DOI: 10.1145/379240.379259

[119] D. Wood, M. Hill, and R. Kessler. A Model for Estimating Trace-Sample Miss Ratios. In Proceedings of SIGMETRICS, 1991. DOI: 10.1145/107972.107981

[120] Z. Hu, S. Kaxiras, and M. Martonosi. Timekeeping in the Memory System: Predicting and Optimizing Memory Behavior. In Proceedings of ISCA, 2002. DOI: 10.1145/545214.545239

[121] M. Ferdman and B. Falsafi. Last-Touch Correlated Data Streaming. In Proceedings of ISPASS, 2007. DOI: 10.1109/ISPASS.2007.363741

[122] K. Flautner, N.S. Kim, S. Martin, D. Blaauw, and T. Mudge. Drowsy Caches: Simple Techniques for Reducing Leakage Power. In Proceedings of ISCA, 2002. DOI: 10.1109/ISCA.2002.1003572

[123] J. Abella, A. Gonzalez, X. Vera, and M. O'Boyle. IATAC: A Smart Predictor to Turn-Off L2 Cache Lines. ACM Trans. on Architecture and Code Optimization, 2005. DOI: 10.1145/1061267.1061271

[124] S. Kaxiras, Z. Hu, and M. Martonosi. Cache Decay: Exploiting Generational Behavior to Reduce Cache Leakage Power. In Proceedings of ISCA, 2001. DOI: 10.1109/ISCA.2001.937453

[125] Z. Hu, M. Martonosi, and S. Kaxiras. TCP: Tag Correlating Prefetchers. In Proceedings of HPCA, 2003. DOI: 10.1109/HPCA.2003.1183549

[126] K. Nesbit and J. E. Smith. Data Cache Prefetching Using a Global History Buffer. In Proceedings

HPCA, 2004. DOI：10.1109/HPCA.2004.10030

[127] R. Manikantan and R. Govindarajan. Focused Prefetching: Performance Oriented Prefetching Based on Commit Stalls. In Proceedings of ICS, 2008. DOI：10.1145/1375527.1375576

[128] H. Zhu, Y. Chen, and X. Sun. Timing Local Streams: Improving Timeliness in Data Prefetching. In Proceedings of ICS, 2010. DOI：10.1145/1810085.1810110

[129] M. Grannaes, M. Jahre, and L. Natvig. Multi-Level Hardware Prefetching using Low Complexity Delta Correlating Prediction Tables with Partial Matching. In Proceedings of HiPEAC, 2010. DOI：10.1007/978-3-642-11515-8_19

[130] V. Srinivasan, E. Davidson, and G. Tyson. A Prefetch Taxonomy. IEEE Trans. on Computers, February 2004. DOI：10.1109/TC.2004.1261824

[131] L. Spracklen, Y. Chou, and S. Abraham. Effective Instruction Prefetching in Chip Multi-processors for Modern Commercial Applications. In Proceedings of HPCA, 2005. DOI：10.1109/HPCA.2005.13

[132] T. Wenisch, S. Somogyi, N. Hardavellas, J. Kim, A. Ailamaki, and B. Falsafi. Temporal Streaming of Shared Memory. In Proceedings of ISCA, 2005. DOI：10.1145/1080695.1069989

[133] T. Wenisch, M. Ferdman, A. Ailamaki, B. Falsafi, and A. Moshovos. Practical Off-Chip Meta-Data for Temporal Memory Streaming. In Proceedings of HPCA, 2009. DOI：10.1109/HPCA.2009.4798239

[134] M. Ferdman, T. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos. Temporal Instruction Fetch Streaming. In Proceedings of MICRO, 2008. DOI：10.1109/MICRO.2008.4771774

[135] S. Somogyi, T. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos. Spatial Memory Streaming. In Proceedings of ISCA, 2006. DOI：10.1145/1150019.1136508

[136] S. Somogyi, T. Wenisch, A. Ailamaki, and B. Falsafi. Spatio-Temporal Memory Streaming. In Proceedings of ISCA, 2009. DOI：10.1145/1555754.1555766

[137] S. Srinath, O. Mutlu, H. Kim, and Y. Patt. Feedback Directed Prefetching: Improving the Performance and Bandwidth-Efficiency of Hardware Prefetchers. In Proceedings of HPCA, 2007. DOI：10.1109/HPCA.2007.346185

[138] J.M. Tendler, S. Dodson, S. Fields, H. Le, and B. Sinharoy. Power4 System Microarchitecture. Technical report, Technical White Paper, IBM, October 2001. DOI：10.1147/rd.461.0005

[139] I. Hur and C. Lin. Memory Prefetching Using Adaptive Stream Detection. In Proceedings of MICRO, 2006. DOI：10.1109/MICRO.2006.32

[140] I. Hur and C. Lin. Feedback Mechanisms for Improving Probabilistic Memory Prefetching. In Proceedings of HPCA, 2009. DOI：10.1109/HPCA.2009.4798282

[141] E. Ebrahimi, O. Mutlu, C. Lee, and Y. Patt. Coordinated Control of Multiple Prefetchers in Multi-Core Systems. In Proceedings of MICRO, 2009. DOI：10.1145/1669112.1669154

[142] S. Khan, Y. Tian, and D. Jimenez. Dead Block Replacement and Bypass with a Sampling Predictor. In Proceedings of MICRO, 2010.

[143] C. Chen, S. Yang, B. Falsafi, and A. Moshovos. Accurate and Complexity-Effective Spatial Pattern

Prediction. In Proceedings of HPCA, 2004. DOI: 10.1109/HPCA.2004.10010

[144] T. Johnson. Run-time Adaptive Cache Management. PhD thesis, University of Illinois, May 1998.

[145] S. Kumar and C. Wilkerson. Exploiting Spatial Locality in Data Caches Using Spatial Footprints. In Proceedings of ISCA, 1998. DOI: 10.1109/ISCA.1998.694794

[146] A. Seznec. Decoupled Sectored Caches: Conciliating Low Tag Implementation Cost. In Proceedings of ISCA, 1994. DOI: 10.1109/ISCA.1994.288133

[147] P. Pujara and A. Aggarwal. Increasing the Cache Efficiency by Eliminating Noise. In Proceedings of HPCA, 2006. DOI: 10.1109/HPCA.2006.1598121

[148] P. Pujara and A. Aggarwal. Increasing Cache Capacity through Word Filtering. In Proceedings of ICS, 2007. DOI: 10.1145/1274971.1275002

[149] M. Ekman and P. Stenstrom. A Robust Main-Memory Compression Scheme. In Proceedings of ISCA, 2005. DOI: 10.1145/1080695.1069978

[150] B. Abali, H. Franke, S. Xiaowei, D. Poff, and T. Smith. Performance of Hardware Compressed Main Memory. In Proceedings of HPCA, 2001. DOI: 10.1109/HPCA.2001.903253

[151] R. Tremaine, P. Franaszek, J. Robinson, C. Schulz, T. Smith, M. Wazlowski, and P. Bland. IBM Memory Expansion Technology (MXT). IBM Journal of Research and Development, 45(2), 2001. DOI: 10.1147/rd.452.0271

[152] J. Yang, Y. Zhang, and R. Gupta. Frequent Value Compression in Data Caches. In Proceedings of MICRO-33, pages 258–265, December 2000. DOI: 10.1145/360128.360154

[153] Y. Zhang, J. Yang, and R. Gupta. Frequent Value Locality and Value-Centric Data Cache Design. In Proceedings of ASPLOS, 2000. DOI: 10.1090/S0002-9939-99-05318-6

[154] M. Qureshi, M. Suleman, and Y. Patt. Line Distillation: Increasing Cache Capacity by Filtering Unused Words in Cache Lines. In Proceedings of HPCA, 2007. DOI: 10.1109/HPCA.2007.346202

[155] Alaa Alameldeen and David Wood. Adaptive Cache Compression for High-Performance Processors. In Proceedings of ISCA, 2004. DOI: 10.1145/1028176.1006719

[156] J. Lee, W. Hong, and S. Kim. Design and Evaluation of a Selective Compressed Memory System. In Proceedings of ICCD, 1999. DOI: 10.1109/ICCD.1999.808424

[157] E. Hallnor and S. Reinhardt. A Unified Compressed Memory Hierarchy. In Proceedings of HPCA, 2005. DOI: 10.1109/HPCA.2005.4

[158] E. Hallnor and S. Reinhardt. A Fully Associative Software-Managed Cache Design. In Proceedings of ISCA, 2000. DOI: 10.1145/339647.339660

[159] X. Jiang, N. Madan, L. Zhao, M. Upton, R. Iyer, S. Makineni, D. Newell, Y. Solihin, and R. Balasubramonian. CHOP: Adaptive Filter-Based DRAM Caching for CMP Server Platforms. In Proceedings of HPCA, 2010.

[160] R. Ho, K.W. Mai, and M.A. Horowitz. The Future of Wires. Proceedings of the IEEE, Vol.89, No.4, April 2001. DOI: 10.1109/5.920580

[161] P. Kongetira, K. Aingaran, and K. Olukotun. Niagara: A 32-Way Multithreaded Sparc Processor.

IEEE Micro, 25(2), 2005. DOI：10.1109/MM.2005.35

[162] R. Kumar, V. Zyuban, and D. Tullsen. Interconnections in Multi-Core Architectures: Understanding Mechanisms, Overheads, and Scaling. In Proceedings of ISCA, 2005. DOI：10.1109/ISCA.2005.34

[163] W.J. Dally and B. Towles. Principles and Practices of Interconnection Networks. Morgan Kaufmann, 1st edition, 2003.

[164] W. Dally and B. Towles. Route Packets, Not Wires: On-Chip Interconnection Networks. In Proceedings of DAC, 2001. DOI：10.1109/DAC.2001.156225

[165] P. Kundu. On-Die Interconnects for Next Generation CMPs. In Workshop on On- and Off-Chip Interconnection Networks for Multicore Systems (OCIN), 2006.

[166] Hang-Sheng Wang, Xinping Zhu, Li-Shiuan Peh, and Sharad Malik. Orion: A Power-Performance Simulator for Interconnection Networks. In Proceedings of MICRO-35, November 2002. DOI：10.1109/MICRO.2002.1176258

[167] H-S. Wang, L-S. Peh, and S. Malik. Power-Driven Design of Router Microarchitectures in On-Chip Networks. In Proceedings of MICRO, 2003. DOI：10.1109/MICRO.2003.1253187

[168] H.-S. Wang, L.-S. Peh, and S. Malik. A Power Model for Routers: Modeling Alpha 21364 and InfiniBand Routers. In IEEE Micro, Vol 24, No 1, January 2003. DOI：10.1109/MM.2003.1179895

[169] T. Moscibroda and O. Mutlu. A Case for Bufferless Routing in On-Chip Networks. In Proceedings of ISCA, 2009. DOI：10.1145/1555754.1555781

[170] R. Mullins, A. West, and S. Moore. Low-Latency Virtual-Channel Routers for On-Chip Networks. In Proceedings of ISCA, 2004. DOI：10.1109/ISCA.2004.1310774

[171] J. Howard et al. A 48-Core IA-32 Message-Passing Processor with DVFS in 45nm CMOS. In Proceedings of ISSCC, 2010. DOI：10.1109/JSSC.2010.2079450

[172] C. Nicopoulos, D. Park, J. Kim, V. Narayanan, M. Yousif, and C. Das. ViChaR: A Dynamic Virtual Channel Regulator for Network-on-Chip Routers. In Proceedings of MICRO, 2006. DOI：10.1109/MICRO.2006.50

[173] J. Kim, C. Nicopoulos, D. Park, V. Narayanan, M. Yousif, and C. Das. A Gracefully Degrading and Energy-Efficient Modular Router Architecture for On-Chip Networks. In Proceedings of ISCA, 2006. DOI：10.1109/ISCA.2006.6

[174] R. Ho. On-Chip Wires: Scaling and Efficiency. PhD thesis, Stanford University, August 2003.

[175] W. J. Dally. Express Cubes: Improving the Performance of k-ary n-cube Interconnection Networks. IEEE Transactions on Computers, 40(9), 1991. DOI：10.1109/12.83652

[176] A. Kumar, L. Peh, P. Kundu, and N. Jha. Express Virtual Channels: Towards the Ideal Interconnection Fabric. In Proceedings of ISCA, 2007. DOI：10.1145/1273440.1250681

[177] J. Balfour and W.J. Dally. Design Tradeoffs for Tiled CMP On-Chip Networks. In Proceedings of ICS, 2006. DOI：10.1145/1183401.1183430

[178] N. Muralimanohar, R. Balasubramonian, and N. Jouppi. Architecting Efficient Interconnects for

Large Caches with CACTI 6.0. IEEE Micro (Special Issue on Top Picks from Architecture Conferences), Jan/Feb 2008. DOI: 10.1109/MM.2008.2

[179] R. Ho, K. Mai, and M. Horowitz. Efficient On-Chip Global Interconnects. In Proceedings of VLSI, 2003. DOI: 10.1023/A:1023331803664

[180] A. Udipi, N. Muralimanohar, and R. Balasubramonian. Non-Uniform Power Access in Large Caches with Low-Swing Wires. In Proceedings of HiPC, 2009. DOI: 10.1109/HIPC.2009.5433222 DOI: 10.1109/HIPC.2009.5433222

[181] M. Minzuno, K. Anjo, Y. Sume, M. Fukaishi, H. Wakabayashi, T. Mogami, T. Horiuchi, and M. Yamashina. Clock Distribution Networks with On-Chip Transmission Lines. In Proceedings of the IEEE International Interconnect Technology Conference, pages 3–5, 2000.

[182] J.D. Warnock, J.M. Keaty, J. Petrovick, J.G. Clabes, C.J. Kircher, B.L. Krauter, P.J. Restle, B.A. Zoric, and C.J. Anderson. The Circuit and Physical Design of the POWER4 Microprocessor. IBM Journal of Research and Development, 46(1):27–51, January 2002. DOI: 10.1147/rd.461.0027

[183] T. Xanthopoulos, D.W. Bailey, A.K. Gangwar, M.K. Gowan, A.K. Jain, and B.K. Prewitt. The Design and Analysis of the Clock Distribution Network for a 1.2GHz Alpha Microprocessor. In Proceedings of the IEEE International Solid-State Circuits Conference, pages 402–403, 2001. DOI: 10.1109/ISSCC.2001.912693

[184] R.T. Chang, N. Talwalkar, C.P. Yue, and S.S. Wong. Near Speed-of-Light Signaling Over On-Chip Electrical Interconnects. IEEE Journal of Solid-State Circuits, 38(5):834–838, May 2003. DOI: 10.1109/JSSC.2003.810060

[185] A. Deutsch. Electrical Characteristics of Interconnections for High-Performance Systems. Proceedings of the IEEE, 86(2):315–355, February 1998. DOI: 10.1109/5.659489

[186] B.M. Beckmann and D.A. Wood. TLC: Transmission Line Caches. In Proceedings of MICRO-36, December 2003. DOI: 10.1109/MICRO.2003.1253182

[187] R. Manevich, I. Walter, I. Cidon, and A. Kolodny. Best of Both Worlds: A Bus-Enhanced NoC (BENoC). In Proceedings of NOCS, 2009. DOI: 10.1109/NOCS.2009.5071465

[188] L. Cheng, N. Muralimanohar, K. Ramani, R. Balasubramonian, and J. Carter. Interconnect-Aware Coherence Protocols for Chip Multiprocessors. In Proceedings of 33rd International Symposium on Computer Architecture (ISCA-33), pages 339–350, June 2006. DOI: 10.1109/ISCA.2006.23

[189] E. Bolotin, Z. Guz, I. Cidon, R. Ginosar, and A. Kolodny. The Power of Priority: NoC based Distributed Cache Coherency. In Proceedings of NOCS, 2007. DOI: 10.1109/NOCS.2007.42

[190] N. Eisley, L-S. Peh, and L. Shang. In-Network Cache Coherence. In Proceedings of MICRO, 2006. DOI: 10.1109/MICRO.2006.27

[191] N. Eisley, L-S. Peh, and L. Shang. Leveraging On-Chip Networks for Cache Migration in Chip Multiprocessors. In Proceedings of PACT, 2008. DOI: 10.1145/1454115.1454144

[192] N. Agarwal, L-S. Peh, and N. Jha. In-Network Snoop Ordering: Snoopy Coherence on Unordered Interconnects. In Proceedings of HPCA, 2009. DOI: 10.1109/HPCA.2009.4798238

[193] N. Agarwal, L-S. Peh, and N. Jha. In-Network Coherence Filtering: Snoopy Coherence without Broadcasts. In Proceedings of MICRO, 2009. DOI: 10.1145/1669112.1669143

[194] R. Das, S. Eachempati, A. K. Mishra, N. Vijaykrishnan, and C. R. Das. Design and Evaluation of Hierarchical On-Chip Network Topologies for Next Generation CMPs. In Proceedings of HPCA, 2009.

[195] S. Borkar. Designing Reliable Systems from Unreliable Components: The Challenges of Transistor Variability and Degradation. IEEE Micro, Nov/Dec 2005. DOI：10.1109/MM.2005.110

[196] Y. B. Kim et al. Low Power 8T SRAM Using 32nm Independent Gate FinFET Technology. In SOC Conference, 2008. DOI：10.1109/SOCC.2008.4641521

[197] S. M. Jahinuzzaman et al. A Soft Error Tolerant 10T SRAM Bit-Cell With DIfferential Read Capability. In IEEE Transactions on Nuclear Science, 2009. DOI：10.1109/TNS.2009.2032090

[198] S. Borkar, T. Karnik, S. Narendra, J. Tschanz, A. Keshavarzi, and V. De. Parameter Variations and Impact on Circuits and Microarchitecture. In Proceedings of DAC, 2003. DOI：10.1109/DAC.2003.1219020

[199] E. Humenay, D. Tarjan, and K. Skadron.

[200] ITRS. International Technology Roadmap for Semiconductors, 2009 Edition.

[201] S. Ozdemir, D. Sinha, G. Memik, J. Adams, and H. Zhou. Yield-Aware Cache Architectures. In Proceedings of MICRO, 2006. DOI：10.1109/MICRO.2006.52

[202] D.H. Albonesi。选择性缓存路径：按需缓存资源分配。1999. DOI：10.1109/MICRO.1999.809463

[203] A. Das, B. Ozisikylmaz, S. Ozdemir, G. Memik, J. Zambreno, and A. Choudhary. Evaluating the Effects of Cache Redundancy on Profit. In Proceedings of MICRO, 2008. DOI：10.1109/MICRO.2008.4771807

[204] R. C. Bose 和 D. K。Ray-Chaudhuri。关于一类纠错二进制群码。载于《信息与控制》，1960. DOI：10.1016/S0019-9958(60)90870-6

[205] J. Kim, N. Hardavellas, K. Mai, B. Falsafi, and J. C. Hoe.

[206] D. H. Yoon 和 M。Erez。内存映射 ECC：末级缓存低成本错误保护。载于 ISCA 会议录，2009. DOI：10.1145/1555815.1555771

[207] Y. Xie, G. Loh, B. Black, and K. Bernstein. Design Space Exploration for 3D Architectures. ACM Journal of Emerging Technologies in Computing Systems, 2(2):65–103, April 2006. DOI：10.1145/1148015.1148016

[208] T. Kgil，S. D'Souza，A。Saidi，N。Binkert，R. Dreslinski，S. Reinhardt，K。Flautner，和 T. Mudge。PicoServer：利用 3D 堆叠技术实现紧凑节能的片上多处理器。载于 ASPLOS 会议录，2006. DOI：10.1145/1168919.1168873

[209] F. Li, C. Nicopoulos, T. Richardson, Y. Xie, N. Vijaykrishnan, and M. Kandemir. Design and Management of 3D Chip Multiprocessors Using Network-in-Memory. In Proceedings of ISCA-33, June 2006. DOI：10.1109/ISCA.2006.18

[210] C. C. Liu，I。Ganusov，M。Burtscher，和 S. Tiwari。用 3D IC 技术弥合处理器-内存性能差距。IEEE 计算机设计与测试，22：556–564，2005 年 11 月。DOI：10.1109/MDT.2005.134

[211] G。Loh。面向多核处理器的 3D 堆叠内存架构。载于 ISCA 会议录，2008. DOI：10.1109/

ISCA.2008.15

[212] G. Loi, B. Agrawal, N. Srivastava, S. Lin, T. Sherwood, and K. Banerjee. A Thermally-Aware Performance Analysis of Vertically Integrated (3-D) Processor-Memory Hierarchy. In Proceedings of DAC-43, June 2006. DOI: 10.1109/DAC.2006.229426

[213] X. Dong, N. Muralimanohar, N. Jouppi, R. Kaufmann, and Y. Xie. Leveraging 3D PCRAM Technologies to Reduce Checkpoint Overhead in Future Exascale Systems. In Proceedings of SC, 2009. DOI: 10.1145/1654059.1654117

[214] X. Wu, J. Li, L. Zhang, E. Speight, R. Rajamony, and Y. Xie. Hybrid Cache Architecture with Disparate Memory Technologies. In Proceedings of ISCA, 2009. DOI: 10.1145/1555754.1555761 DOI: 10.1145/1555754.1555761

[215] G. Sun, X. Dong, Y. Xie, J. Li, and Y. Chen. A Novel Architecture of the 3D Stacked MRAM L2 Cache for CMPs. In Proceedings of HPCA, 2009. DOI: 10.1109/HPCA.2009.4798259

[216] B. Zhao, Y. Du, Y. Zhang, and J. Yang. Variation-Tolerant Non-Uniform 3D Cache Management in Die Stacked Multicore Processor. In Proceedings of MICRO, 2009. DOI: 10.1145/1669112.1669141

[217] Y. Xu, Y. Du, B. Zhao, X. Zhou, Y. Zhang, and J. Yang. A Low-Radix and Low-Diameter 3D Interconnection Network Design. In Proceedings of HPCA, 2009. DOI: 10.1109/HPCA.2009.4798234 DOI: 10.1109/HPCA.2009.4798234

[218] K. Puttaswamy and G. Loh. Implementing Caches in a 3D Technology for High Performance Processors. In Proceedings of ICCD, October 2005. DOI: 10.1109/ICCD.2005.65

[219] J. Poulton, R. Palmer, A. M. Fuller, T. Greer, J. Eyles, W. J. Dally, and M. Horowitz. A 14mW 6.25-Gb/s Transceiver in 90nm CMOS. In IEEE Journal of Solid State Circuits, 2009. DOI: 10.1109/JSSC.2007.908692

[220] IBM. IBM Unveils New POWER7 Systems To Manage Increasingly Data-Intensive Services. http://www-03.ibm.com/press/us/en/pressrelease/29315.wss.111,113

[221] W. K. Luk and R. H. Denard. A Novel Dynamic Memory Cell with Internal Voltage Gain. In IEEE Journal of Solid State Circuits, 2005. DOI: 10.1109/JSSC.2004.842854

[222] X. Liang, R. Canal, G. Wei, and D. Brooks. Process Variation Tolerant 3T1D-Based Cache Architectures. In Proceedings of MICRO, 2007. DOI: 10.1109/MICRO.2007.40

[223] C. Wilkerson, A.R. Alameldeen, Z. Chishti, W. Wu, D. Somasekhar, and S.-L Lu. Reducing Cache Power with Low-Cost, Multi-Bit Error Correcting Codes. In Proceedings of ISCA, 2010. DOI: 10.1145/1816038.1815973

[224] X. Dong, Y. Xie, N. Muralimanohar, and N. Jouppi. Simple but Effective Heterogeneous Main Memory with On-Chip Memory Controller Support. In Proceedings of SC, 2010. DOI: 10.1109/SC.2010.50

[225] J. Stuecheli, D. Kaseridis, D. Daly, H. Hunter, and L. John. The Virtual Write Queue: Coordinating DRAM and Last-Level Cache Policies. In Proceedings of ISCA, 2010. DOI: 10.1145/1816038.1815972

[226] S. Liu, B. Leung, A. Neckar, S. Memik, G. Memik, and N. Hardavellas. Hardware/Software Tech-

niques for DRAM Thermal Management. In Proceedings of HPCA, 2011. DOI：10.1109/HPCA.2011.5749756

作者简介

Rajeev Balasubramonian

Rajeev Balasubramonian 是犹他大学计算学院的副教授。他于 1998 年获得印度理工学院孟买分校的计算机科学与工程学士学位，并于 2000 年和 2003 年获得罗切斯特大学的硕士和博士学位。他的主要研究领域包括内存层次结构和片上互连。Balasubramonian 教授是 NSF CAREER 奖和计算学院教学奖的获得者。他参与合著的论文曾入选 IEEE Micro Top Picks（2007 年和 2010 年），并获得最佳论文奖（HiPC'09 和 PACT'10）。

Norman P. Jouppi

Norman P. Jouppi 是惠普实验室的高级研究员兼智能基础设施实验室主任。他因在计算机内存系统方面的创新而闻名，包括流预取缓冲区、受害者缓存、多级独占缓存以及开发用于建模缓存时序、面积和功耗的 CACTI 工具。他还曾担任多个微处理器的主要架构师和首席设计师，为图形加速器的架构和设计做出贡献，并广泛研究了视频、音频和物理远程呈现。Jouppi 于 1984 年获得斯坦福大学电气工程博士学位，期间他是 MIPS 微处理器的主要架构师和设计师之一，也是 CMOS VLSI 时序验证技术的开发者。他目前担任 ACM SIGARCH 的前任主席，并且是计算研究协会（CRA）董事会成员。他是《ACM 通讯》和 IEEE Micro 的编委会成员。他是 ACM 和 IEEE 的会士，拥有超过 50 项美国专利。他已发表超过 100 篇技术论文，获得多项最佳论文奖和一次计算机体系结构研讨会（ISCA）影响力论文奖。

Naveen Muralimanohar

Naveen Muralimanohar 是惠普实验室智能基础设施实验室的高级研究员。他的研究重点是为高性能系统设计可靠且高效的内存层次结构和通信网络。他发表了多篇关于片上缓存的有影响力的论文，包括凭借在大型缓存模型与 CACTI 方面的工作获得的最佳论文奖和 IEEE Micro Top Pick。他拥有犹他大学计算机科学博士学位和马德拉斯大学电气工程学士学位。